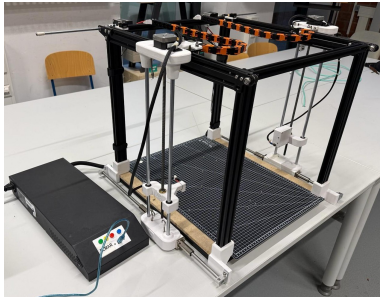




Dokumentation

ELMAR Heißdrahtschneider

Autoren: Alexandra Roth 7025637
David Atrops 7013979
Dennis Smirnow 7023434
Leon Ronge 7026641
Samuel Drees 7026354

Studiengang: Maschinenbau und Design

Erstprüfer: Prof. Dr. Elmar Wings
Malena Wilken

Abgabedatum: 23. Juni 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	3
Abbildungsverzeichnis	11
1. Einleitung	13
1.1. Das Projekt	13
1.2. Problemstellung	13
1.3. Stand der Technik	13
1.4. Besondere Herausforderungen	14
1.5. Lösungsansatz	14
I. Software	17
2. Arduino IDE 2.3.6	19
2.1. Arduino IDE Beschreibung	19
2.2. Installation der Arduino IDE	20
2.2.1. Download oder Arduino IDE	20
2.2.2. Anforderungen Betriebssystem	21
2.2.3. Installation auf Windows	21
2.2.4. Installation (MacOS)	24
2.3. Übersicht der Eigenschaften	25
2.3.1. Sketchbook	26
2.3.2. Boards Manager	27
2.3.3. Library Manager	27
2.4. Integration und Verwendung einer benutzerdefinierten Bibliothek in der Arduino IDE	28
2.4.1. Erstellen der Bibliotheksdateien	28
2.4.2. Installation und Integration der Arduino IDE	28
2.4.3. Verwendung symbolischer Verknüpfungen zur Bibliotheksinte- gration	29
2.4.4. Windows-Tool mklink zum Erstellen symbolischer Verknüpfungen	29
2.4.5. Überprüfen der Verfügbarkeit der Bibliothek in der Arduino IDE	30
2.4.6. Serieller Monitor	31
2.4.7. Serieller Plotter	31
2.5. Beispiele	32
2.6. Debuggen	32
2.7. Autovervollständigung	33
2.8. Bibliotheken und Modularität	34
3. Einrichtung des Arduino UNO	35
3.1. Installation der Pakete für den Arduino UNO	35
3.2. Verbindung und Auswahl des Boards	35
3.3. Auswahl des Ports	36
3.4. Hochladen und Überprüfen eines Sketches	36
3.5. Test der Konfiguration anhand des Beispiels Blink	36

4. Arduino Web Editor	39
4.1. Using the Arduino Web Editor	39
4.1.1. Using the online IDE	39
4.2. Arduino Libraries	40
4.3. Data Security	41
4.3.1. Authentication and Authorization	41
4.3.2. Secure Code Execution	41
4.3.3. Protection Against Malware	41
4.3.4. Privacy Policies	41
4.3.5. Regular Updates and Maintenance	42
4.3.6. User Awareness	42
5. Universal G-Code Sender	43
5.1. Einleitung	43
5.2. Beschreibung des Universal G-Code Senders	43
5.3. Aufbau der Benutzeroberfläche	43
5.3.1. Jog Control	44
5.3.2. Visualizer	44
5.3.3. Console	45
5.3.4. Controller State / DRO	45
5.3.5. Zusammenspiel der Bedienbereiche	46
5.4. Installation des Universal G-Code Senders	46
5.4.1. Systemvoraussetzungen	46
5.4.2. Download und Installation	47
5.4.3. Erstverbindung mit der CNC-Maschine	47
5.5. Konfiguration und Diagnose in UGS	47
5.6. Verwendung	48
5.6.1. G-Code selbst schreiben	48
5.6.2. G-Code-Datei importieren	48
6. GRBL v1.1	51
6.1. Beschreibung der GRBL-Firmware	51
6.2. Version	51
6.3. Installation auf dem Arduino Uno	52
6.3.1. Voraussetzungen	52
6.3.2. Schritt-für-Schritt-Installation	52
6.3.3. Hardware-Anschlüsse – Pin-Belegung	53
6.4. Konfiguration	53
6.4.1. Laufzeitkonfiguration über GRBL-Variablen	53
6.4.2. GRBL-Variablen Referenztafel	54
6.4.3. Diagnose und Fehlerbehebung	55
II. Hardware	57
7. Arduino Uno R3	59
7.1. Arduino Allgemein	59
7.2. Arduino Uno R3	59
7.3. Module	60
7.3.1. Mikrocontroller ATmega328P	60
7.3.2. Eingebaute LED	60
7.3.3. Reset-Taster	60
7.3.4. USB-Schnittstelle	60
7.3.5. Spannungsversorgung	61

7.4. Anschlüsse des Arduino Uno	61
7.4.1. Digitale Ein- und Ausgangspins	61
7.4.2. PWM-Ausgangspins	61
7.4.3. Analoge Eingänge	63
7.4.4. Versorgungsanschlüsse	63
7.4.5. Serielle Ein- und Ausgangspins D0 und D1	63
7.4.6. I2C-Kommunikationsanschlüsse	63
7.4.7. SPI-Kommunikationsanschlüsse	64
7.4.8. AREF-Referenzeingang	65
7.4.9. RESET-Eingang	65
7.5. Beispiel Code	66
7.5.1. Manual	66
7.5.2. Code	66
7.6. Anwendung	66
7.6.1. Manual	66
7.6.2. Code	67
7.7. Testcode	68
7.7.1. Manual	68
7.7.2. Code	69
8. CNC Shield V3	71
8.1. CNC Shields / CNC-Controllerboards	71
8.2. Einleitung	71
8.3. Grundlagen	71
8.4. Spezifisches CNC Shield V3	72
8.5. Spezifikation	72
8.6. Bibliothek und Steuerung	73
8.6.1. Daten und Signale	73
8.7. Installation und Montage	74
8.8. Beispiel	74
8.8.1. Anleitung	74
8.8.2. Erklärung des Beispiel-Codes	75
8.8.3. Code	75
8.9. Kalibrierung	76
8.10. Einfache Anwendung	76
8.11. Tests	78
8.11.1. Einfacher Funktionstest	78
8.11.2. Test aller Funktionen	78
8.12. Dateien	79
8.13. Weiterführende Literatur	79
9. Sensor: Endschalter	81
9.1. Einleitung: Positionssensoren	81
9.2. Definition und Funktion: Elektromechanischer Endschalter	81
9.3. Specification	81
9.4. Verwendung und Schaltfunktionen	82
9.5. Analyse: Vor- und Nachteile	83
9.6. Ausblick: Intelligente und vernetzte Sensorik	84
9.7. Library	84
9.8. Beispiel	84
9.8.1. Manual	84
9.8.2. Erklärung des Beispiel-Codes	84
9.9. Justage	85
9.10. Tests	85
9.10.1. Einfacher Funktions-Test	85

9.10.2. Manual	86
9.10.3. Einfacher Test-Code	86
9.11. Einfache Anwendung	86
9.11.1. Manual	87
9.12. Weiterführende Literatur	88
10. Druckknopf	89
10.1. General	89
10.2. Joy IT	89
10.3. Spezifikationen	89
10.4. Beispiel Code	89
10.4.1. Manual	89
10.4.2. Code	89
10.5. Einfacher Testcode	90
10.5.1. Manual	90
10.5.2. Code	91
10.6. Einfache Anwendung	91
10.6.1. Manual	91
10.6.2. Code	92
11. MOSFET PWM-Schaltmodul	95
11.1. Einleitung	95
11.2. Grundlegende Informationen	95
11.3. Spezifisches MOSFET-Modul	95
11.4. Spezifikation	95
11.4.1. Eigenschaften des IRF9540N	95
11.4.2. Anschlüsse	96
11.5. Bibliothek	96
11.5.1. Beschreibung	96
11.5.2. Funktionen	96
11.6. Beispiel	97
11.6.1. Versuchsaufbau	97
11.6.2. Funktionsweise des Beispiels	97
11.6.3. Einfacher Code	97
11.7. Kalibrierung	98
11.8. Einfache Anwendung	98
11.9. Tests	99
11.9.1. Einfacher Funktionstest	99
11.9.2. Test aller Funktionen	100
11.10 Weiterführende Literatur	101
12. A4988-Schrittmotortreiber	103
12.1. Einleitung	103
12.2. Grundlagen	103
12.3. Spezifischer Treiber	103
12.3.1. Installation	103
12.4. Spezifikation	104
12.5. Bibliothek	104
12.5.1. Beschreibung	104
12.5.2. Funktionen	104
12.6. Beispiel	104
12.6.1. Funktionsweise des Beispiels	105
12.6.2. Einfacher Code	105
12.7. Kalibrierung	106
12.8. Einfache Anwendung	106

12.9. Tests	108
12.9.1. Test aller Funktionen	108
12.10Weiterführende Literatur	110
13. Schrittmotor	111
13.1. Beschreibung eines Schrittmotors	111
13.1.1. Aufbau und Funktionsweise eines Schrittmotors	111
13.1.2. Schrittmotor Bauformen	112
13.1.3. Betriebsarten unipolar und bipolar	113
13.1.4. Mikroschrittverfahren	113
13.2. Schrittverluste verhindern	114
13.2.1. Auswahl des Schrittmotors	114
13.2.2. Betriebsart	114
13.2.3. Externe Ereignisse	116
13.3. Beschreibung des verwendeten Schrittmotors	117
13.4. Bibliotheken	117
13.4.1. AccelStepper	117
13.4.2. Wire	118
13.4.3. Adafruit GFX	118
13.4.4. Adafruit SSD1306	118
13.4.5. Encoder	119
13.5. Beschreibung der Anwendung	119
13.5.1. Aufgabe des Programms	119
13.5.2. Erklärung des Codes	121
13.5.3. Code	121
13.6. Einfacher Code	127
13.7. Test	127
14. DC-DC-Abwärtswandler	129
14.1. Eingesetzter Spannungswandler	129
14.2. Spezifikation	129
14.3. Funktionsprinzip	130
14.3.1. Wirkungsgrad	130
14.3.2. Verlustleistungen	130
14.4. Schutzvorrichtungen	130
14.4.1. Überstromschutz	131
14.4.2. Kurzschlusschutz	131
14.4.3. Thermischer Schutz	131
14.5. Montage	131
14.6. Weiterführende Literatur	131
15. Komplettes System	133
15.1. Beschreibung des Gesamtsystems	133
15.2. Design und Aufbau	134
15.3. Hardware-Schnittstellen	135
15.4. Eigenschaften des Gesamtsystems	137
15.5. Software-Schnittstellen	137
15.6. Installation des Gesamtsystems	138
15.7. Konfiguration	138
15.8. Daten, Strukturen und Typen	139
15.9. Einschränkungen und offene Punkte	140
15.10Abmaße	140
15.11Schlussfolgerung	141

16. Hardware	143
16.0.1. Grundrahmen	143
 III. Domain Knowledge - Tools	 147
17. Softwareseitiges Domänenwissen	149
17.1. Digitale Prozesskette	149
17.2. CNC-Steuerung und Bewegungsdaten	149
17.3. Koordinatensystem, Nullpunkt und Referenzfahrt	150
17.4. Schritt-Millimeter-Kalibrierung	150
17.5. Bewegungsalgorithmus	150
17.6. Vorschubgeschwindigkeit	151
17.7. Softwareseitige Fehlerquellen	151
 18. Hardwareseitiges Domänenwissen	 153
18.1. Fachliche Grundlagen des Heißdrahtschneidens	153
18.2. Werkstoff Schaumstoff	153
18.3. Thermischer Schneidprozess	153
18.4. Widerstandsdraht und Heizleistung	154
18.5. PWM-Ansteuerung des Heißdrahts	154
18.6. Drahtspannung und mechanische Führung	154
18.7. Achsen, Schrittmotoren und mechanische Belastung	155
18.8. Prozessparameter des Schnitts	155
18.9. Fehlerbilder und Ursachen	155
18.10. Bewertung der Schnittqualität	155
 IV. Nutzung	 157
19. Von der Idee zum Werkstück	159
19.1. Idee und Entwurf der Kontur	159
19.2. Erzeugung der Steuerbefehle (G-Code)	159
19.3. Einrichten und Referenzieren der Maschine	160
19.4. Der Schneidvorgang	160
19.5. Fertigstellung des Werkstücks	160
 20. Programmablaufplan	 161
 V. Erfahrungen	 163
21. Erfahrungen bei Aufbau, Verkabelung, Inbetriebnahme und Steuerung der CNC-Heißdrahtschneidanlage	165
21.1. Hinweis zur Erstellung dieses Erfahrungsberichts	165
21.2. Erfahrungen im Projektverlauf	165
21.3. Erfahrungen im Projektverlauf	166
 VI. Anhang	 173
22. Bill of Material	175
22.1. Mechanische Komponenten	175
22.2. Elektronische / Elektrische Komponenten	177

Inhaltsverzeichnis	9
Literaturverzeichnis	179
Literaturverzeichnis	179

Abbildungsverzeichnis

2.1. Arduino IDE 2.3.6	21
2.2. Arduino IDE Downloadoptionen	22
2.3. Lizenzbestätigung Installation	23
2.4. Wahl Zielverzeichnis	23
2.5. Startfenster Arduino-IDE	24
2.6. ArduinoIDE macOS	25
2.7. Überblick	26
2.8. Sketchbook	26
2.9. Boards Manager	27
2.10. Library Manager	28
2.11. Eingabeaufforderung mit Administrator Rechten	29
2.12. Eingabeaufforderung	30
2.13. Einbinden der Nano33BLESenseLED Bibliothek	31
2.14. Serieller Monitor	31
2.15. Beispiele	32
2.16. Beispiel Debuggen	33
2.17. Autovervollständigung	33
3.1. Installation der Arduino AVR Boards	35
3.2. Beispiel-Sketch Blink	37
4.1. Arduino Web Editor	39
4.2. finding-an-example	40
5.1. Jog Control – Bedienfeld in UGS	44
5.2. Visualizer – Grafikanzeigefeld in UGS	45
5.3. Console – Eingabefeld in UGS	45
5.4. Controller State (DRO) – Positionsanzeige in UGS	46
5.5. Übersicht über die Bedien- und Anzeigefelder in UGS	46
8.1. Anschlussübersicht des CNC Shield V3	72
8.2. CNC Shield V3 mit eingesetzten A4988-Schrittmotortreibern	74
9.1. Darstellung des im Projekt verwendeten Endschalters	82
9.2. Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO	83
13.1. Prinzipieller Aufbau Schrittmotor (2-phasig) [Hag21]	112
13.2. Start-Stopp Frequenz [Dr.20]	114
13.3. Trapezförmiges Geschwindigkeitsprofil [Dr.20]	115
13.4. Flussdiagramm des Programms	120
14.1. Blockschaltbild der Spannungsversorgung	129
15.1. Schematische Darstellung des CNC-Heißdrahtschneiders als Automatisierungssystem	134
15.2. Schaltplan des CNC-Heißdrahtschneiders	136

1. Einleitung

1.1. Das Projekt

Im Rahmen des Projekts wird ein Heißdrahtschneider entwickelt. Dabei handelt es sich um eine automatisierte Schneidvorrichtung zur Bearbeitung von Schaumstoffplatten. Der Schnitt erfolgt durch einen elektrisch erwärmten Widerstandsdraht. Der Draht wird entlang einer vorgegebenen Bahn durch das Material geführt und trennt den Schaumstoff infolge der lokal eingebrachten Wärme.

Der Aufbau verbindet mehrere technische Teilgebiete. Die Bewegungsführung gehört zur CNC-Technik, da die Achsen computergesteuert entlang definierter Koordinaten verfahren werden. Die Erwärmung des Schneiddrahts gehört zur Leistungselektronik, da eine elektrische Last geschaltet und in ihrer Leistung beeinflusst werden muss. Das Werkstoffverhalten des Schaumstoffs gehört zur Kunststofftechnik, da die Schnittqualität stark davon abhängt, wie das Material auf Wärme, Vorschubgeschwindigkeit und Schnittgeometrie reagiert [Bon25].

CNC-Systeme werden eingesetzt, um Bewegungen reproduzierbar und unabhängig von der manuellen Führung einer Bedienperson auszuführen. Für die Herstellung eines Werkstücks werden digitale Geometriedaten in Bewegungsdaten überführt. In der computergesteuerten Produktion bilden CAD/CAM-Prozessketten, CNC-Technik und Softwarewerkzeuge eine zusammenhängende Prozesskette von der Konstruktion bis zur Fertigung [Heh20].

1.2. Problemstellung

Die zentrale Problemstellung besteht darin, eine digitale Schnittkontur mit einem thermischen Schneidverfahren in ein reales Werkstück zu übertragen. Dafür müssen mechanische Achsbewegung, elektrische Heißdrahtansteuerung und Softwaresteuerung aufeinander abgestimmt werden. Eine reine Bewegung der Achsen reicht nicht aus, da der Schnitt nur dann sauber entsteht, wenn Heizleistung, Drahtspannung und Vorschubgeschwindigkeit zum verwendeten Schaumstoff passen.

Bei einem manuellen Heißdrahtschneider hängt die Schnittqualität wesentlich von der Handführung ab. Beim Heißdrahtschneider wird diese Aufgabe durch eine computergesteuerte Bewegung ersetzt. Dadurch können Konturen reproduzierbarer hergestellt werden. Gleichzeitig entstehen neue Anforderungen: Die Achsen müssen kalibriert sein, die Motoren dürfen keine Schritte verlieren, der Schneiddraht muss ausreichend gespannt sein und die Heizleistung muss so eingestellt werden, dass eine saubere Schnittfuge entsteht.

Die Herausforderung liegt also im Zusammenspiel des Gesamtsystems. Zusätzlich muss der Prozess dokumentiert werden, damit Aufbau, Inbetriebnahme, Wartung und Fehlersuche nachvollziehbar bleiben.

1.3. Stand der Technik

In der computergesteuerten Produktion werden digitale Daten zur Planung, Simulation und Durchführung von Fertigungsprozessen verwendet. Dazu gehören unter anderem CAD-Daten, CAM-Prozessketten, CNC-Techniken und Softwarewerkzeuge [Heh20]. CNC-Maschinen setzen Bewegungsinformationen in definierte Werkzeug- oder

Werkstückbewegungen um. Die Steuerung verarbeitet Koordinaten, Vorschubwerte und Bewegungsbefehle und erzeugt daraus Signale für die Achsantriebe.

Für kleine CNC-Systeme eignen sich Schrittmotoren, da sie eine Bewegung in diskreten Winkelschritten ausführen. Schrittmotoren gehören zu den Sonderbauformen elektrischer Maschinen und bewegen sich nicht kontinuierlich wie klassische rotierende Maschinen, sondern schrittweise [Sch17]. Dadurch eignen sie sich für einfache Positionierungen, solange die Last und die Dynamik so gewählt werden, dass keine Schritte verloren gehen.

Die elektrische Ansteuerung des Heißdrahts ist dem Bereich der Leistungselektronik zuzuordnen. Leistungselektronik beschreibt das Schalten, Steuern und Umformen elektrischer Energie mit elektronischen Mitteln. Für das Projekt ist dies relevant, da der Heißdraht nicht direkt über einen Mikrocontroller-Pin betrieben werden darf, sondern über eine geeignete Leistungsstufe geschaltet werden muss [Zac22].

1.4. Besondere Herausforderungen

Eine besondere Herausforderung besteht in der Abstimmung von Heizleistung und Vorschubgeschwindigkeit. Beim Heißdrahtschneiden entsteht die Schnittfuge durch Wärmeeintrag in den Schaumstoff. Wird der Draht zu kalt betrieben oder zu schnell bewegt, wird das Material nicht vollständig getrennt. Wird der Draht zu heiß betrieben oder zu langsam bewegt, kann die Schnittfuge zu breit werden. Untersuchungen zur thermischen Bearbeitung von EPS zeigen, dass die Schnittfuge und die Temperaturverteilung wesentliche Größen des Prozesses sind.

Eine weitere Herausforderung ist die Positionsgenauigkeit der Achsen. Die Steuerung geht im offenen Betrieb davon aus, dass jeder ausgegebene Schritt auch mechanisch umgesetzt wird. Bei Überlastung, zu hoher Beschleunigung oder ungünstiger Geschwindigkeit können jedoch Schritte verloren gehen. Das führt unmittelbar zu Maßabweichungen am Werkstück. Deshalb müssen Motortreiberstrom, Mikroschrittbetrieb, Geschwindigkeit und mechanische Last zusammen betrachtet werden [Sch17].

Zusätzlich ist die elektrische Heißdrahtansteuerung sicherheitsrelevant. Der Schneiddraht ist eine Last mit erhöhter Stromaufnahme und hoher Temperatur. Die Ansteuerung muss daher von der Logik des Mikrocontrollers getrennt betrachtet werden. Der Mikrocontroller liefert nur das Steuersignal, während die eigentliche Leistung über ein MOSFET-Schaltmodul geschaltet wird. [Zac22].

1.5. Lösungsansatz

Das Projekt wird modular aufgebaut. Die mechanische Baugruppe führt den Schneiddraht entlang der vorgesehenen Bahn. Die elektronische Baugruppe stellt die Steuerung und Leistungsversorgung bereit. Die Software erzeugt die Schritt- und Richtungssignale für die Achsantriebe. Durch diese Aufteilung können die einzelnen Funktionen getrennt aufgebaut, getestet und anschließend zum Gesamtsystem verbunden werden.

Die Achsbewegung wird mit Schrittmotoren realisiert. Die Schrittmotortreiber erhalten STEP- und DIR-Signale. Jeder STEP-Impuls erzeugt eine Bewegung um einen Schritt oder Mikroschritt. Das DIR-Signal legt die Bewegungsrichtung fest. Die Anzahl der Impulse bestimmt den Fahrweg, der zeitliche Abstand der Impulse beeinflusst die Geschwindigkeit. Die Kalibrierung erfolgt über die Zuordnung von Schritten zu Millimetern.

Die Heißdrahtansteuerung wird über ein MOSFET-PWM-Schaltmodul umgesetzt. Dadurch wird der Schneiddraht nicht direkt vom Mikrocontroller versorgt. Die PWM-Ansteuerung ermöglicht eine Veränderung der mittleren elektrischen Leistung. Die passende Einstellung wird nicht rein theoretisch festgelegt, sondern durch Probeschnitte

überprüft. Dabei werden Schnittkante, Schnittfugenbreite, Drahtverhalten und Maßhaltigkeit bewertet.

Teil I.

Software

2. Arduino IDE 2.3.6

Dieses Kapitel liefert eine detaillierte Einführung in die Arduino IDE, einschließlich ihrer zentralen Funktionen, verfügbaren Schnittstellen und grundlegenden Arbeitsweise. Es erläutert die einzelnen Bestandteile der Entwicklungsumgebung, beschreibt den Installationsprozess anhand klar strukturierter Schritte und bietet eine Anleitung zur Ersteinrichtung sowie zur grundlegenden Konfiguration der Programmierungsumgebung.

2.1. Arduino IDE Beschreibung

Die Arduino IDE ist die offizielle, Open-Source Entwicklungsumgebung zur Programmierung und Verwaltung von Arduino Mikrocontrollern. Sie ermöglicht das Schreiben, Kompilieren und Übertragen von Quellcode auf eine Vielzahl unterstützter Boards. Als plattformübergreifende Anwendung ist sie mit allen gängigen Betriebssystemen wie Windows, Linux und macOS kompatibel. Ursprünglich auf der Java-Plattform basierend, bietet die IDE Unterstützung für zahlreiche Arduino-Boards sowie eine auf C und C++ basierende Programmiersprache. Durch ihre intuitive Benutzeroberfläche richtet sie sich sowohl an Einsteiger als auch an erfahrene Entwickler im Bereich der Mikrocontroller-Programmierung [FAD18].

Die Mikrocontroller auf Arduino-Boards werden mithilfe von Programmen gesteuert, die in Form von Quellcode verfasst und über die Entwicklungsumgebung verarbeitet werden. In der Arduino IDE erstellte Programme werden als sogenannte „Sketche“ bezeichnet. Dabei handelt es sich um Skripte, die den Funktionsablauf des Mikrocontrollers definieren – etwa das Einlesen von Sensorwerten, die Ansteuerung von Aktoren oder die Kommunikation mit anderen Geräten. Nach dem Schreiben wird der Sketch von der IDE kompiliert und in eine maschinenlesbare Hex-Datei übersetzt. Diese Datei wird im Anschluss über die serielle Schnittstelle auf den Mikrocontroller hochgeladen, wo sie dauerhaft gespeichert und beim nächsten Start ausgeführt wird [FAD18].

Die Arduino-IDE setzt sich im Wesentlichen aus zwei funktionalen Kernkomponenten zusammen: dem Editor und dem Compiler. Der integrierte Editor dient als zentrale Arbeitsfläche zum Schreiben und Bearbeiten von Quellcode, während der Compiler für die Übersetzung des Codes in maschinenlesbare Anweisungen sowie dessen Übertragung auf das angeschlossene Arduino-Modul verantwortlich ist [FAD18].

In der Version 2.3.6 der Arduino IDE spielt die Menüleiste am oberen Rand der Benutzeroberfläche eine zentrale Rolle. Sie bietet direkten Zugriff auf essentielle Funktionen wie das Hochladen von Sketchen auf das Board, das Kompilieren zur Überprüfung auf Fehler sowie das Speichern von Projekten. Unter dem Menüpunkt „Datei“ lassen sich neue Sketche anlegen, bestehende Projekte öffnen oder speichern. Ein weiterer bedeutender Abschnitt ist „Beispiele“ – dort finden sich eine Vielzahl vorgefertigter Sketche, die typische Anwendungsszenarien demonstrieren, darunter klassische Programme wie „Blink“ zur LED-Steuerung oder „Fade“ zur Helligkeitssteuerung mittels Pulsweitenmodulation. Diese Beispiele erleichtern insbesondere Einsteigern den Zugang zur Mikrocontroller-Programmierung und dienen oft als Ausgangspunkt für eigene Entwicklungen [FAD18].

Die fünf Schaltflächen am oberen Bildschirmrand sind wie folgt:



-  Das Häkchen-Symbol wird verwendet, um den geschriebenen Code zu **Überprüfen**, nachdem der Code eingegeben wurde, wird der Code auf Syntaxfehler geprüft. Dieser Schritt stellt sicher, dass der Code für die Verwendung mit der Hardware bereit ist.
-  Das Symbol **Hochladen** in der Arduino IDE kompiliert den Code und lädt ihn auf den angeschlossenen Arduino hoch.
-  Das Symbol **Start Debugging** in der Arduino IDE startet eine Debugging Sitzung. Damit kann der Code analysiert werden, indem Haltepunkte gesetzt werden. Beim Debuggen wird die Ausführung schrittweise durchlaufen und Variablen auf Kompatibilität und Funktion geprüft, ohne das ganze Programm in einem Stück zu starten.
-  Das Symbol **Serial Plotter** in der Arduino IDE öffnet ein Tool, das Echtzeitdaten, die vom Board über die serielle Verbindung gesendet werden, grafisch darstellt. Dadurch wird die Analyse der Daten erleichtert.
-  Das Symbol **Serial Monitor** in der Arduino IDE öffnet ein Terminal, um textbasierte Daten über die serielle Verbindung anzuzeigen und zu senden. Dadurch wird eine Echtzeitkommunikation mit dem angeschlossenen Board ermöglicht.

2.2. Installation der Arduino IDE

2.2.1. Download oder Arduino IDE

Der Arduino Nano 33 BLE Sense wird über die Arduino Integrated Development Environment (IDE) programmiert, die als Standardumgebung für sämtliche Arduino-Boards etabliert ist. Sie zählt zu den am weitesten verbreiteten Entwicklungsplattformen im Bereich der Mikrocontroller-Programmierung. Die IDE ist sowohl als Offline-Variante zur lokalen Installation als auch als Online-Editor verfügbar. Als Open-Source-Software ist sie darauf ausgelegt, den Programmierprozess möglichst zugänglich zu gestalten – vom Schreiben des Codes bis hin zum Hochladen auf das jeweilige Board.

Für alle gängigen Betriebssysteme – einschließlich macOS, Linux und Windows – stehen passende Versionen zur Verfügung. Die Installation ist unkompliziert und wird durch die breite Unterstützung der Arduino-Community zusätzlich erleichtert. Neben der Desktop-Version bietet Arduino auch eine cloudbasierte Entwicklungsumgebung an: den Online-Arduino-Editor. Diese moderne Webanwendung ermöglicht das plattformunabhängige Programmieren, Speichern und Verwalten von Sketchen direkt im Browser. Sie enthält alle gängigen Bibliotheken und bietet sofortige Unterstützung für aktuelle Arduino-Boards, einschließlich des Nano 33 BLE Sense.

Zugriff auf die verschiedenen Softwarepakete erhalten Sie über die offizielle Website der Arduino-Entwickler: [Arduino-Software](https://www.arduino.cc/en/software). Dort finden sich stets die neuesten Versionen der IDE sowie weiterführende Informationen. Eine Übersicht der verfügbaren Downloadoptionen ist in Abbildung 2.1 dargestellt.

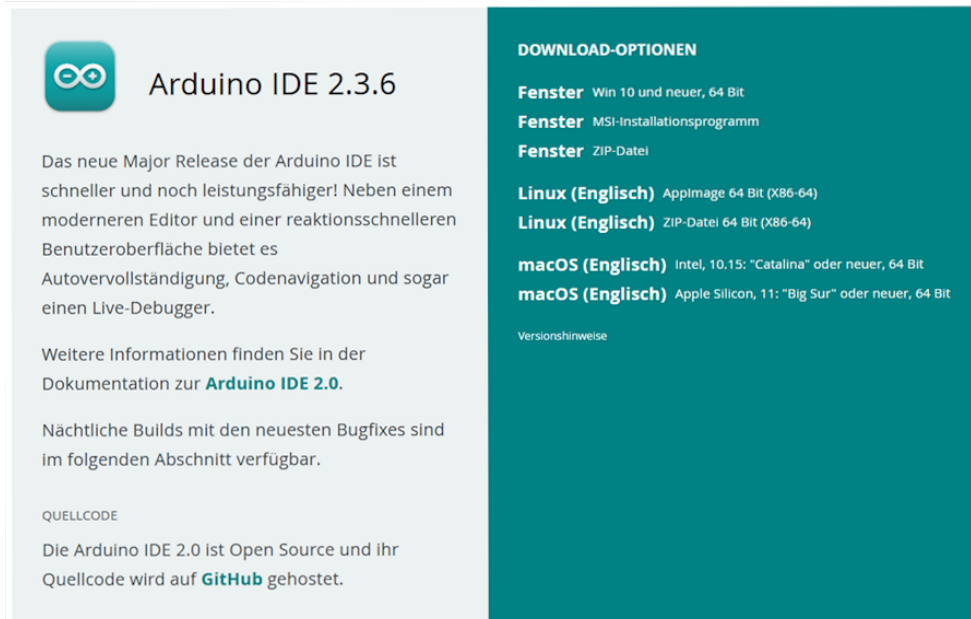


Abbildung 2.1.: Arduino IDE 2.3.6

2.2.2. Anforderungen Betriebssystem

Im Folgenden sind die Systemanforderungen und Varianten der verfügbaren Downloadoptionen für die Arduino-IDE aufgeführt. Diese sind jeweils auf unterschiedliche Betriebssysteme und Nutzerbedürfnisse zugeschnitten. Um einen reibungslosen Betrieb sowie den Zugriff auf alle aktuellen Funktionen und Sicherheitsupdates zu gewährleisten, sollte stets die zur eigenen Plattform passende Version der IDE ausgewählt werden.

- Windows– Win 10 und neuer, 64 Bit
- Linux– 64 Bit
- macOS– Intel, Version 10.15: „Catalina“ oder neuer, 64 Bit
- macOS– Apple Silicon, 11: „Big Sur“ oder neuer, 64 Bit

2.2.3. Installation auf Windows

Die Installation der Arduino IDE auf einem Windows-System wird im folgenden Abschnitt Schritt für Schritt erläutert. Im Anschluss daran folgt eine Einführung in zentrale Funktionen der Umgebung, darunter der Board-Manager, der Bibliotheks-Manager, das Sketchbook sowie weitere essentielle Werkzeuge der Benutzeroberfläche.

Um die Arduino IDE auf einem Windows-Computer zu installieren, laden Sie zunächst die entsprechende Installationsdatei von der offiziellen Website unter [Arduino-Software](#) herunter. Starten Sie anschließend die heruntergeladene Datei und folgen Sie den angezeigten Installationsanweisungen. Diese beinhalten in der Regel die Auswahl des Installationsverzeichnis sowie optionaler Komponenten wie Verknüpfungen. Sobald

der Vorgang abgeschlossen ist, steht die Entwicklungsumgebung zur sofortigen Nutzung bereit.

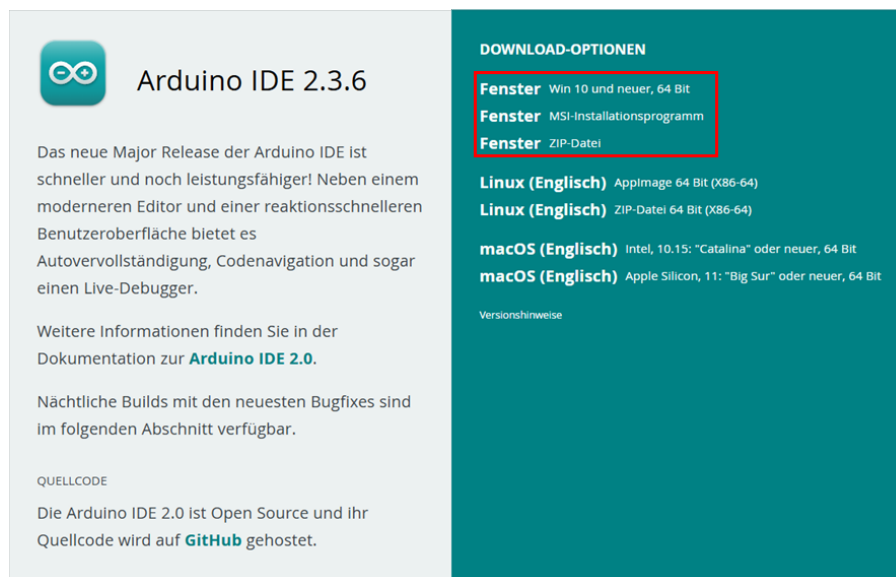


Abbildung 2.2.: Arduino IDE Downloadoptionen

Nachdem der Download der Arduino IDE abgeschlossen wurde, erscheint im nächsten Schritt das Fenster mit den Lizenzbedingungen. Diese sollten aufmerksam durchgelesen werden, bevor mit der Installation fortgefahren wird. Um die Installation zu starten, müssen alle Lizenzbedingungen akzeptiert werden. Dies geschieht durch einen Klick auf die Schaltfläche **Annehmen**, wie in Abbildung 2.3 dargestellt.

Im Anschluss wird der Benutzer aufgefordert, ein Zielverzeichnis für die Installation auszuwählen. Standardmäßig ist ein Pfad vorgegeben, dieser kann jedoch bei Bedarf individuell angepasst werden. Nachdem das gewünschte Verzeichnis ausgewählt wurde, wird der Installationsvorgang durch einen Klick auf **Installieren** gestartet, siehe Abbildung 2.4.

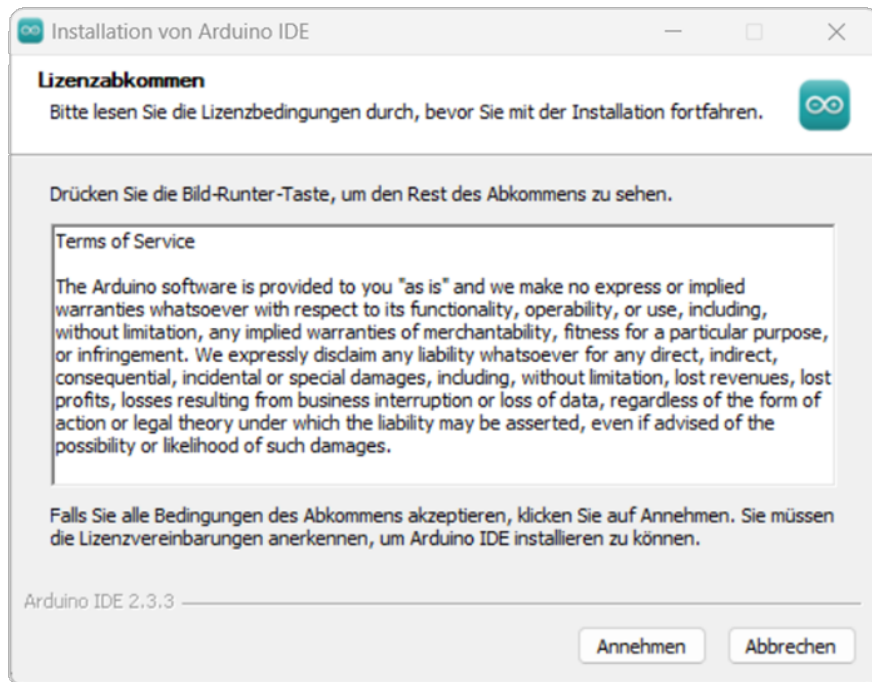


Abbildung 2.3.: Lizenzbestätigung Installation

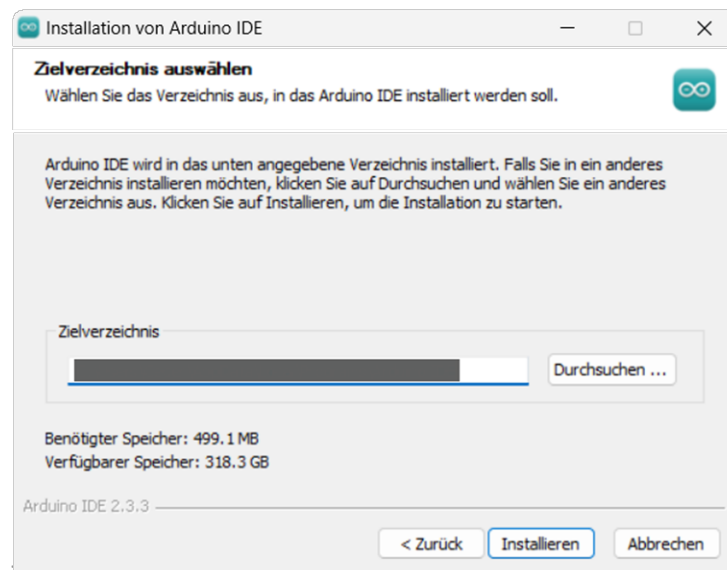


Abbildung 2.4.: Wahl Zielverzeichnis

Sobald die Installation der Arduino IDE erfolgreich abgeschlossen ist, wird die Anwendung automatisch gestartet. Beim ersten Öffnen der Arduino IDE wird ein Standard-Sketch angezeigt, der als Beispiel dient. Dieser ist im Startfenster sichtbar, wie in Abbildung 2.5 dargestellt.

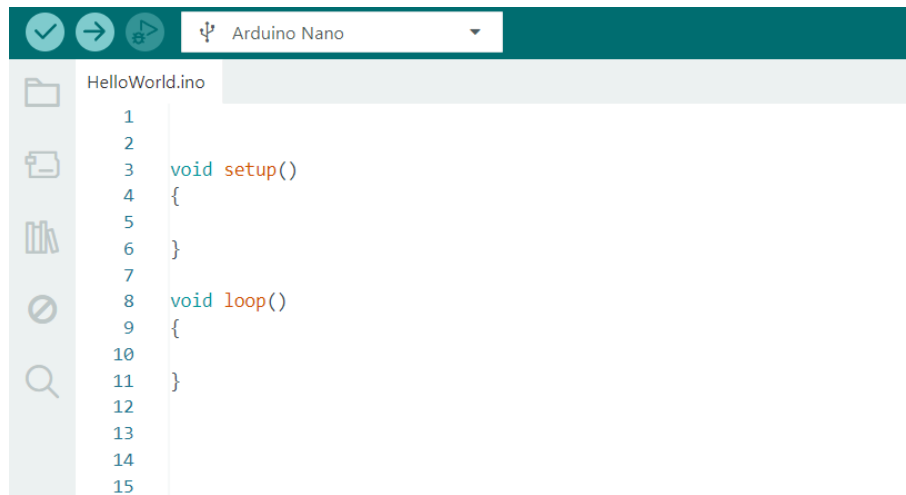


Abbildung 2.5.: Startfenster Arduino-IDE

Aus der oben dargestellten Abbildung 2.5 geht hervor, dass der grundlegende Arduino-Sketch aus zwei Hauptteilen besteht. Der erste Teil ist die Funktion `void setup()`, die keinen Rückgabewert liefert und in der typischerweise die Initialisierungen vorgenommen werden. Hier werden beispielsweise Einstellungen wie die Konfiguration der Ausgangs-LED, die Festlegung von Pin-Modi oder die Initialisierung von Variablen durchgeführt.

Der zweite Teil des Sketches ist die Funktion `void loop()`, in der der Hauptcode geschrieben wird. Funktionen, die wiederholt ausgeführt werden sollen, werden hier definiert und laufen fortlaufend in der Schleife. Diese Funktionen sind ebenfalls zwischen geschweiften Klammern `{ }` eingefasst. Jede Funktion in Arduino hat einen Rückgabewert – in diesem Fall handelt es sich um den Rückgabebetyp `void`, was bedeutet, dass die Funktion keinen Wert zurückgibt.

2.2.4. Installation (MacOS)

Um die Arduino IDE zu installieren, beginnen Sie mit dem Herunterladen der neuesten Version von der offiziellen Arduino-Website unter [Arduino-Software](#). Wählen Sie die Version, die mit Ihrem Betriebssystem kompatibel ist. In diesem Beispiel wird die Version Arduino 2.6.3 für macOS (Sonoma 14.4.1) verwendet. Die Setup-Datei trägt den Namen „Arduino-Software“ und hat eine Größe von etwa 193.600 KB. Diese Datei ist im Zip-Format.

Für Nutzer, die die Arduino IDE 2.3.6 mit Safari herunterladen, wird die Datei nach Abschluss des Downloads automatisch entpackt. Andere Browser hingegen erfordern möglicherweise eine manuelle Entpackung. Die folgende Abbildung 2.6 zeigt die neueste Offline-Version der Arduino IDE 2.3.6, die ebenfalls mit allen gängigen Betriebssystemen kompatibel ist.

Da in diesem Kapitel keine entsprechende Hardware für macOS zur Verfügung steht, wird der Installationsprozess für dieses Betriebssystem nicht im Detail behandelt.

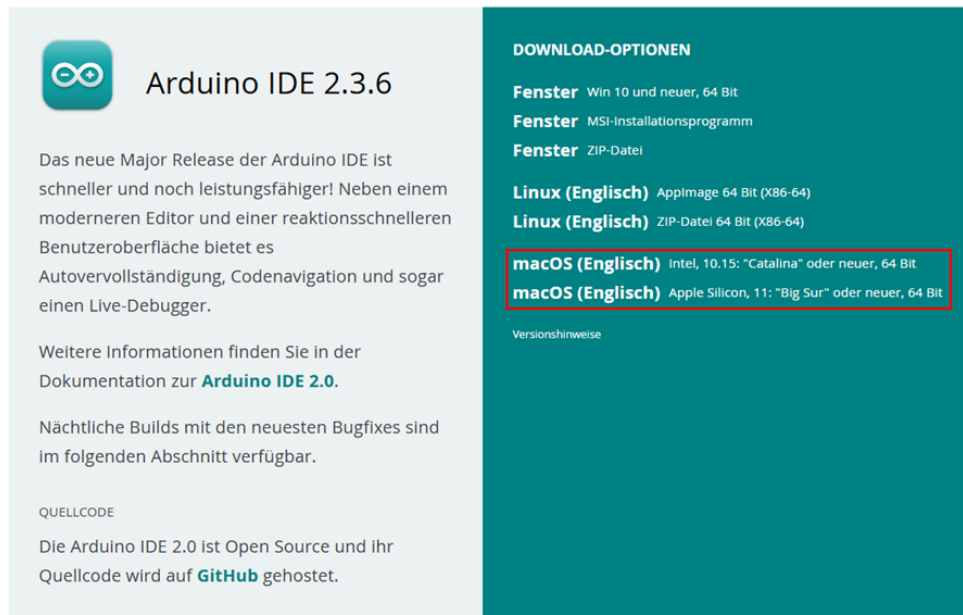


Abbildung 2.6.: ArduinoIDE macOS

2.3. Übersicht der Eigenschaften

Die Arduino IDE verfügt über eine neue Seitenleiste, die den Zugriff auf die am häufigsten verwendeten Werkzeuge erleichtert. 2.7

- **Verify / Upload** Diese Funktionen werden verwendet, um Code zu kompilieren und auf ein Arduino-Board hochzuladen.
- **Select Board and Port** Automatisch erkannte Arduino-Boards werden hier zusammen mit der Portnummer angezeigt.
- **Sketchbook** Dieser Bereich dient als zentrales Verzeichnis für alle Sketche, die lokal auf dem Computer des Benutzers gespeichert sind. Das Sketchbook bietet einen strukturierten, leicht zugänglichen Raum zum Verwalten, Organisieren und Bearbeiten von Code, der in der Arduino-Umgebung entwickelt wurde. Darüber hinaus bietet das Sketchbook eine Synchronisierungsfunktion mit der Arduino Cloud, die einen nahtlosen Zugriff auf gespeicherte Sketche über verschiedene Geräte hinweg ermöglicht. Diese Cloud-Integration erlaubt es Nutzern, Sketche aus der Online-Arduino-Umgebung abzurufen und zu bearbeiten.
- **Boards Manager** Durchsuchen Sie Arduino- und Drittanbieterpakete, die installiert werden können. Zum Beispiel erfordert die Verwendung eines MKR WiFi 1010-Boards die Installation des Arduino SAMD Boards-Pakets.
- **Library Manager** Durchsuchen Sie tausende Arduino-Bibliotheken, die von Arduino und seiner Community erstellt wurden.
- **Debugger** Programme in Echtzeit testen und debuggen.
- **Search** Nach Schlüsselwörtern im geschriebenen Code suchen.
- **Open Serial Monitor** Öffnet das Werkzeug „Serial Monitor“ als neuen Tab in der Konsole.



Abbildung 2.7.: Überblick

2.3.1. Sketchbook

Das Sketchbook ist der Ort, an dem Arduino-Code-Dateien - sogenannte Sketche - gespeichert werden. Diese Dateien haben die Dateiendung .ino. Um eine saubere Organisation zu gewährleisten, muss jeder Sketch in einem Ordner gespeichert werden, der genau denselben Namen wie die Sketch-Datei trägt. Beispiel: Ein Sketch namens MySketch.ino muss in einem Ordner namens MySketch gespeichert werden. Diese Namenskonvention ist notwendig, damit die Arduino IDE die Sketche korrekt erkennen und verwalten kann 2.8.

In der Regel werden Sketche in einem Ordner namens Arduino gespeichert, der sich im Dokumente-Verzeichnis des Systems befindet.

Um auf das Sketchbook zuzugreifen, kann in der Arduino IDE das Ordnersymbol in der Seitenleiste ausgewählt werden. Dadurch wird das Verzeichnis geöffnet, in dem die Sketche gespeichert sind, und ein effizienter Zugriff sowie eine einfache Verwaltung der Sketche ermöglicht.

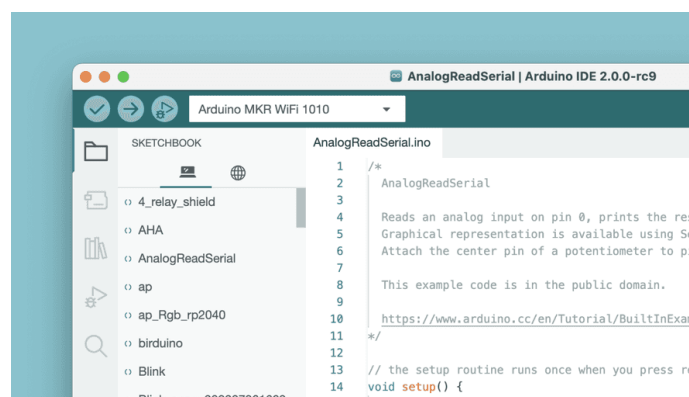


Abbildung 2.8.: Sketchbook

2.3.2. Boards Manager

Der Boards Manager kann über folgende Schritte in der Arduino IDE aufgerufen werden: **Tools** » **Board** » **Board Manager**.

Der Boards Manager ermöglicht die Installation verschiedener Board-Pakete, wie in Abbildung 2.9 dargestellt.

Ein Board-Paket enthält die notwendigen Dateien und Anweisungen, um Code für bestimmte Arduino-Boards zu kompilieren und hochzuladen.

Es gibt verschiedene Board-Pakete, z.B. avr, samd und megaavr. Jedes dieser Pakete unterstützt unterschiedliche Familien von Arduino-Boards:

- avr wird für ältere Boards wie den Arduino Uno verwendet,
- samd ist für neuere Boards wie den Arduino Zero gedacht.

Durch die Verwendung des Boards Managers wird sichergestellt, dass die richtigen Werkzeuge und Dateien für das gewählte Board installiert sind.

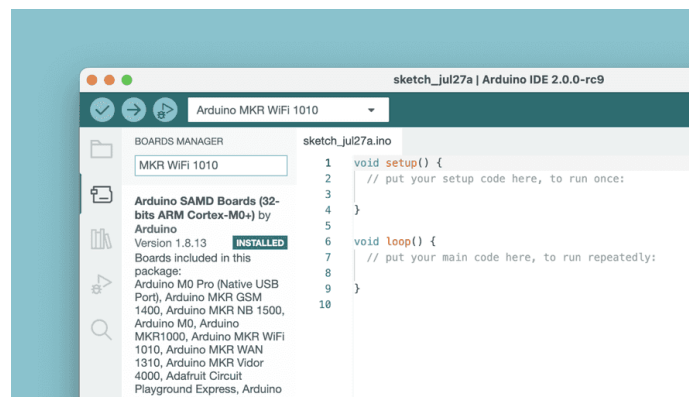


Abbildung 2.9.: Boards Manager

2.3.3. Library Manager

Der Library Manager kann in der Arduino IDE über folgende Schritte aufgerufen werden: **Tools** » **Manage Libraries**.

Der Bibliotheksverwalter ermöglicht es Nutzerinnen und Nutzern, eine große Auswahl an Bibliotheken zu durchsuchen und zu installieren. Bibliotheken erweitern die Grundfunktionen von Arduino und erleichtern Aufgaben wie:

- das Steuern von Servomotoren,
- das Auslesen von Daten bestimmter Sensoren oder
- die Arbeit mit Modulen wie WLAN.

Diese Bibliotheken enthalten vorgefertigten Code, um bestimmte Hardware-Komponenten anzusteuern und komplexe Aufgaben zu vereinfachen. Dadurch wird die Entwicklung von Arduino-Projekten schneller und effizienter – wie in 2.10 dargestellt.



Abbildung 2.10.: Library Manager

2.4. Integration und Verwendung einer benutzerdefinierten Bibliothek in der Arduino IDE

Bei der Entwicklung von eingebetteten Systemen auf der Arduino-Plattform sind Bibliotheken unerlässlich, um komplexe Funktionen zu abstrahieren und zu vereinfachen. Eine benutzerdefinierte Bibliothek ist besonders dann sinnvoll, wenn eine bestimmte Funktionalität oder eine spezifische Hardware-Schnittstelle häufig in verschiedenen Projekten verwendet wird.

Dieser Abschnitt beschreibt den Ablauf zum Erstellen, Installieren und Verwenden einer eigenen Bibliothek innerhalb eines Arduino-Projekts - in der Arduino Integrated Development Environment (IDE).

2.4.1. Erstellen der Bibliotheksdateien

Eine gut strukturierte, benutzerdefinierte Arduino-Bibliothek besteht typischerweise aus mindestens zwei zentralen Komponenten:

Header-Datei (.h): Diese Datei enthält die Deklarationen von Klassen, Funktionen und Variablen, die die Schnittstelle (Interface) der Bibliothek definieren. Sie stellt die notwendigen Definitionen bereit, damit externe Programme die Funktionen und Klassen verwenden können.

Quellcode-Datei (.cpp): Diese Datei enthält die tatsächliche Implementierung der in der Header-Datei deklarierten Funktionen und Methoden. Sie definiert das Verhalten der Funktionen und Klassen.

Die Struktur der Bibliothek ist so organisiert, dass eine klare Trennung zwischen Deklaration und Implementierung ihrer Funktionalitäten besteht.

2.4.2. Installation und Integration der Arduino IDE

Sobald die benutzerdefinierte Bibliothek erstellt wurde, muss sie korrekt installiert und in die Arduino IDE integriert werden, um in Projekten verwendet werden zu können. Um eine benutzerdefinierte Bibliothek in der Arduino IDE zu installieren, befolgen Sie diese Schritte:

1. **Bibliotheksordner finden:** Die Arduino IDE sucht nach Bibliotheken im Ordner `libraries`, der sich im Sketchbook-Verzeichnis befindet. Der typische Pfad zu diesem Verzeichnis lautet:

`C:/Users/<Benutzername>/Documents/Arduino/libraries`

- Benutzerdefinierten Bibliotheksordner kopieren: Kopieren Sie den Ordner ihrer Bibliothek, z.B. Nano33BLESenseLED, in das Verzeichnis `libraries`. Der vollständige Pfad sollte dann wie folgt aussehen:

`C:/Users/<Benutzername>/Documents/Arduino/libraries/Nano33BLESenseLED`

- Arduino IDE neu starten: Nachdem die Bibliothek in das richtige Verzeichnis kopiert wurde, starten sie die Arduino IDE neu, damit sie die neu hinzugefügte Bibliothek erkennt.

2.4.3. Verwendung symbolischer Verknüpfungen zur Bibliotheksintegration

In manchen Fällen kann es bequemer oder notwendig sein, die Bibliotheksdateien außerhalb des Standardbibliotheksverzeichnis zu speichern. Zum Beispiel, wenn die Bibliotheken in einem GitHub-Repository gespeichert sind oder zur besseren Versionskontrolle, kann der Befehl `mklink` verwendet werden, um eine symbolische Verknüpfung zu erstellen.

Dies ermöglicht es der Arduino IDE, auf die Bibliotheksdateien zuzugreifen, als ob sie sich im Standardverzeichnis befinden, ohne dass die Dateien dupliziert werden müssen.

2.4.4. Windows-Tool `mklink` zum Erstellen symbolischer Verknüpfungen

Symbolische Verknüpfungen ermöglichen es, Bibliotheken an einem anderen Speicherort abzulegen, während sie von der Arduino IDE trotzdem so behandelt werden, als befänden sie sich im Standardverzeichnis `libraries`.

Dies kann besonders nützlich sein in versionierten Umgebungen, z.B. bei der Verwendung von Git, oder um Bibliotheken besser zu organisieren, ohne Dateien zu duplizieren.

Schritte zum Erstellen einer symbolischen Verknüpfung:

- Öffnen der Eingabeaufforderung mit Administratorrechten:** Drücken Sie `Win + S` und suchen Sie nach `cmd`, klicken Sie dann auf `Eingabeaufforderung` und wählen Sie `Als Administrator ausführen` wie in 2.11 gezeigt.

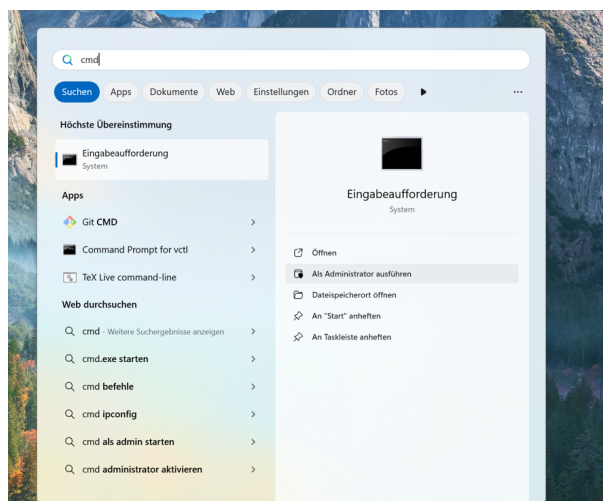


Abbildung 2.11.: Eingabeaufforderung mit Administrator Rechten

2. **Führen Sie den Befehl `mklink` aus:** Der Befehl zum Erstellen einer symbolischen Verknüpfung lautet: `mklink /D TTargetPathSourcePath"`

- **TargetPath:** Der Ort, an dem die Arduino IDE die Bibliothek erwartet – also im Ordner `libraries`.
- **SourcePath:** Der tatsächliche Speicherort der Bibliothek.

In diesem Beispiel befindet sich die Bibliothek unter:

`C:/.../.../.../GitHub/241031ArduinoNano33BLESense/report/Code/Nano33BLESense/Nano33BLESenseLED`

Der Zielpfad im `libraries`-Ordner der Arduino IDE lautet:

`C:/.../.../.../Arduino/libraries/Nano33BLESenseLED`

Der vollständige Command ist in 2.12 zu sehen.

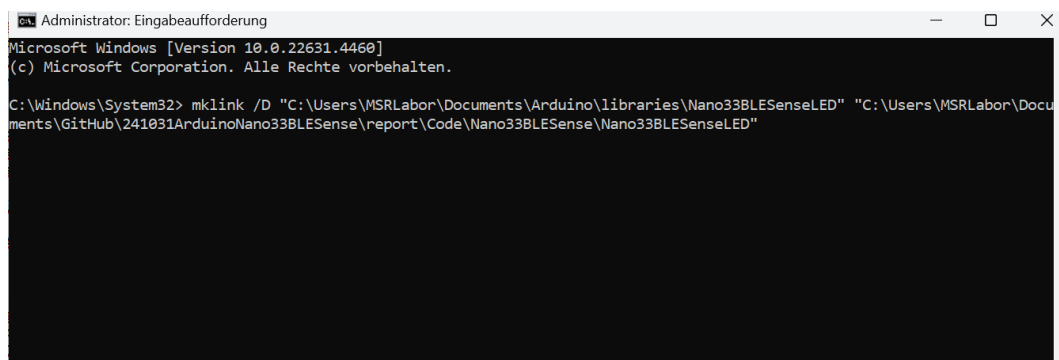


Abbildung 2.12.: Eingabeaufforderung

Überprüfen der symbolischen Verknüpfung: Nach dem Ausführen des Befehls navigieren Sie zum Ordner `libraries`. Dort sollte ein Ordner mit dem Namen `Nano33BLESenseLED` vorhanden sein, der als symbolische Verknüpfung fungiert und auf den tatsächlichen Speicherort der Bibliothek verweist.

2.4.5. Überprüfen der Verfügbarkeit der Bibliothek in der Arduino IDE

Um sicherzustellen, dass die benutzerdefinierte Bibliothek erfolgreich integriert wurde und von der Arduino IDE erkannt wird, befolgen Sie diese Schritte:

1. Arduino IDE neu starten: Falls die Arduino IDE während der Installation der Bibliothek oder nach dem Erstellen der symbolischen Verknüpfung geöffnet war, schließen und öffnen Sie sie erneut. Dadurch wird die Liste der verfügbaren Bibliotheken neu geladen.
2. Bibliotheksmenü überprüfen: Navigieren Sie zu `Sketch` gehen Sie zu `Include Library` und wählen Sie die Bibliothek `Nano33BLESenseLED` aus, wie in 2.13 gezeigt.
3. Überprüfung in der Liste: Wenn die Bibliothek in der Liste erscheint, bedeutet das, dass sie von der IDE erfolgreich erkannt wurde. Falls nicht, überprüfen Sie nochmals die Verzeichnisstruktur und die symbolische Verknüpfung, um sicherzustellen, dass sie korrekt eingerichtet sind.

4. Testprogramm kompilieren: Schreiben Sie ein einfaches Testprogramm, das Funktionen oder Klassen aus der Bibliothek verwendet. Kompilieren Sie den Sketch, um sicherzustellen, dass keine Fehler auftreten – das bestätigt, dass die Bibliothek erfolgreich integriert wurde.

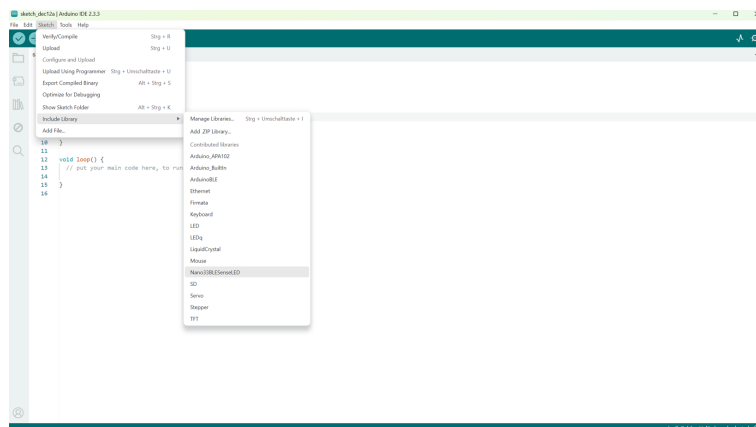


Abbildung 2.13.: Einbinden der Nano33BLESenseLED Bibliothek

Sobald das Programm ohne Fehler kompiliert, ist die Bibliothek einsatzbereit und kann bedenkenlos in Arduino-Projekten verwendet werden.

2.4.6. Serieller Monitor

Der Serieller Monitor kann über folgende Schritte aufgerufen werden: **Tool** » **Serial Monitor**

Der Serieller Monitor ist ein Werkzeug, das es ermöglicht, Daten vom Board anzuzeigen, zum Beispiel mithilfe des Befehls `Serial.print()`.

Früher wurde dieses Tool in einem separaten Fenster geöffnet, ist aber jetzt in den Editor integriert, was es erleichtert, mehrere Instanzen gleichzeitig auf dem Computer auszuführen.

Der Serieller Monitor ist in 2.14 dargestellt.

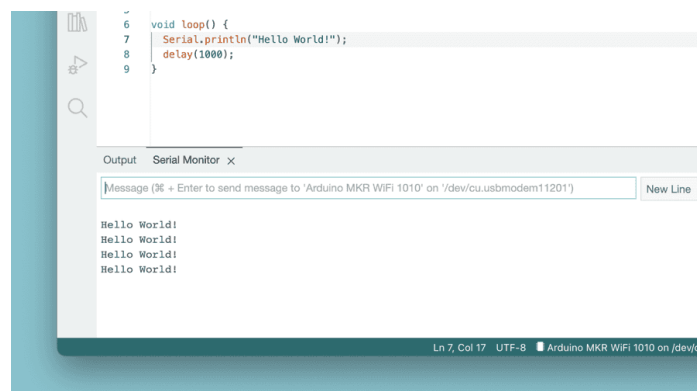


Abbildung 2.14.: Serieller Monitor

2.4.7. Serieller Plotter

Das Werkzeug **Serial Plotter** eignet sich hervorragend zur Visualisierung von Daten in Form von Diagrammen, z.B. zur Überwachung von Spannungsspitzen.

Es ermöglicht das gleichzeitige Beobachten mehrerer Variablen und bietet die Möglichkeit, bestimmte Datentypen gezielt ein- oder auszublenden.

2.5. Beispiele

Ein wesentlicher Bestandteil der Arduino Dokumentation ist die Sammlung von Beispielsketches, die mit verschiedenen Bibliotheken gebündelt sind.

Diese Beispiele zeigen die praktische Anwendung von Bibliotheksfunktionen und veranschaulichen deren typische Einsatzmöglichkeiten und Hauptfunktionen. Auch Bibliotheken, die mit bestimmten Board-Paketen geliefert werden, können eigene Beispielsketches enthalten.

Um auf diese Beispielsketches zuzugreifen – egal ob sie manuell installierten Bibliotheken oder mitgelieferten Board-Paketen zugeordnet sind – navigiere in der Arduino IDE zu: **File** » **Examples**, und wähle dann die gewünschte Bibliothek aus der Liste aus. Beispielsweise erscheint beim Anschluss eines UNO R4 WiFi-Boards eine Liste mit bezugsspezifischen Beispielen. Ein Beispielsketch mit einer vorgeladenen Tetris-Animation kann aufgerufen werden über: **File** » **Examples** » **LED Matrix** » **MatrixIntro** und auf das angeschlossene Board hochgeladen werden.

Dieses Beispiel zeigt, wie der mitgelieferte Code des Boards in der Praxis genutzt werden kann und hilft den Nutzern, verschiedene Hardwarefunktionen direkt zu steuern und kennenzulernen.

In Abbildung 2.15 wird die Beispielliste angezeigt, wenn ein UNO R4 WiFi-Board mit dem Computer verbunden ist.

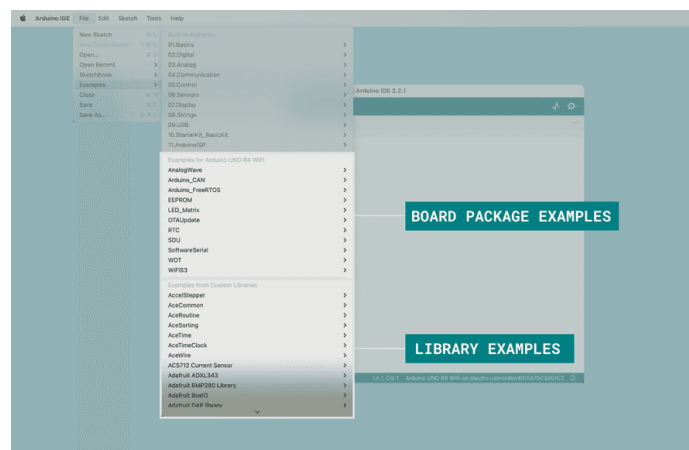


Abbildung 2.15.: Beispiele

2.6. Debuggen

Das Tool **Debugger** in der Arduino IDE 2.3.3 ist dafür konzipiert, Programme beim Testen und Debuggen zu unterstützen, indem es ermöglicht, die Codeausführung auf kontrollierte Weise Schritt für Schritt durchzugehen.

Dieses Tool erlaubt es den Nutzern, Haltepunkte zu setzen, den Code Zeile für Zeile durchzugehen und Variablen zu inspizieren, um Probleme wie Logikfehler oder falsche Variablenwerte zu identifizieren.

Der Debugger ermöglicht es, die Ausführung an bestimmten Punkten im Programm anzuhalten, um das Verhalten des Programms und den Speicherzustand im Detail zu untersuchen.

Diese Funktion verbessert den Entwicklungsprozess, indem sie es erleichtert, Fehler zur Laufzeit zu finden und zu beheben, anstatt sich ausschließlich auf print-Anweisungen oder Versuch-und-Irrtum zu verlassen 2.16.

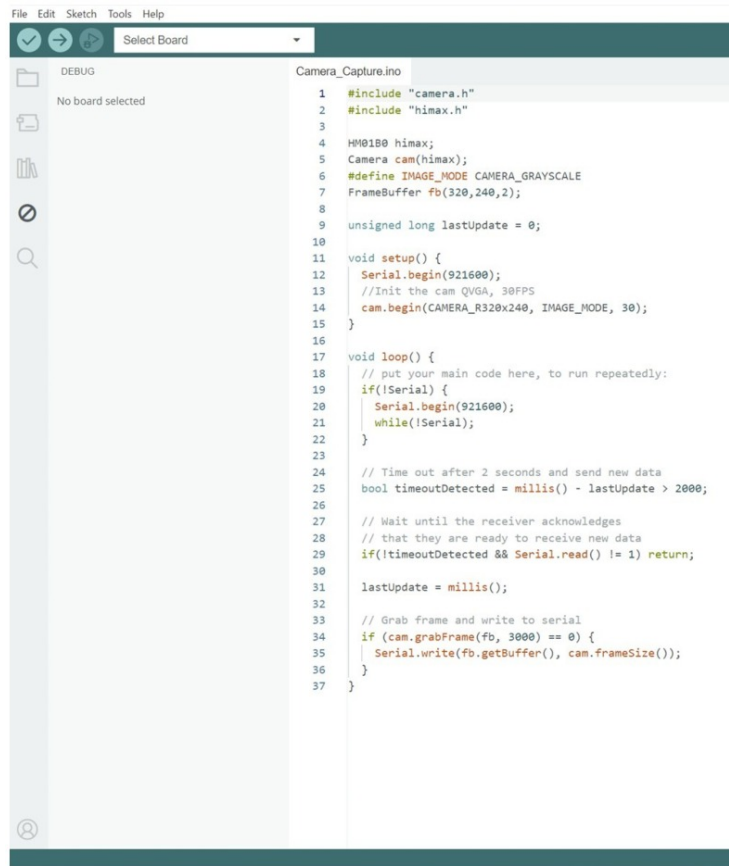


Abbildung 2.16.: Beispiel Debuggen

2.7. Autovervollständigung

Die Autovervollständigung ist eine zentrale Funktion der Arduino IDE Version 2, die das Programmieren erleichtert, indem sie Funktionen und Elemente aus der Arduino-API vorschlägt. Diese Funktion hilft dabei, Code schneller und genauer zu schreiben, was die Effizienz und Genauigkeit verbessert.

Damit die Autovervollständigung funktioniert, muss in der Arduino IDE das verwendete Board ausgewählt sein. Dadurch erkennt der Editor die richtigen Funktionen und Bibliotheken für das jeweilige Board und kann präzise Vorschläge machen 2.17.

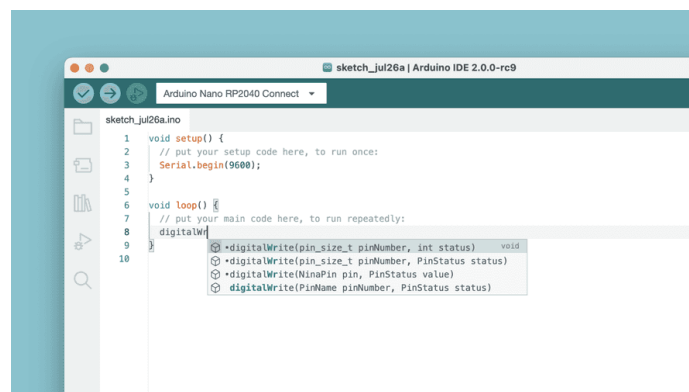


Abbildung 2.17.: Autovervollständigung

2.8. Bibliotheken und Modularität

Die Arduino IDE erlaubt es, Funktionen in Bibliotheken auszulagern und wiederzuverwenden. Viele Sensoren oder Module wie DHT11, LCD oder Motorshields bringen eigene Bibliotheken mit, die über die Bibliotheksverwaltung eingebunden werden können [Smi11]:

```
#include <ArduinoBLE.h>
```

Die Verwendung von Bibliotheksfunktionen ermöglicht es, größere Programme in modulare Teilfunktionen zu gliedern und dadurch Struktur sowie Wartbarkeit deutlich zu verbessern. So kann beispielsweise mit der Funktion `BLE.setLocalName()` ein Bluetooth-Name für die Applikation festgelegt werden, ohne dass die zugrunde liegenden Befehle und Abläufe im Detail bekannt sein müssen.

3. Einrichtung des Arduino UNO

In diesem Abschnitt wird der Vorgang beschrieben, wie der Arduino UNO mit einem Computer verbunden und in der Arduino IDE eingerichtet wird. Dazu werden die notwendigen Pakete installiert, das Board sowie der zugehörige Port ausgewählt und abschließend anhand eines Beispiel-Sketches überprüft, ob die Einrichtung erfolgreich war.

3.1. Installation der Pakete für den Arduino UNO

Um den Arduino UNO zu programmieren, wird zunächst das passende Treiberpaket in der Arduino IDE installiert. Dazu wird in der Menüleiste der Punkt Tools ausgewählt und darunter der Eintrag Board Manager geöffnet. Im Suchfeld des Board Managers wird **Ärduino UNO** eingegeben 3.1. Aus den Suchergebnissen wird das Paket **Ärduino AVR Boards** ausgewählt und über die Schaltfläche **Install** installiert.

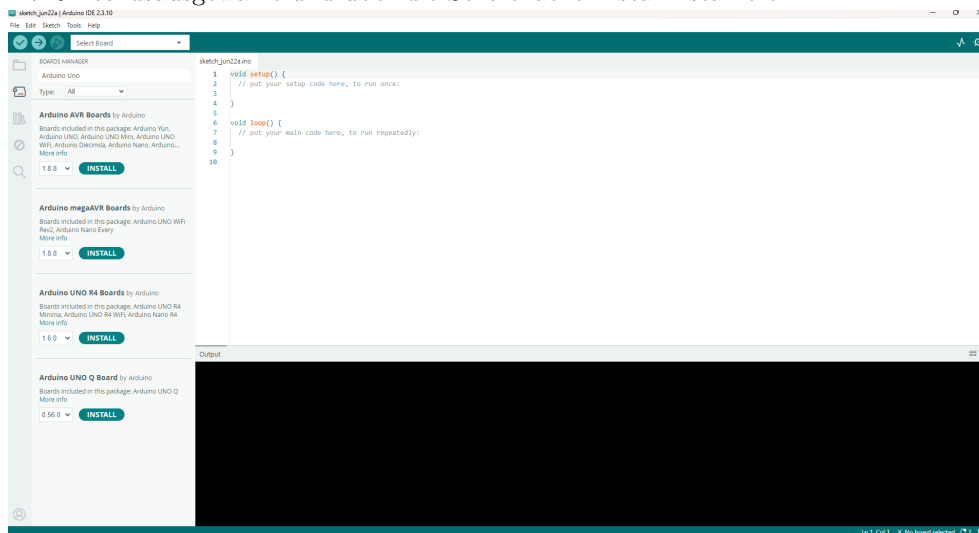


Abbildung 3.1.: Installation der Arduino AVR Boards

Das Paket **Arduino AVR Boards** enthält die Unterstützung für mehrere Boards der AVR-Familie, unter anderem für den **Arduino UNO**. Nach Abschluss der Installation wird der **Arduino UNO** über ein **USB-A-auf-USB-B-Kabel** mit dem Computer verbunden.

3.2. Verbindung und Auswahl des Boards

Nachdem der **Arduino UNO** über das **USB-Kabel** mit dem Computer verbunden wurde, wird die **Arduino IDE** geöffnet. Es erscheint eine leere Umgebungsseite mit den standardmäßigen Funktionen `void setup()` und `void loop()`. Anschließend wird in der Menüleiste unter dem Punkt **Tools** der Unterpunkt **Board** ausgewählt und dort der **Arduino UNO** als angeschlossenes Board festgelegt (vgl. Abbildung 1.25).

Abbildung 1.25: Auswahl des angeschlossenen Boards – hier **Arduino UNO**

Mit dieser Auswahl ist die Arduino IDE darauf vorbereitet, den Arduino UNO zu erkennen und mit ihm zu kommunizieren.

3.3. Auswahl des Ports

Nach der Boardauswahl wird der verbundene Port überprüft. Im Gegensatz zu einigen anderen Arduino-Boards muss der Arduino UNO dafür nicht zusätzlich in einen Bootloader-Modus versetzt werden, der Port wird beim Anschließen automatisch erkannt. Dazu wird in der Menüleiste unter dem Punkt Tools der Unterpunkt Port ausgewählt und der verfügbare Port für das Hochladen des Programms festgelegt (vgl. Abbildung 1.26).

Abbildung 1.26: Auswahl des verfügbaren Ports zum Hochladen des Arduino-Sketches

3.4. Hochladen und Überprüfen eines Sketches

Nachdem der passende Port ausgewählt wurde, kann das Arduino-Programm hochgeladen werden. Vor dem Hochladen wird der Code zunächst überprüft (verifiziert), um sicherzustellen, dass keine Fehler oder Warnungen vorliegen. Anschließend wird der Code über die Schaltfläche Upload auf das Board übertragen (vgl. Abbildung 1.27). Ob der Arduino UNO korrekt mit der Arduino IDE verbunden ist, lässt sich anhand der Meldung in der unteren rechten Ecke des IDE-Fensters überprüfen, welche das verbundene Board und den Port anzeigt.

Abbildung 1.27: Hochladen des Programms auf das Arduino Board

Nach dem Hochladen wird der Code kompiliert und etwaige Fehler werden im unteren Ausgabefenster angezeigt. Sobald der Code erfolgreich übertragen wurde, wird der Port erneut über Tools und Port überprüft (vgl. Abbildung 1.28), um die Ausgabe im Serial Monitor anzeigen zu können.

Abbildung 1.28: Einstellen des Ports

3.5. Test der Konfiguration anhand des Beispiels Blink

Um zu überprüfen, ob das Board korrekt mit der Arduino IDE verbunden ist, wird das eingebaute Beispiel *Blink* verwendet. Dieser einfache Test lässt die auf dem Board integrierte LED blinken und bestätigt damit sowohl die Verbindung als auch die grundlegende Funktionalität des Arduino UNO.

Dazu wird in der Arduino IDE über das Menü File der Punkt Examples ausgewählt. Unter den eingebauten Beispielen wird unter 01.Basics der Eintrag Blink ausgewählt 3.2. Das Beispiel-Sketch öffnet sich daraufhin in einem neuen Fenster innerhalb der IDE. Über die Schaltfläche Upload in der Werkzeugleiste wird das Beispiel kompiliert und auf den Arduino UNO übertragen. Nach Abschluss des Uploads beginnt die integrierte LED zu blinken.

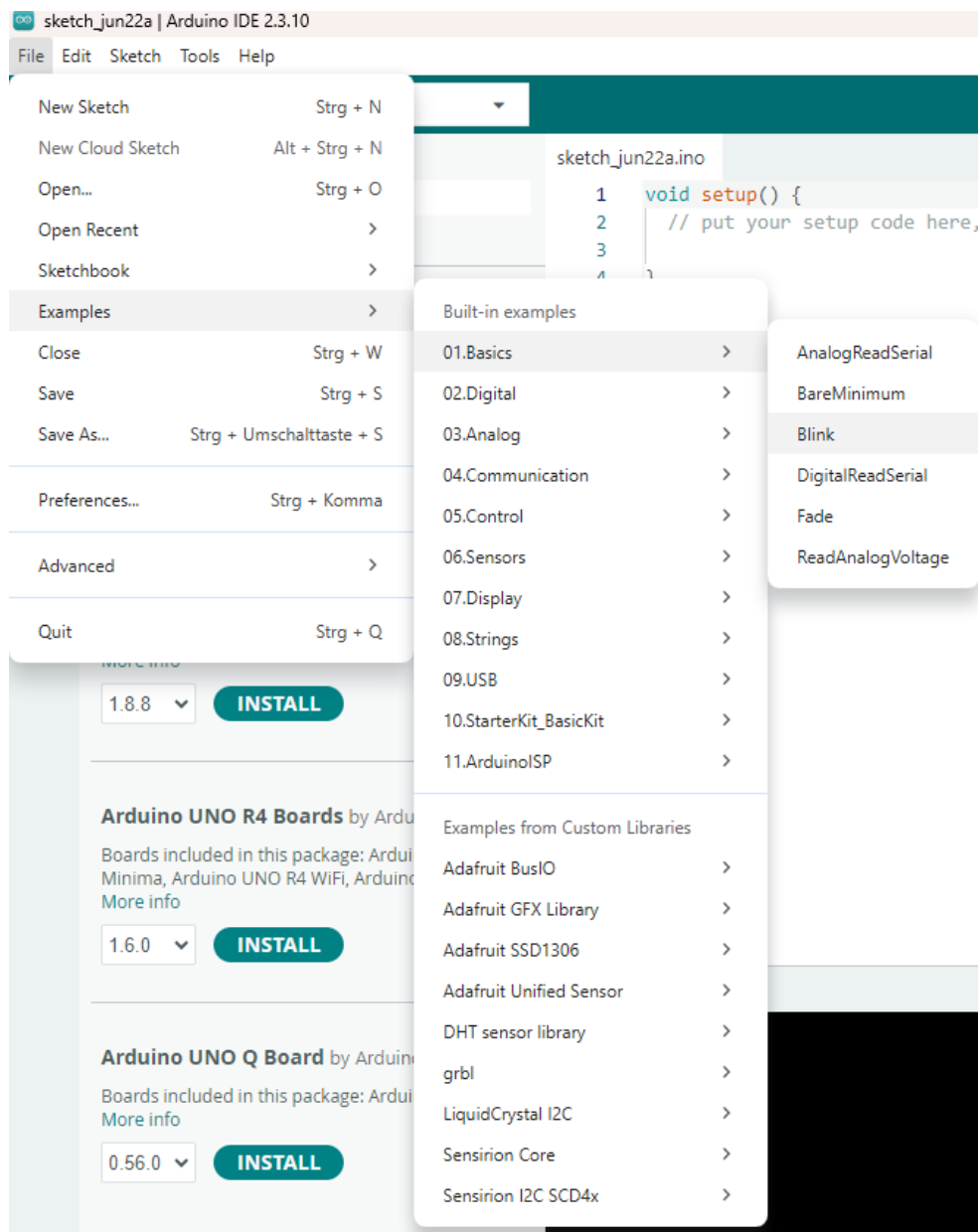


Abbildung 3.2.: Beispiel-Sketch Blink

Listing 3.1.: Blink Programm

```

1000 /**
1001  * @file blink.ino
1002  * @brief Simple example to blink the on-board LED on and off.
1003  *
1004  * @details
1005  * Turns the built-in LED on for one second, then off for one second
1006  * repeatedly.
1007  * Most Arduino boards have a built-in LED connected to a specific
1008  * digital pin.
1009  * The macro LED_BUILTIN is used to ensure compatibility across
1010  * different boards.
1011  *
1012  * @note For more details about your board's built-in LED, see:
1013  * https://www.arduino.cc/en/Main/Products

```

```
1012 *
1013 * @see https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
1014 *
1015 * @author
1016 * - Scott Fitzgerald (original)
1017 * - Arturo Guadalupi (modification)
1018 * - Colby Newman (modification)
1019 *
1020 * @date Modified:
1021 * - 8 May 2014
1022 * - 2 Sep 2016
1023 * - 8 Sep 2016
1024 *
1025 * @copyright Public Domain
1026 */
1027
1028 /**
1029 * @brief Initializes the LED pin as an output.
1030 *
1031 * This function runs once at startup after reset or power-up.
1032 */
1033 void setup() {
1034     pinMode(LED_BUILTIN, OUTPUT); ///< Set the LED pin as output
1035 }
1036
1037 /**
1038 * @brief Toggles the LED on and off every second.
1039 *
1040 * This function runs repeatedly after setup().
1041 */
1042 void loop() {
1043     digitalWrite(LED_BUILTIN, HIGH); ///< Turn the LED on
1044     delay(1000);                      ///< Wait for one second
1045     digitalWrite(LED_BUILTIN, LOW);  ///< Turn the LED off
1046     delay(1000);                      ///< Wait for one second
1047 }
```

../Code/Blink/Blink.ino

Mit dem erfolgreichen Blinken der integrierten LED ist bestätigt, dass der Arduino UNO ordnungsgemäß mit der Arduino IDE verbunden ist und seine grundlegende Funktionalität überprüft wurde. Das Board ist damit bereit für die weitere Entwicklung.

4. Arduino Web Editor

4.1. Using the Arduino Web Editor

The Arduino Web Editor allows you to write code and upload sketches to any official Arduino board directly from your web browser (Chrome, Firefox, Safari and Edge). [Ard24]

This IDE (Integrated Development Environment) is part of Arduino Create, an online platform that enables developers to write code, access tutorials, configure boards, and share projects. Designed to provide users with a continuous workflow, Arduino Create connects the dots between each part of a developer's journey from inspiration to implementation. Meaning, you now have the ability to manage every aspect of your project right from a single dashboard.

The Arduino Web Editor is hosted online, therefore it is always be up-to-date with the latest features and support for new boards. This IDE lets you write code and save it to the Cloud, always backing it up and making it accessible from any device. It automatically recognizes any Arduino board connected to your PC, and configures itself accordingly.

All you need to get started is an Arduino account. The following steps can guide you to start using the Arduino Web Editor: [Ard24]

4.1.1. Using the online IDE

After logging in, you are ready to start using the Arduino Web Editor. The web app is divided into three main columns. 4.1

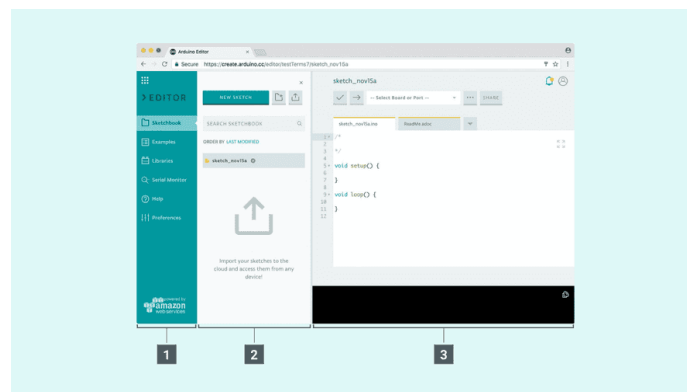


Abbildung 4.1.: Arduino Web Editor

The Arduino Web Editor's interface is as follows:

1. The first column lets you navigate between:
 - **Your Sketchbook:** a collection of all your sketches (a sketch is a program you upload on your board).
 - **Examples:** read-only sketches that demonstrate all the basic Arduino commands (built-in tab), and the behavior of your libraries (from the libraries tab).

- **Libraries:** packages that can be included to your sketch to provide extra functionalities.
 - **Serial monitor:** a feature that enables you to monitor, receive and send data to and from your board via the USB cable.
 - **Help:** helpful links and a glossary about Arduino terms.
 - **Preferences:** options to customize the look and behavior of your editor, such as text size and color theme.
2. The second column views the content of the chosen option.
 3. The third column, the code area, is the one you will use the most. Here, you can write code, verify it and upload it to your boards, save your sketches on the Cloud, and share them with anyone you want.

Now that we are all set up, let's try to make the board blink!

1. Connect your Arduino or Genuino board to your computer. Boards and serial ports are auto-discovered and selectable in a single dropdown. Pick the Arduino/Genuino board you want to upload to from the list at the top of the third column.[Ard24]
2. Let's try with an example: Choose Examples on the menu on the left (first column), then Basic and Blink. The Blink sketch is now displayed in the code area.
3. To upload it to your board, press the "Upload" button near the dropdown menu. While the code is verifying and uploading, a "BUSY" label replaces the upload button. If the upload is successful, the message "Success: done uploading" will appear in the bottom output area.
4. Once the upload is complete, you should see on your board the yellow LED with an L next to it start blinking. You can adjust the speed of blinking by changing the delay number in the parenthesis to 100, and upload the Blink sketch again. Now the LED should blink much faster ??.



Abbildung 4.2.: finding-an-example

4.2. Arduino Libraries

The process of setting up libraries on the online IDE (Arduino Cloud Editor) is quite similar to the offline one: [Ard25b; Ard25a]

1. Login to the Arduino Cloud.

2. Create or open a sketch.
3. Open the LLibrariesttab from the left menu, and search for libraries. The list displays read-only libraries, authored and maintained by the Arduino team and its partners.
4. When you find the library, you can add it to your sketch by selecting the "Include" button. You can also see the related examples, and select a specific version, if available.
5. If you can't find a specific library on the list, you can search every existing library through the search bar. You also have the option to add them to your favorites list by clicking on the star next to the library you want. Once you star a library, you can view it under the ffavoritesttab and use its examples (if available).

4.3. Data Security

- **Encryption:** The web version of Arduino IDE should use strong encryption protocols to secure data transmitted between the user's browser and the server, preventing unauthorized access.
- **Secure Storage:** User data, including login credentials and project files, should be encrypted at rest to protect against unauthorized access or tampering.

4.3.1. Authentication and Authorization

- **Strong Password Policies:** Enforce password complexity and offer multi-factor authentication to enhance account security.
- **Role-Based Access Control:** Implement role-based access to ensure users only access necessary resources, preventing unauthorized access to sensitive data.

4.3.2. Secure Code Execution

- **Sandboxing:** Execute user-uploaded code in a sandboxed environment to restrict access to system resources and prevent malicious actions.
- **Code Validation:** Validate user code to detect and prevent vulnerabilities like code injection or buffer overflows before execution.

4.3.3. Protection Against Malware

- **Code Scanning:** Use code scanning to detect and block malicious code by analyzing for malware signatures or suspicious patterns.
- **Antivirus Integration:** Integrate antivirus solutions to scan uploaded files for viruses or malicious content before execution.

4.3.4. Privacy Policies

- **Transparency:** Maintain clear privacy policies outlining data collection, storage, usage, retention periods, and third-party sharing practices.
- **User Consent:** Inform users about privacy practices and obtain explicit consent for data processing, allowing them to review and modify privacy settings.

4.3.5. Regular Updates and Maintenance

- **Patch Management:** Regularly update software components, libraries, and dependencies to address security vulnerabilities promptly.
- **Security Audits:** Conduct regular security audits and penetration testing to identify and remediate potential weaknesses.

4.3.6. User Awareness

- **Security Education:** Provide resources on secure coding practices to help users avoid vulnerabilities like XSS, SQL injection, and insecure object references.
- **Security Alerts:** Notify users about security-related issues, such as suspicious login attempts or unauthorized access.

5. Universal G-Code Sender

5.1. Einleitung

Dieses Kapitel beschreibt den Universal G-Code Sender, kurz UGS, als Bediensoftware für GRBL-basierte CNC-Systeme. Die Software stellt die Verbindung zwischen dem Computer und dem Mikrocontroller her, lädt G-Code-Dateien, visualisiert Werkzeugpfade und überträgt die Befehle zeilenweise an die Steuerung. Damit bildet UGS die Benutzeroberfläche des Heißdraht Schneiders, während GRBL die Bewegungsbefehle auf dem Arduino verarbeitet.

5.2. Beschreibung des Universal G-Code Senders

Der Universal G-Code Sender ist eine quelloffene, Java-basierte Steuerungssoftware zur seriellen Kommunikation zwischen einem Host-Computer und dem Mikrocontroller einer CNC-Maschine. UGS ist plattformübergreifend und kann unter anderem unter Windows, Linux und macOS verwendet werden. Die Software bündelt die notwendigen Laufzeitkomponenten in einem ausführbaren Paket und kann dadurch ohne separat installierte Java-Laufzeitumgebung genutzt werden [WU25].

UGS übernimmt im Betrieb mehrere zentrale Aufgaben:

- Kommunikationsschicht: Verwaltung der seriellen Verbindung über USB einschließlich Portauswahl, Baudrateneinstellung und Puffersteuerung.
- G-Code-Sender: Laden, Prüfen und zeilenweises Übertragen von G-Code-Dateien an den Controller.
- Maschinenzustandsanzeige: Darstellung des aktuellen Zustands, der Achspositionen und weiterer Steuerungsinformationen im Digital Read-Out.
- Werkzeugpfad-Visualisierung: Grafische Vorschau des geladenen G-Codes in einer 2D- beziehungsweise 3D-Ansicht.
- Konsolenzugriff: Direkte Eingabe einzelner Befehle und Anzeige von Rückmeldungen der Firmware.

Die Anwendungsbereiche von UGS umfassen Maschinen, die G-Code-basierte Steuerbefehle verarbeiten. Dazu gehören CNC-Fräsmaschinen, Laserschneider, Lasergraviermaschinen, Plotter und andere kartesische Bewegungssysteme [WU25].

5.3. Aufbau der Benutzeroberfläche

Die Benutzeroberfläche von UGS ist in mehrere Funktionsbereiche gegliedert. Diese Trennung erleichtert die Bedienung, weil Maschinenzustand, manuelle Steuerung, Werkzeugpfad-Vorschau und Diagnose voneinander getrennt dargestellt werden. Für die praktische Anwendung sind besonders die Bereiche *Jog Control*, *Visualizer*, *Console* und *Controller State / DRO* wichtig [WU25].

5.3.1. Jog Control

Die Jog Control dient zur manuellen Bewegung der Achsen. Mit ihrer Hilfe kann die Maschine schrittweise in positive oder negative Richtung verfahren werden. Dies ist zum Beispiel notwendig, um den Werkstücknullpunkt anzufahren, die Achsrichtung zu prüfen oder die Maschine vor einem Bearbeitungsvorgang zu positionieren.

Typische Anwendungsfälle der Jog-Steuerung sind:

- Anfahren des Werkstücknullpunkts,
- Test der Achsbewegungen nach dem Verbindungsaufbau,
- Verfahren zu Referenz- oder Prüfpositionen,
- Kontrolle der richtigen Bewegungsrichtung einzelner Achsen.

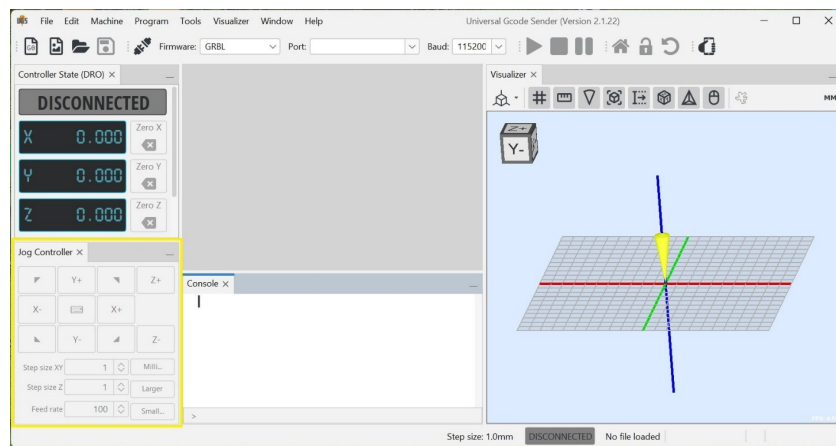


Abbildung 5.1.: Jog Control – Bedienfeld in UGS

5.3.2. Visualizer

Der Visualizer dient der grafischen Darstellung des geladenen G-Code-Programms. In diesem Fenster wird die Werkzeugbahn als 2D- oder 3D-Vorschau angezeigt. Dadurch kann vor dem Start geprüft werden, ob die Kontur, die Bewegungsrichtung und die Ausrichtung des Programms plausibel sind.

Der Visualizer unterstützt insbesondere:

- die Kontrolle der importierten Werkzeugbahn,
- die Erkennung fehlerhafter Konturen oder unerwarteter Verfahrswege,
- die Darstellung der aktuellen Werkzeugposition während der Bearbeitung,
- die Überprüfung von Start- und Zielpunkten.

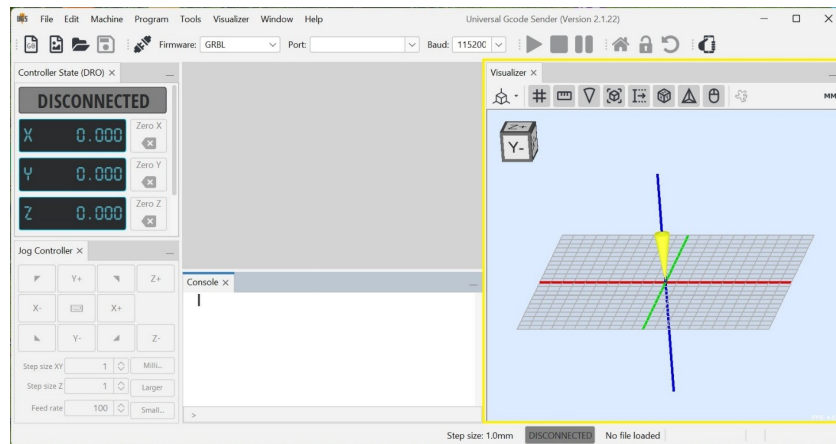


Abbildung 5.2.: Visualizer – Grafikanzeigefeld in UGS

5.3.3. Console

Die Console ist die textbasierte Kommunikationsschnittstelle zwischen UGS und der Firmware. In diesem Bereich werden gesendete Befehle, empfangene Rückmeldungen, Statusmeldungen und Fehlermeldungen angezeigt. Sie ist deshalb ein wichtiges Werkzeug für Diagnose, Konfiguration und Fehlersuche [WU25].

Über die Console können einzelne Befehle direkt eingegeben werden, zum Beispiel:

- **\$\$** zum Anzeigen aller gespeicherten GRBL-Parameter,
- **\$G** zum Anzeigen der aktiven modalen Zustände,
- **?** zur Statusabfrage,
- **\$H** zum Starten des Homing-Zyklus,
- **\$X** zum Entsperren nach einem Alarmzustand.

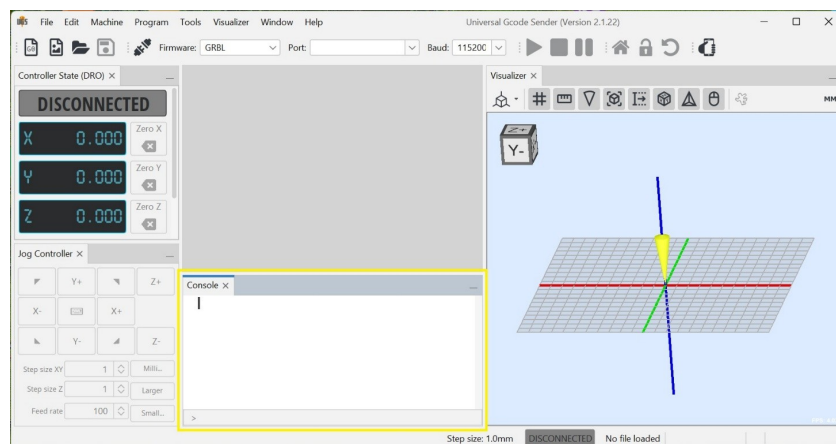


Abbildung 5.3.: Console – Eingabefeld in UGS

5.3.4. Controller State / DRO

Der Bereich Controller State, häufig auch als DRO (Digital Read-Out) bezeichnet, zeigt den aktuellen Maschinenzustand in Echtzeit an. Dort werden die aktuellen Koordinaten der Achsen sowie Zustände wie **Idle**, **Run**, **Hold** oder **Alarm** angezeigt.

Dieser Bereich dient zur Kontrolle der aktuellen Achspositionen, zur Überwachung des Maschinenzustands, zur Prüfung des Werkstücknullpunkts und zur Rückmeldung über laufende oder fehlerhafte Bewegungsabläufe.

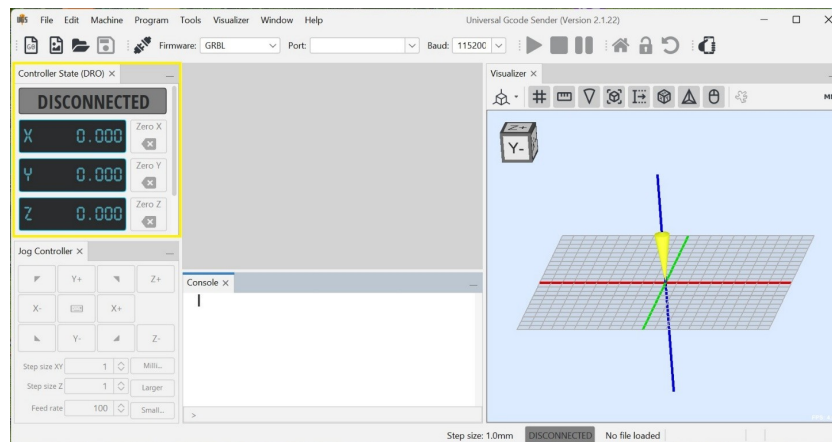


Abbildung 5.4.: Controller State (DRO) – Positionsanzeige in UGS

5.3.5. Zusammenspiel der Bedienbereiche

Die genannten Bereiche arbeiten im praktischen Betrieb eng zusammen. Während die Jog Control zur manuellen Positionierung genutzt wird, erlaubt das DRO die unmittelbare Kontrolle der erreichten Koordinaten. Gleichzeitig zeigt die Console, welche Befehle an die Firmware übermittelt werden und wie diese darauf reagiert. Der Visualizer ergänzt diese Informationen durch die grafische Darstellung des geplanten beziehungsweise laufenden Bewegungsablaufs.

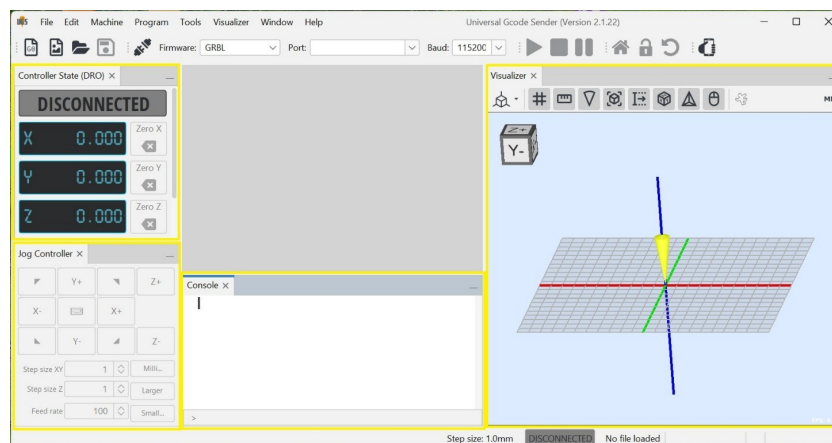


Abbildung 5.5.: Übersicht über die Bedien- und Anzeigefelder in UGS

5.4. Installation des Universal G-Code Senders

5.4.1. Systemvoraussetzungen

- Betriebssystem: Windows 10 oder Windows 11, Linux oder macOS,
- Arbeitsspeicher: mindestens 2 GB RAM, empfohlen 4 GB RAM,
- Festplattenspeicher: mindestens 200 MB freier Speicherplatz,

- Schnittstelle: USB-Anschluss zur Verbindung mit der CNC-Maschine,
- USB-Treiber: abhängig vom verbauten USB-Chip des Controllers, zum Beispiel CH340 oder CP210x.

5.4.2. Download und Installation

Der Download erfolgt über die offizielle Download-Seite des Projekts beziehungsweise über die GitHub-Releases-Seite. Dort wird die aktuelle stabile Version ausgewählt und die für das jeweilige Betriebssystem passende Variante heruntergeladen, zum Beispiel [ugs-platform-app-win-x64.zip](#) für Windows 64-Bit [WU25].

Die Installation erfolgt in folgenden Schritten:

1. Das heruntergeladene ZIP-Archiv in ein Verzeichnis eigener Wahl entpacken.
2. In den entpackten Ordner wechseln.
3. Den Unterordner `bin` öffnen.
4. Die Anwendung durch Doppelklick auf `ugsplatform64.exe` starten.
5. Eine mögliche Windows-Sicherheitswarnung über *Weitere Informationen* und *Trotzdem ausführen* bestätigen.

5.4.3. Erstverbindung mit der CNC-Maschine

1. Den Arduino beziehungsweise Controller über USB mit dem Computer verbinden.
2. Im Windows-Geräte-Manager den zugewiesenen COM-Port unter *Anschlüsse (COM & LPT)* ermitteln.
3. In UGS den ermittelten COM-Port im Dropdown-Menü *Port* auswählen.
4. Die Baudrate auf `115200` einstellen, da dies der Standard für GRBL v1.1 ist.
5. Auf *Connect* klicken.
6. Bei erfolgreicher Verbindung erscheint im Konsolenfenster die GRBL-Versionsmeldung, zum Beispiel `Grbl 1.1h ['$' for help]`.
7. Als Erfolgstest kann der Befehl `$$` eingegeben und die ausgegebenen Parameter überprüft werden.

5.5. Konfiguration und Diagnose in UGS

Die serielle Verbindung zwischen UGS und dem GRBL-Controller wird über den Port, die Baudrate und weitere Verbindungsparameter eingerichtet. Für GRBL v1.1 wird normalerweise eine Baudrate von `115200` Baud verwendet. Die Datenübertragung erfolgt mit 8 Datenbits, 1 Stoppbit und ohne Parität. Diese Konfiguration wird auch als `8N1` bezeichnet.

Für Diagnosezwecke können über die Console Status- und Steuerbefehle gesendet werden. Mit `?` wird der aktuelle Maschinenstatus abgefragt. Über `$G` können die aktiven modalen G-Code-Zustände angezeigt werden. Die Eingabe von `$I` gibt Versions- und Build-Informationen der Firmware aus. Fehlermeldungen und Alarmzustände werden ebenfalls in der Console angezeigt und können dort ausgewertet werden.

5.6. Verwendung

5.6.1. G-Code selbst schreiben

G-Code ist die Programmiersprache für numerisch gesteuerte Maschinen. In UGS können G-Code-Befehle entweder einzeln im Kommandozeilenfeld eingegeben oder als vollständige Datei geladen und im Batch-Mode ausgeführt werden [WU25].

Tabelle 5.1.: Häufig verwendete G- und M-Codes

Befehl	Funktion
G0	Eilgang beziehungsweise nicht bearbeitende Bewegung mit maximaler Geschwindigkeit
G1	Linearinterpolation mit programmiertem Vorschub
G2 / G3	Kreisinterpolation im beziehungsweise gegen den Uhrzeigersinn
G20 / G21	Umschaltung zwischen Zoll- und Millimeter-Modus
G90 / G91	Absolutes beziehungsweise inkrementales Koordinatensystem
G28	Fahrt zur vordefinierten Referenzposition
M3 / M5	Spindel oder Laser einschalten beziehungsweise ausschalten
M2 / M30	Programmende
F	Vorschubgeschwindigkeit in mm/min
S	Spindeldrehzahl oder Leistung bezogen auf den eingestellten Maximalwert

Das folgende Listing zeigt ein Beispielprogramm zur Abfahrt einer rechteckigen Kontur.

Listing 5.1: Beispielprogramm zur Abfahrt einer rechteckigen Kontur

```

1      ; Initialisierung
2      G21          ; Metrischer Modus (mm)
3      G90          ; Absolutes Koordinatensystem
4      G17          ; XY-Ebene waehlen
5
6      ; Startposition anfahren
7      G0 X0 Y0 Z5   ; Eilgang zur Startposition (Z5 mm Sicherheitshoehe)
8      M3 S1000      ; Spindel einschalten (1000 RPM)
9
10     ; Eintauchen
11     G1 Z-1 F100    ; Eintauchen auf Z-1 mm mit Vorschub 100 mm/min
12
13     ; Rechteckige Kontur (50 mm x 50 mm)
14     G1 X50 Y0 F300 ; Seite 1
15     G1 X50 Y50 F300 ; Seite 2
16     G1 X0 Y50 F300 ; Seite 3
17     G1 X0 Y0 F300 ; Zurueck zum Ursprung
18
19     ; Werkzeug anheben und Programm beenden
20     G0 Z5          ; Werkzeug auf Sicherheitshoehe
21     M5             ; Spindel ausschalten
22     M2             ; Programmende

```

5.6.2. G-Code-Datei importieren

Für die maschinelle Fertigung wird G-Code typischerweise mit CAM-Software aus einem CAD-Modell generiert und als Datei exportiert. UGS unterstützt unter anderem die Dateiformate **.nc**, **.gcode**, **.txt** und **.tap** [WU25].

Der Importprozess in UGS erfolgt schrittweise:

1. Datei über *File* und *Open* öffnen.
2. Im Datei-Dialog die G-Code-Datei auswählen und mit *Öffnen* bestätigen.
3. UGS lädt und analysiert die Datei.
4. Im Visualizer-Fenster wird der Werkzeugpfad als 2D/3D-Grafik dargestellt.
5. Vor dem Start den Werkzeug-Nullpunkt beziehungsweise Werkstück-Ursprung an der Maschine definieren und in UGS über *Zero All* zurücksetzen.
6. Maschinenstatus auf **Idle** prüfen.
7. Die Übertragung durch Klick auf die Play-Schaltfläche beziehungsweise *Send File* starten.
8. Die Befehle werden im Batch-Mode sequenziell an die Maschine übermittelt.

6. GRBL v1.1

Dieses Kapitel beschreibt die GRBL-Firmware, deren Installation auf einem Arduino Uno sowie die grundlegende Konfiguration. GRBL bildet die Echtzeitschicht der CNC-Steuerung und wird in Verbindung mit UGS als übergeordneter Bediensoftware eingesetzt [Jc19].

6.1. Beschreibung der GRBL-Firmware

GRBL ist ein quelloffener, eingebetteter G-Code-Parser und CNC-Bewegungscontroller, der in hochoptimiertem C geschrieben ist und auf Arduino-basierter Hardware ausgeführt wird. Das Projekt wurde 2009 von Simen Svale Skogsrud initiiert und wird seitdem als Community-Projekt weiterentwickelt [Jc19].

Wesentliche Merkmale der GRBL-Firmware:

- Hochperformanter G-Code-Parser: GRBL verarbeitet G-Code-Kommandos in Echtzeit und ist auf die rechenleistungsbeschränkte AVR-Architektur des ATmega328P optimiert.
- Präzise Schrittmotorsteuerung: Erzeugung stabiler, jitterfreier Steuerimpulse mit bis zu 30 kHz Impulsfrequenz durch Nutzung von Hardware-Timern des AVR-Chips.
- Asynchroner Betrieb: Trennung von Kommunikations- und Bewegungssteuerungs-Tasks ermöglicht gleichzeitigen Empfang neuer Befehle während laufender Maschinenbewegungen.
- Echtzeit-Overrides: Anpassung von Vorschubgeschwindigkeit, Eilgang-Geschwindigkeit und Spindeldrehzahl während des laufenden Betriebs ohne Programmunterbrechung.
- Laser-Modus ([\\$32=1](#)): Dynamische Anpassung der Laserleistung (PWM) proportional zur Bewegungsgeschwindigkeit für gleichmäßige Ergebnisse.
- Low-Cost-Alternative: Ersatz kostenintensiver paralleler Portlösungen (LPT-Port-basierter Motion-Controller) durch kostengünstige Arduino-Hardware.

6.2. Version

Tabelle 6.1.: Eckdaten zu GRBL v1.1

Aktuelle stabile Version	GRBL v1.1h
Veröffentlichungsdatum	25. August 2019
GitHub-Repository	https://github.com/gnea/grbl
Zielformat	Arduino mit ATmega328P (Uno, Nano, Duemilanove)
Implementierungssprache	Hochoptimiertes C mit AVR-Assembler-Optimierungen

Hinweis: Das [gnea/grbl](https://github.com/gnea/grbl)-Repository wurde 2019 abgeschlossen und erhält keine weiteren Commits. Aktive Weiterentwicklungen erfolgen in Projekten wie grblHAL und FluidNC, die auf leistungsfähigeren Mikrocontrollern basieren. Für Arduino-Uno-basierte Systeme bleibt GRBL v1.1h der empfohlene und weitverbreitete Standard [Jc19].

6.3. Installation auf dem Arduino Uno

6.3.1. Voraussetzungen

- Hardware: Arduino Uno (Original oder kompatibles Board mit ATmega328P).
- Verbindungskabel: USB-Kabel (Typ A auf Typ B für Uno).
- Software: Arduino IDE (Version 1.8.x oder 2.x), Download unter <https://www.arduino.cc/en/software>.
- GRBL-Quellcode: Download von <https://github.com/gnea/grbl/archive/master.zip>.
- USB-Treiber: CH340- oder ATmega16U2-Treiber (bei Original-Arduino in der Regel automatisch installiert).

6.3.2. Schritt-für-Schritt-Installation

1. Arduino IDE herunterladen und installieren (siehe Kapitel Arduino IDE).
2. GRBL-Repository herunterladen: Auf <https://github.com/gnea/grbl> den Button *Code* → *Download ZIP* klicken (Datei: [grbl-master.zip](#)).
3. GRBL in den Arduino-Libraries-Ordner kopieren: Das ZIP-Archiv entpacken, den Ordner [grbl](#) (nicht [grbl-master](#)) in das Arduino-Libraries-Verzeichnis kopieren:

Listing 6.1: Arduino-Libraries-Verzeichnis

```
1 C:\Users\<Benutzername>\Documents\Arduino\libraries\
```

4. Arduino Uno über USB verbinden.
5. In der Arduino IDE: Menü *File* → *Examples* → *grbl* → *grblUpload* auswählen.
6. Board konfigurieren: Menü *Tools* → *Board* → *Arduino AVR Boards* → *Arduino Uno*.
7. Port setzen: Menü *Tools* → *Port* → zugewiesenen COM-Port auswählen.
8. Sketch kompilieren: Menü *Sketch* → *Verify/Compile* (Strg+R) – auf Fehlermeldungen achten.
9. Upload durchführen: Menü *Sketch* → *Upload* (Strg+U) – der Upload dauert ca. 10–30 Sekunden.
10. Erfolgsbestätigung: In der IDE-Konsole erscheint [avrdude done. Thank you.](#) – GRBL ist nun installiert.

6.3.3. Hardware-Anschlüsse – Pin-Belegung

Die Standard-Pin-Belegung von GRBL auf dem Arduino Uno (ATmega328P):

Tabelle 6.2.: Standard-Pin-Belegung von GRBL auf dem Arduino Uno

Funktion	Arduino-Pin
X-Achse Schritt (Step)	Digital Pin 2
X-Achse Richtung (Direction)	Digital Pin 5
Y-Achse Schritt (Step)	Digital Pin 3
Y-Achse Richtung (Direction)	Digital Pin 6
Z-Achse Schritt (Step)	Digital Pin 4
Z-Achse Richtung (Direction)	Digital Pin 7
Schritt-Enable (Step Enable)	Digital Pin 8 (LOW-aktiv)
X-Endschalter (Limit Switch)	Digital Pin 9
Y-Endschalter (Limit Switch)	Digital Pin 10
Z-Endschalter (Limit Switch)	Digital Pin 11
Spindel/Laser PWM	Digital Pin 12
Spindel-Richtung	Digital Pin 13
Tastensystem (Probe)	Analog Pin A5

Die Schrittmotortreiber (z. B. A4988, DRV8825) werden typischerweise über ein CNC-Shield-Board (Arduino CNC Shield V3) angeschlossen, das die oben genannte Pin-Belegung elektrisch bereitstellt und eine einfache Schraubklemmen-Verbindung zu den Schrittmotoren ermöglicht (vgl. Kapitel CNC-Shield V3) [Jc19].

6.4. Konfiguration

6.4.1. Laufzeitkonfiguration über GRBL-Variablen

Nach dem Upload erfolgt die Maschinenkonfiguration über die `$$`-Variablen (vgl. Referenztabelle in Abschnitt 6.4.2). Eine typische Erstkonfiguration:

Listing 6.2: Typische Erstkonfiguration von GRBL

1	<code>\$100=250.000 ; X-Achse: 250 Schritte/mm</code>
2	<code>\$101=250.000 ; Y-Achse: 250 Schritte/mm</code>
3	<code>\$102=250.000 ; Z-Achse: 250 Schritte/mm</code>
4	
5	<code>\$110=3000.000 ; X-Achse max. Vorschub: 3000 mm/min</code>
6	<code>\$111=3000.000 ; Y-Achse max. Vorschub: 3000 mm/min</code>
7	<code>\$112=1000.000 ; Z-Achse max. Vorschub: 1000 mm/min</code>
8	
9	<code>\$120=300.000 ; X-Achse Beschleunigung: 300 mm/s^2</code>
10	<code>\$121=300.000 ; Y-Achse Beschleunigung: 300 mm/s^2</code>
11	<code>\$122=100.000 ; Z-Achse Beschleunigung: 100 mm/s^2</code>
12	
13	<code>\$30=1000 ; Max. Spindel-RPM (= 100% PWM)</code>
14	<code>\$31=0 ; Min. Spindel-RPM (= 0% PWM)</code>
15	<code>\$32=1 ; Laser-Modus aktivieren (bei Laserbetrieb)</code>
16	
17	<code>\$22=1 ; Homing aktivieren</code>
18	<code>\$23=0 ; Homing-Richtung (0 = negative Richtung)</code>

Wichtig: Alle über `$x=y` gesetzten Werte werden automatisch im EEPROM des ATmega328P-Chips gespeichert und bleiben nach einem Neustart erhalten. Eine explizite Speicheranweisung ist nicht erforderlich [Sai25].

6.4.2. GRBL-Variablen Referenztablelle

Die folgende Tabelle enthält die wichtigsten GRBL-Konfigurationsparameter mit deutschen Beschreibungen, typischen Startwerten und Einheiten [Sai25]:

Tabelle 6.3.: Wichtige GRBL-Konfigurationsparameter

Variable	Name	Beschreibung	Typischer Wert	Einheit
\$0	Schrittpulszeit	Dauer des Schrittpulses	10	µs
\$1	Schritt-Leerlaufverzögerung	Verzögerung bis Schrittmotoren deaktiviert werden	25	ms
\$2	Schritt-Port-Inversion	Bitmaske zur Invertierung der Schrittpulse	0	Maske
\$3	Richtungs-Port-Inversion	Bitmaske zur Invertierung der Fahrtrichtung	0	Maske
\$4	Schritt-Enable-Inversion	Invertiert das Enable-Signal der Schrittmotoren	0	Bool
\$5	Limit-Pins-Inversion	Invertiert die Endschalter-Signale (NC/NO)	0	Bool
\$6	Sonde-Pin-Inversion	Invertiert das Tastsystem-Signal	0	Bool
\$10	Statusmeldung	Bestimmt den Inhalt der Statusberichte (?)	1	Maske
\$11	Verbindungsabweichung	Max. Abweichung bei Bogenbewegungen	0.010	mm
\$12	Bogensschritt-Auflösung	Auflösung für Bogenbewegungen (G2/G3)	0.002	mm
\$13	Zoll-Modus	Statusberichte in Zoll statt Millimeter	0	Bool
\$20	Soft-Limits	Aktiviert Software-Endanschläge	0	Bool
\$21	Hard-Limits	Aktiviert Hardware-Endschalter als Schutz	0	Bool
\$22	Homing-Zyklus	Aktiviert den automatischen Referenzpunkt-Zyklus	0	Bool
\$23	Homing-Richtung	Bitmaske: Homing-Richtung der Achsen invertieren	0	Maske
\$24	Homing-Geschwindigkeit	Langsame Geschwindigkeit beim Homing	25.000	mm/min
\$25	Homing-Suchgeschw.	Schnelle Suchgeschwindigkeit beim Homing	500.000	mm/min
\$26	Homing-Entprellung	Entprellzeit für Endschalter beim Homing	250	ms
\$27	Homing-Rückzug	Rückzugdistanz nach Endschalter beim Homing	1.000	mm
\$30	Max. Spindel-RPM	Maximale Spindeldrehzahl (= 100% PWM)	1000	U/min
\$31	Min. Spindel-RPM	Minimale Spindeldrehzahl (kleinste PWM)	0	U/min
\$32	Laser-Modus	Aktiviert den Laser-Modus (0 = aus, 1 = an)	0	Bool
\$100	X-Schritte/mm	Anzahl der Motorschritte pro Millimeter X-Achse	250.000	Schritte/mm

Variable	Name	Beschreibung	Typischer Wert	Einheit
\$101	Y-Schritte/mm	Anzahl der Motorschritte pro Millimeter Y-Achse	250.000	Schritte/mm
\$102	Z-Schritte/mm	Anzahl der Motorschritte pro Millimeter Z-Achse	250.000	Schritte/mm
\$110	Max. X-Vorschub	Maximale Vorschubgeschwindigkeit der X-Achse	500.000	mm/min
\$111	Max. Y-Vorschub	Maximale Vorschubgeschwindigkeit der Y-Achse	500.000	mm/min
\$112	Max. Z-Vorschub	Maximale Vorschubgeschwindigkeit der Z-Achse	500.000	mm/min
\$120	X-Beschleunigung	Beschleunigung der X-Achse	10.000	mm/s ²
\$121	Y-Beschleunigung	Beschleunigung der Y-Achse	10.000	mm/s ²
\$122	Z-Beschleunigung	Beschleunigung der Z-Achse	10.000	mm/s ²
\$130	X-Max. Verfahrensweg	Maximaler Verfahrensweg der X-Achse (Soft-Limits)	200.000	mm
\$131	Y-Max. Verfahrensweg	Maximaler Verfahrensweg der Y-Achse (Soft-Limits)	200.000	mm
\$132	Z-Max. Verfahrensweg	Maximaler Verfahrensweg der Z-Achse (Soft-Limits)	200.000	mm

6.4.3. Diagnose und Fehlerbehebung

GRBL stellt folgende Diagnose-Schnittstellen zur Verfügung [Sai25]:

- Statusabfrage: Eingabe von [?](#) – Antwort im Format [<Status|MPos:x,y,z|FS:Vorschub,Spindel>](#).
- Homing-Zyklus testen: Eingabe von [\\$H](#) – GRBL fährt alle Achsen an die Endschalter-Position und setzt den Maschinennullpunkt.
- Software-Reset: [Ctrl+X](#) (Byte 0x18) – setzt GRBL ohne Hardware-Neustart in den Ausgangszustand zurück.
- G-Code-Parser-Status: Eingabe von [\\$G](#) – gibt alle aktiven modalen G-Code-Zustände aus.
- Alarm freigeben: Eingabe von [\\$X](#) – hebt einen Alarmzustand auf.

Häufige Alarm-Codes und deren Bedeutung:

Tabelle 6.4.: Häufige GRBL-Alarm-Codes

Alarm-Code	Ursache
ALARM:1	Hard Limit ausgelöst – Endschalter während Bewegung aktiviert
ALARM:2	Soft Limit ausgelöst – Bewegung überschreitet \$130-\$132
ALARM:3	Reset bei aktiver Maschinenbewegung
ALARM:6	Probe fehlgeschlagen – kein Tastersignal empfangen
ALARM:9	Homing-Fehler – Endschaltersignal nicht empfangen (\$5 prüfen)

Teil II.

Hardware

7. Arduino Uno R3

7.1. Arduino Allgemein

Arduino ist eine open-source-electronic-prototyping-plattform für Mikrocontroller. Sie besteht aus programmierbarer Hardware und einer Entwicklungsumgebung, mit der Programme geschrieben und auf ein Board übertragen werden können [Ard26a]. Arduino-Boards werden für Steuerungen, Messaufgaben, Prototypen und Lernprojekte verwendet [Ard26c].

Ein Arduino kann Eingangssignale verarbeiten und Ausgangssignale erzeugen. Eingangssignale können zum Beispiel von Tastern, Sensoren oder Modulen kommen. Ausgangssignale können LEDs, Motoren, Displays oder andere Bauteile steuern [Ard25c]. Die Programmierung erfolgt meist über die Arduino IDE. Das Programm wird am Computer geschrieben, über USB auf das Board übertragen und danach direkt vom Mikrocontroller ausgeführt [Ard25c]. Arduino-Programme verwenden Funktionen wie `setup()` und `loop()` [Ard26a].

7.2. Arduino Uno R3

Der Arduino Uno ist ein Mikrocontroller-Board auf Basis des ATmega328P. Er besitzt 14 digitale Ein- und Ausgänge, davon 6 mit PWM-Funktion (Pulsweitenmodulation-Funktion), 6 analoge Eingänge, eine USB-Schnittstelle, eine Power-Buchse, einen ICSP-Header (In-Circuit Serial Programming) und einen Reset-Taster [Ard26c][Ard26b].

PWM bedeutet Pulsweitenmodulation und beschreibt ein sehr schnelles Ein- und Ausschalten eines digitalen Ausgangspins, wodurch zum Beispiel LEDs gedimmt oder Motoren gesteuert werden können.

Der ICSP-Header ist eine Stiftleiste zur direkten Programmierung des Mikrocontrollers, ohne den Mikrocontroller dafür aus der Schaltung zu entfernen.

Wichtige Eigenschaften:

- Mikrocontroller: ATmega328P
- Betriebsspannung: 5 V
- Empfohlene Eingangsspannung: 7 V bis 12 V
- Digitale Ein- und Ausgangspins: 14
- PWM-Ausgangspins: 6
- Analoge Eingänge: 6
- Flash-Speicher: 32 KB
- SRAM: 2 KB
- EEPROM: 1 KB
- Taktfrequenz: 16 MHz [Ard26b]

7.3. Module

7.3.1. Mikrocontroller ATmega328P

Der ATmega328P ist der Hauptbaustein des Arduino Uno. Er führt das hochgeladene Programm aus, verarbeitet Eingangssignale und steuert Ausgangssignale [Ard26b]. Auf diesem Mikrocontroller laufen später die Funktionen `setup()` und `loop()` [Ard26a]. Der Mikrocontroller enthält Speicher für das Programm und Speicher für Daten während der Ausführung. Das Programm bleibt auch nach dem Ausschalten gespeichert, weil es im Flash-Speicher abgelegt wird [Ard26b].

Eigenschaften

- 8-Bit-Mikrocontroller
- Taktfrequenz: 16 MHz
- Flash-Speicher: 32 KB
- SRAM: 2 KB
- EEPROM: 1 KB [Ard26b]
- Führt den Arduino-Code aus [Ard26a]

7.3.2. Eingebaute LED

Der Arduino Uno besitzt eine eingebaute LED. Diese LED ist mit dem digitalen Ausgangspin **D13** verbunden [Ard25c]. **D13** wird dafür als Ausgangspin verwendet. Sie kann für erste Funktionstests verwendet werden [Ard26c].

Die LED kann direkt über den Ausgangspin **D13** geschaltet werden. Dafür wird **D13** als Ausgangspin verwendet. Wird der Ausgangspin auf **HIGH** gesetzt, leuchtet die LED. Wird der Ausgangspin auf **LOW** gesetzt, ist die LED ausgeschaltet [Ardf].

Die eingebaute LED eignet sich besonders für das Blink-Beispiel und einfache Tests mit `digitalWrite()` [Ardf].

7.3.3. Reset-Taster

Der Reset-Taster ist auf dem Arduino Uno eingebaut [Ard26c]. Er startet den Mikrocontroller neu. Dabei wird das gespeicherte Programm nicht gelöscht [Ard25c].

Ein Reset ist hilfreich, wenn ein Programm neu gestartet werden soll oder sich das Board nicht wie erwartet verhält [Ard25c].

Zuerst wird `setup()` einmal ausgeführt. Danach läuft `loop()` dauerhaft weiter [Ard26a].

7.3.4. USB-Schnittstelle

Die USB-Schnittstelle verbindet den Arduino Uno mit dem Computer [Ard25c]. Über USB wird das Programm auf das Board übertragen. Außerdem kann der Arduino Uno darüber mit Spannung versorgt werden [Ard26b].

Die USB-Verbindung wird auch für den seriellen Monitor verwendet. Damit können Werte und Zustände vom Arduino an den Computer gesendet werden [Ard26a].

Für Ausgaben im seriellen Monitor werden Funktionen wie `Serial.begin()`, `Serial.print()` und `Serial.println()` verwendet [Ard26a].

7.3.5. Spannungsversorgung

Der Arduino Uno kann über USB-B, eine DC-Hohlsteckerbuchse oder den Versorgungseingang **VIN** mit Spannung versorgt werden [Ard26b]. Auf dem Board befinden sich Spannungsregler, die die benötigte Betriebsspannung bereitstellen [Ard25c].

Die Betriebsspannung des Arduino Uno beträgt 5 V. Für eine externe Spannungsversorgung wird eine Eingangsspannung von 7 V bis 12 V empfohlen [Ard26b].

Für einfache Tests reicht meistens die Spannungsversorgung über USB [Ard25c]. Für eigenständige Projekte kann eine externe Spannungsquelle über die Power-Buchse oder den Versorgungseingang **VIN** verwendet werden [Ard26b].

Die Anschlüsse **5V** und **3.3V** sind Versorgungsausgänge und können kleine externe Bauteile mit Spannung versorgen [Ard26b]. Große Verbraucher wie Motoren oder starke LEDs sollten nicht direkt über diese Versorgungsausgänge betrieben werden [Ard25c].

7.4. Anschlüsse des Arduino Uno

Die Anschlüsse sind die elektrischen Verbindungen des Arduino Uno. Über diese Anschlüsse wird der Mikrocontroller mit externen Bauteilen verbunden. Der Arduino Uno besitzt 14 digitale Ein- und Ausgänge, 6 analoge Eingangspins und mehrere Anschlüsse für Spannungsversorgung und Kommunikation [Ard26c][Ard26b].

Der Arduino Uno arbeitet mit einer Betriebsspannung von 5 V. Deshalb verwenden die digitalen Ein- und Ausgangspins 5-V-Logikpegel. Eine zu hohe Spannung an einem Eingangspin kann das Board beschädigen [Ard26b].

7.4.1. Digitale Ein- und Ausgangspins

Die digitalen Ein- und Ausgangspins des Arduino Uno sind mit **D0** bis **D13** beschriftet. Sie können je nach Programmierung als Eingangspins oder Ausgangspins verwendet werden [Ard26c][Ard26b].

Digitale Ein- und Ausgangspins arbeiten mit zwei Zuständen: **HIGH** und **LOW**. Bei einem Ausgangspin entspricht **HIGH** einem hohen Spannungspegel und **LOW** einem niedrigen Spannungspegel [Ardf]. Beim Arduino Uno liegt der Logikpegel bei 5 V [Ard26b].

Ein digitaler Anschluss wird im Programm mit `pinMode()` als Eingangspin oder Ausgangspin festgelegt [Ardi]. Danach kann der Anschluss gelesen oder gesetzt werden. Als Eingangspin wird ein digitaler Anschluss zum Beispiel für Taster oder digitale Signale verwendet. Der Zustand wird mit `digitalRead()` ausgelesen [Arde].

Als Ausgangspin kann ein digitaler Anschluss LEDs, Steuersignale oder kleine Lasten ansteuern. Der Zustand wird mit `digitalWrite()` auf **HIGH** oder **LOW** gesetzt [Ardf]. Codebeispiel wären für die Ausgangspins 7.3 und für die Eingangspins 7.5 wie auch 10.2. Kombiniert eingesetzt werden Aus- und Eingangspin in 7.4.

7.4.2. PWM-Ausgangspins

Der Arduino Uno besitzt 6 digitale Anschlüsse, die als PWM-Ausgangspins verwendet werden können [Ard26c]. PWM steht für Pulsweitenmodulation. Dabei wird ein digitaler Ausgangspin sehr schnell ein- und ausgeschaltet [Ard26e].

PWM wird verwendet, wenn ein Ausgangspin nicht nur ein- oder ausgeschaltet werden soll. Anwenden kann man es für das Dimmen einer LED oder das Steuern der Geschwindigkeit eines Motors über einen Motortreiber [Ard26e].

Beim Arduino Uno unterstützen die digitalen Anschlüsse **D3**, **D5**, **D6**, **D9**, **D10** und **D11** PWM [Ard26c][Ard26b]. Diese Anschlüsse werden bei PWM als Ausgangspins verwendet und sind auf dem Board meist mit einem Tilde-Zeichen markiert [Ard25c]. PWM wird im Programm mit `analogWrite()` genutzt [Ard26e]. Obwohl die Funktion `analogWrite()` heißt, wird dabei kein echter analoger Spannungswert ausgegeben. Es

handelt sich um ein digitales PWM-Signal [Ard26e].

Der Wert bei `analogWrite()` liegt zwischen 0 und 255. Der Wert 0 bedeutet dauerhaft aus. Der Wert 255 bedeutet dauerhaft an. Werte dazwischen erzeugen ein schnelles Ein- und Ausschalten [Ard26e].

Beispielcode

In diesem Beispiel wird eine LED über einen PWM-Ausgangspin gedimmt. Dafür wird die LED an den digitalen PWM-Ausgangspin **D9** angeschlossen. Der lange Anschluss der LED ist die Anode und wird über einen Vorwiderstand mit **D9** verbunden. Der kurze Anschluss der LED ist die Kathode und wird mit **GND** verbunden.

Im Programm wird **D9** mit `pinMode()` als Ausgangspin festgelegt. Anschließend wird mit `analogWrite()` ein PWM-Signal ausgegeben. Obwohl die Funktion `analogWrite()` heißt, wird beim Arduino Uno kein echter analoger Spannungswert erzeugt. Stattdessen wird der Ausgangspin sehr schnell ein- und ausgeschaltet.

Der Wert bei `analogWrite()` liegt zwischen 0 und 255. Der Wert 0 bedeutet, dass die LED ausgeschaltet ist. Der Wert 255 bedeutet, dass die LED dauerhaft eingeschaltet ist. Werte dazwischen verändern das Verhältnis zwischen Ein- und Ausschaltzeit und dadurch die wahrgenommene Helligkeit der LED.

Dieses Beispiel zeigt die Verwendung eines PWM-Ausgangspins zur Helligkeitssteuerung einer LED.

Code

Listing 7.1.: LED dimmen mit PWM

```

1000 /**
1001  * @file LEDDimmenPWM.ino
1002  * @brief Dims an LED using a PWM output pin.
1003  *
1004  * The LED is connected to digital PWM output pin D9.
1005  * The brightness is changed with analogWrite().
1006  */
1007
1008 const int ledPin = 9; ///< Digital PWM output pin connected to the LED.
1009
1010 /**
1011  * @brief Configures the LED pin as an output.
1012  */
1013 void setup()
1014 {
1015     pinMode(ledPin, OUTPUT);
1016 }
1017
1018 /**
1019  * @brief Increases and decreases the LED brightness repeatedly.
1020  */
1021 void loop()
1022 {
1023     for (int brightness = 0; brightness <= 255; brightness++)
1024     {
1025         analogWrite(ledPin, brightness);
1026         delay(10);
1027     }
1028
1029     for (int brightness = 255; brightness >= 0; brightness--)
1030     {
1031         analogWrite(ledPin, brightness);
1032         delay(10);
1033     }
1034 }

```

../Code/LEDDimmenPWM/LEDDimmenPWM.ino

7.4.3. Analoge Eingänge

Die analogen Eingänge des Arduino Uno sind mit **A0** bis **A5** beschriftet. Sie werden normalerweise als Eingangspins verwendet, messen Spannungen und wandeln diese in digitale Werte um [Ard26c][Ard26d].

Der Arduino Uno besitzt einen Analog-Digital-Wandler. Dieser wandelt eine Eingangsspannung in einen Zahlenwert um. Der Wertebereich bei `analogRead()` reicht von 0 bis 1023 [Ard26d].

Analoge Eingänge werden mit `analogRead()` ausgelesen. Ein Messwert von 0 entspricht ungefähr 0 V. Ein Messwert von 1023 entspricht ungefähr der verwendeten Referenzspannung [Ard26d].

Analoge Eingänge werden zum Beispiel für Potentiometer, Fotowiderstände, analoge Temperatursensoren oder Spannungsteiler verwendet [Ard26d].

Die analogen Anschlüsse können bei Bedarf auch als digitale Eingangspins oder Ausgangspins verwendet werden [Ard26b]. Dann entsprechen sie zusätzlichen digitalen Eingangspins oder Ausgangspins.

7.4.4. Versorgungsanschlüsse

Die Versorgungsanschlüsse liefern Spannungen oder dienen als Masseanschluss. Sie werden verwendet, um kleine externe Schaltungen, Sensoren oder Module mit Strom zu versorgen [Ard26b].

Diese Versorgungsanschlüsse sind keine normalen Eingangspins oder Ausgangspins. Sie dienen nur zur Stromversorgung oder als Bezugspunkt [Ard25c].

Der Anschluss **5V** liefert die geregelte 5-V-Spannung des Boards. Der Anschluss **3.3V** liefert eine 3,3-V-Spannung für kleine Verbraucher. **GND** ist der Masseanschluss. **VIN** ist ein Versorgungseingang. Über diesen Anschluss kann der Arduino Uno mit einer externen Spannung versorgt werden [Ard26b]. Bei allen angeschlossenen Bauteilen muss **GND** mit dem Arduino verbunden sein. Ohne gemeinsame Masse können Signale nicht zuverlässig erkannt werden [Ard25c].

7.4.5. Serielle Ein- und Ausgangspins D0 und D1

Die Anschlüsse **D0** und **D1** sind digitale Ein- und Ausgangspins mit einer zusätzlichen Funktion. Sie werden für die serielle Kommunikation verwendet [Ard26b].

D0 ist der Empfangspin **RX** und wird als Eingangspin für serielle Daten verwendet. **D1** ist der Sendepin **TX** und wird als Ausgangspin für serielle Daten verwendet. Über diese Verbindung kommuniziert der Arduino Uno unter anderem mit dem Computer [Ard26b][Ard26a].

Die serielle Kommunikation wird zum Beispiel für den seriellen Monitor verwendet. Im Programm wird sie mit `Serial.begin()` gestartet. Danach können Daten mit `Serial.print()` oder `Serial.println()` gesendet werden [Ard26a].

Da **D0** und **D1** auch für die USB-Kommunikation verwendet werden, sollten sie bei einfachen Projekten möglichst frei bleiben. Angeschlossene Bauteile können sonst das Hochladen von Programmen oder die serielle Ausgabe stören [Ard25c].

Beispielcodes hierfür wären 7.5, 10.2 wie auch 7.4.

7.4.6. I2C-Kommunikationsanschlüsse

I2C ist eine Schnittstelle zur Kommunikation mit mehreren Bauteilen über zwei Datenleitungen. Beim Arduino Uno werden dafür die Kommunikationsanschlüsse **A4/SDA** und **A5/SCL** verwendet [Ard26b].

SDA ist die Datenleitung und arbeitet als bidirektionale Kommunikationsleitung. **SCL** ist die Taktleitung und wird beim Arduino Uno meist als Ausgang für das Taktsignal verwendet. Über diese beiden Leitungen können zum Beispiel Displays,

Sensoren oder Erweiterungsmodule angesprochen werden [Ard26a].

Ein **I2C**-Bauteil wird mit **SDA**, **SCL**, Versorgungsspannung und **GND** verbunden [Ard26b]. Mehrere **I2C**-Bauteile können an denselben zwei Leitungen angeschlossen werden, wenn sie unterschiedliche Adressen besitzen [Ard26a].

Im Programm kann **I2C** über die Bibliothek **Wire** verwendet werden [Ard26a].

Ein Beispielcode wäre hierfür ?? wie auch ??.

7.4.7. SPI-Kommunikationsanschlüsse

SPI ist eine schnelle Schnittstelle zur Kommunikation mit Bauteilen wie Displays, Speicherbausteinen oder Funkmodulen [Ard26a]. Beim Arduino Uno liegen die SPI-Signale auf den Kommunikationsanschlüssen **D10** bis **D13** [Ard26b].

SPI verwendet mehrere Leitungen. Dadurch ist die Verbindung schneller als einfache digitale Signale, benötigt aber mehr Anschlüsse [Ard26a].

Die wichtigsten SPI-Anschlüsse beim Arduino Uno sind:

- **D10: SS** oder **CS**, meist Ausgangspin zur Auswahl eines Bauteils
- **D11: MOSI**, Ausgangspin für Daten vom Arduino zum Bauteil
- **D12: MISO**, Eingangspin für Daten vom Bauteil zum Arduino
- **D13: SCK**, Ausgangspin für das Taktsignal [Ard26b]

MOSI sendet Daten vom Arduino zum Bauteil und ist deshalb ein Ausgangssignal. **MISO** empfängt Daten vom Bauteil und ist deshalb ein Eingangssignal. **SCK** ist die Taktleitung und wird beim Arduino als Ausgang verwendet. **SS** oder **CS** wählt das jeweilige Bauteil aus und wird meist als Ausgang verwendet [Ard26a].

Beispielcode

In diesem Beispiel wird die SPI-Kommunikation mit einem einfachen Testprogramm gezeigt. Beim Arduino Uno werden für SPI die Kommunikationsanschlüsse **D10**, **D11**, **D12** und **D13** verwendet. **D10** wird als **SS** oder **CS** verwendet, **D11** ist **MOSI**, **D12** ist **MISO** und **D13** ist **SCK**.

Für den Test wird ein SPI-Bauteil mit dem Arduino verbunden. Der Anschluss **MOSI** sendet Daten vom Arduino zum Bauteil. Der Anschluss **MISO** empfängt Daten vom Bauteil. Der Anschluss **SCK** gibt das Taktsignal vor. Der Anschluss **SS** oder **CS** wählt das angeschlossene Bauteil aus.

Im Programm wird die SPI-Schnittstelle mit **SPI.begin()** gestartet. Der Chip-Select-Anschluss wird mit **pinMode()** als Ausgangspin festgelegt. Vor der Datenübertragung wird **CS** auf **LOW** gesetzt. Dadurch wird das SPI-Bauteil ausgewählt. Danach wird mit **SPI.transfer()** ein Datenbyte übertragen. Nach der Übertragung wird **CS** wieder auf **HIGH** gesetzt.

Dieses Beispiel zeigt die grundlegende Verwendung der SPI-Kommunikationsanschlüsse des Arduino Uno.

Code

Listing 7.2.: SPI Grundtest

```

1000 /**
      * @file SPIGrundtest.ino
1002  * @brief Basic SPI communication test for the Arduino Uno.
      *
1004  * The Arduino Uno uses D10 as CS/SS, D11 as MOSI,
      * D12 as MISO and D13 as SCK.
1006  */

```

```

1008 #include <SPI.h>
1010 const int chipSelectPin = 10; ///< Digital output pin used as chip
      select.
1012 /**
      * @brief Initializes SPI and configures the chip select pin.
1014 */
void setup()
1016 {
    pinMode(chipSelectPin, OUTPUT);
1018     digitalWrite(chipSelectPin, HIGH);

    SPI.begin();
1020
    Serial.begin(115200);
    Serial.println("SPI Grundtest gestartet");
1024 }
1026 /**
      * @brief Sends one byte over SPI repeatedly.
1028 */
void loop()
1030 {
    byte sendData = 0x55;
1032
    digitalWrite(chipSelectPin, LOW);
1034     byte receivedData = SPI.transfer(sendData);
    digitalWrite(chipSelectPin, HIGH);
1036
    Serial.print("Gesendet: ");
1038     Serial.print(sendData, HEX);
    Serial.print(" Empfangen: ");
1040     Serial.println(receivedData, HEX);
1042
    delay(1000);
}

```

../..Code/SPIGrundtest/SPIGrundtest.ino

7.4.8. AREF-Referenzeingang

Der Anschluss **AREF** steht für Analog Reference. Er ist ein Referenzeingang und kann als externe Referenzspannung für die analogen Eingänge verwendet werden [Ard26b]. Normalerweise verwendet der Arduino Uno eine Standardreferenz für `analogRead()`. Mit **AREF** kann eine andere Referenzspannung verwendet werden, wenn genauere Messungen in einem kleineren Spannungsbereich benötigt werden [Ard26d].

Der **AREF**-Referenzeingang wird nur für spezielle analoge Messungen verwendet. In einfachen Projekten bleibt dieser Referenzeingang normalerweise unbenutzt [Ard25c]. Wenn eine externe Referenzspannung verwendet wird, muss sie im Programm korrekt eingestellt werden. Dafür wird `analogReference()` genutzt [Ard26d].

7.4.9. RESET-Eingang

Der **RESET**-Eingang kann den Mikrocontroller neu starten. Er ist ein spezieller Eingangspin und hat dieselbe Grundfunktion wie der Reset-Taster auf dem Board [Ard26b][Ard25c].

Wird der **RESET**-Eingang kurz mit **GND** verbunden, startet der Arduino Uno neu. Das gespeicherte Programm bleibt erhalten und beginnt wieder bei `setup()` [Ard25c][Ard26a].

7.5. Beispiel Code

7.5.1. Manual

Ein einfaches Beispiel ist das Blinken der eingebauten LED. Dafür wird die LED am Ausgangspin **D13** verwendet [Ard25c]. **D13** wird dabei als Ausgangspin genutzt. Da diese LED bereits auf dem Arduino Uno verbaut ist, muss keine externe LED angeschlossen werden.

Der Arduino Uno muss nur über USB mit dem Computer verbunden sein. Die Stromversorgung und das Hochladen des Programms erfolgen über das USB-Kabel.

Der Anschluss wird in `setup()` mit `pinMode()` als Ausgangspin festgelegt [Ardi]. In `loop()` wird die LED mit `digitalWrite()` ein- und ausgeschaltet [Ardf].

Mit `delay()` wird eine Pause zwischen den Zuständen erzeugt. Dadurch blinkt die LED sichtbar [Ard26a].

7.5.2. Code

Listing 7.3.: Blink Internal LED

```

1000 /**
1001  * @file BlinkInternalLED.ino
1002  * @brief Blinks the built-in LED on the Arduino Uno.
1003  *
1004  * The Arduino Uno has a built-in LED connected to digital pin D13.
1005  * No external LED or resistor is required for this test.
1006  * The LED is switched on and off with a fixed delay.
1007  */
1008
1009 const int ledPin = 13; ///< Digital output pin connected to the built-
1010                        in LED.
1011
1012 /**
1013  * @brief Configures the built-in LED pin as an output.
1014  */
1015 void setup() {
1016     pinMode(ledPin, OUTPUT);
1017 }
1018
1019 /**
1020  * @brief Switches the built-in LED on and off repeatedly.
1021  */
1022 void loop() {
1023     digitalWrite(ledPin, HIGH);
1024     delay(1000);
1025
1026     digitalWrite(ledPin, LOW);
1027     delay(1000);
1028 }

```

../..Code/BlinkInternalLED/BlinkInternalLED.ino

7.6. Anwendung

7.6.1. Manual

In dieser Anwendung wird ein einfacher Reaktionstest mit einer LED und einem Druckknopf aufgebaut. Die LED zeigt an, wann der Benutzer reagieren soll. Sobald die LED leuchtet, muss der Druckknopf möglichst schnell gedrückt werden.

Der Druckknopf wird zwischen den digitalen Eingangspin **D2** und **GND** angeschlossen. **D2** wird dabei als Eingangspin verwendet. Im Programm wird der interne Pull-up-

Widerstand mit **INPUT PULLUP** verwendet. Dadurch wird kein externer Widerstand für den Druckknopf benötigt.

Die LED wird an den digitalen Ausgangspin **D13** angeschlossen. **D13** wird dabei als Ausgangspin verwendet. Alternativ kann auch die eingebaute LED des Arduino Uno verwendet werden, da diese ebenfalls mit dem Ausgangspin **D13** verbunden ist.

Nach dem Start wartet der Arduino eine zufällige Zeit. Danach wird die LED eingeschaltet. Wenn der Druckknopf gedrückt wird, erkennt der Arduino den Tastendruck mit **digitalRead()**. Im seriellen Monitor wird angezeigt, dass der Druckknopf gedrückt wurde. Danach beginnt der Test erneut.

Die Anwendung zeigt, wie ein Eingangssignal von einem Druckknopf ausgewertet und mit einer Ausgabe über LED und seriellen Monitor verbunden wird.

7.6.2. Code

Listing 7.4.: Reaktionstest mit LED und Druckknopf

```

1000 /**
1001  * @file ReaktionstestDruckknopf.ino
1002  * @brief Simple reaction test using an LED and a push button.
1003  */
1004
1005 /** @brief Digital input pin for the push button. */
1006 const int druckknopfPin = 2;
1007
1008 /** @brief Digital output pin for the LED. */
1009 const int ledPin = 13;
1010
1011 /** @brief Stores the state of the push button. */
1012 int druckknopfZustand = HIGH;
1013
1014 /**
1015  * @brief Initializes the pins and the serial monitor.
1016  */
1017 void setup()
1018 {
1019     pinMode(druckknopfPin, INPUT_PULLUP);
1020     pinMode(ledPin, OUTPUT);
1021
1022     Serial.begin(9600);
1023
1024     digitalWrite(ledPin, LOW);
1025
1026     randomSeed(analogRead(A0));
1027
1028     Serial.println("Reaktionstest gestartet");
1029     Serial.println("Druecke den Druckknopf, sobald die LED leuchtet.");
1030 }
1031
1032 /**
1033  * @brief Runs the reaction test repeatedly.
1034  */
1035 void loop()
1036 {
1037     Serial.println("Warten...");
1038     digitalWrite(ledPin, LOW);
1039
1040     delay(random(2000, 6000));
1041
1042     Serial.println("Jetzt druecken!");
1043     digitalWrite(ledPin, HIGH);
1044
1045     while (digitalRead(druckknopfPin) == HIGH)
1046     {
1047         // Warten, bis der Druckknopf gedrueckt wird.
1048     }

```

```

1050     digitalWrite(ledPin, LOW);
1052     Serial.println("Druckknopf wurde gedrueckt.");
1053     Serial.println("Naechster Durchlauf startet gleich.");
1054     Serial.println();
1056     delay(2000);
}

```

../../Code/ReaktionstestDruckknopf/ReaktionstestDruckknopf.ino

Verwendete Arduino-Befehle

- `setup()`: Wird einmal beim Start des Arduino ausgeführt.
- `loop()`: Wird nach `setup()` dauerhaft wiederholt.
- `pinMode()`: Legt fest, ob ein Anschluss als Eingangspin oder Ausgangspin verwendet wird.
- `INPUT PULLUP`: Aktiviert den internen Pull-up-Widerstand für den Druckknopf.
- `digitalRead()`: Liest den Zustand des Druckknopfes aus.
- `digitalWrite()`: Schaltet die LED ein oder aus.
- `Serial.begin()`: Startet die serielle Kommunikation.
- `Serial.println()`: Gibt Text im seriellen Monitor aus.
- `delay()`: Erzeugt eine Pause im Programm.
- `random()`: Erzeugt eine zufällige Wartezeit.
- `randomSeed()`: Startet den Zufallsgenerator mit einem veränderlichen Startwert.
- `analogRead()`: Liest einen analogen Wert ein, der hier als Startwert für den Zufallsgenerator genutzt wird.

7.7. Testcode

7.7.1. Manual

Der Testcode prüft, ob der Arduino einen Tastendruck erkennt. Der Taster wird zwischen den digitalen Eingangspin **D2** und **GND** angeschlossen. **D2** wird dabei als Eingangspin verwendet. Eine Seite des Tasters kommt an **D2**, die andere Seite an **GND**.

Im Programm wird der Eingangspin mit `pinMode()` als `INPUT PULLUP` eingestellt. Dadurch wird kein externer Widerstand benötigt. Der Arduino liest bei nicht gedrücktem Taster den Zustand `HIGH`. Beim Drücken verbindet der Taster **D2** mit **GND**, wodurch der Zustand `LOW` gelesen wird.

Der Zustand wird mit `digitalRead()` ausgelesen und mit `Serial.println()` im seriellen Monitor angezeigt. `delay()` erzeugt eine Pause im Programm [Ardi][Arde][Ard26a].

7.7.2. Code

Listing 7.5.: Test Button

```
1000 \begin{lstlisting}[language=C++]
1001 /**
1002  * @file TestButton.ino
1003  * @brief Tests a push button connected to digital pin D2.
1004  *
1005  * The button is connected between digital pin D2 and GND.
1006  * The internal pull-up resistor is enabled with INPUT_PULLUP.
1007  * Therefore, the input reads HIGH when the button is not pressed
1008  * and LOW when the button is pressed.
1009  */
1010 const int buttonPin = 2; ///< Digital input pin connected to the button
1011
1012 /**
1013  * @brief Initializes serial communication and configures the button pin
1014  */
1015 void setup() {
1016     Serial.begin(115200);
1017     pinMode(buttonPin, INPUT_PULLUP);
1018 }
1019
1020 /**
1021  * @brief Reads the button state and prints it to the serial monitor.
1022  */
1023 void loop() {
1024     int buttonState = digitalRead(buttonPin);
1025
1026     if (buttonState == LOW) {
1027         Serial.println("Button pressed");
1028     } else {
1029         Serial.println("Button not pressed");
1030     }
1031
1032     delay(200);
1033 }
1034 \end{lstlisting}
```

../Code/TestButton/TestButton.ino

8. CNC Shield V3

8.1. CNC Shields / CNC-Controllerboards

CNC Shields sind Erweiterungsboards für Arduino-kompatible Steuerungen, die die Ansteuerung von Schrittmotoren in CNC-Anwendungen vereinfachen. Sie bilden die Schnittstelle zwischen Mikrocontroller und Motortreibern und werden typischerweise in CNC-Fräsen, Plottern oder vergleichbaren Achssystemen eingesetzt [Joy22].

Zur Beschreibung des verwendeten Shields werden vor allem das Datenblatt und die Anleitung des Herstellers verwendet, da diese die technischen Spezifikationen, den Aufbau und die Anschlussbelegung der Komponente enthalten [Joy22][Joy23].

CNC-Systeme werden allgemein eingesetzt, um Bewegungen von Werkzeugen oder Werkstücken rechnergestützt und reproduzierbar auszuführen. Dabei werden Bewegungsdaten in Steuersignale für die Achsantriebe umgesetzt [Heh20].

8.2. Einleitung

Das CNC Shield V3 dient zur einfachen Ansteuerung mehrerer Schrittmotoren über einen Arduino Uno. Es wird direkt auf den Arduino aufgesteckt und stellt Steckplätze für Schrittmotortreiber sowie Anschlüsse für Motoren, Endschalter und weitere CNC-Funktionen bereit.

Im Projekt wird das CNC Shield V3 zur Ansteuerung der Achsmotoren des Heißdraht Schneiders verwendet. Die Schrittmotoren bewegen den Heißdraht entlang der gewünschten Kontur. Dadurch ist das Shield eine zentrale Schnittstelle zwischen dem Arduino Uno, den Motortreibern und den mechanischen Achsen.

Das verwendete CNC Shield V3 ist ein Controllerboard für CNC-Anwendungen mit vier A4988-Motortreibern. Es ist für den Einsatz mit dem Arduino Uno oder kompatiblen Boards vorgesehen. Das Shield unterstützt bis zu vier Achsen und ist GRBL-kompatibel, wodurch es sich in standardisierte CNC-Steuerungen integrieren lässt [Joy22][Joy23].

8.3. Grundlagen

Ein CNC Shield erweitert den Arduino Uno um Anschlüsse und Steckplätze, die für kleinere CNC-Anwendungen benötigt werden. Der Arduino übernimmt dabei die Erzeugung der Steuersignale. Das Shield verteilt diese Signale auf die einzelnen Achsen und stellt die Verbindung zu den Schrittmotortreibern her.

Für die Bewegung der Achsen werden Schrittmotoren verwendet. Die eigentliche Leistungsansteuerung übernimmt nicht das CNC Shield selbst, sondern der jeweilige Schrittmotortreiber. Das Shield stellt dafür die STEP-, DIR- und ENABLE-Signale bereit. Über das STEP-Signal werden Schrittimpulse erzeugt, über das DIR-Signal wird die Drehrichtung festgelegt und über ENABLE werden die Motortreiber aktiviert oder deaktiviert.

Das CNC Shield V3 ist für den Einsatz mit steckbaren Motortreibermodulen wie dem A4988 oder kompatiblen Treibern vorgesehen. Die Motortreiber werden auf die dafür vorgesehenen Steckplätze des Shields gesetzt. Dadurch können die einzelnen Achsen X, Y, Z und A separat angesteuert werden [Joy23].

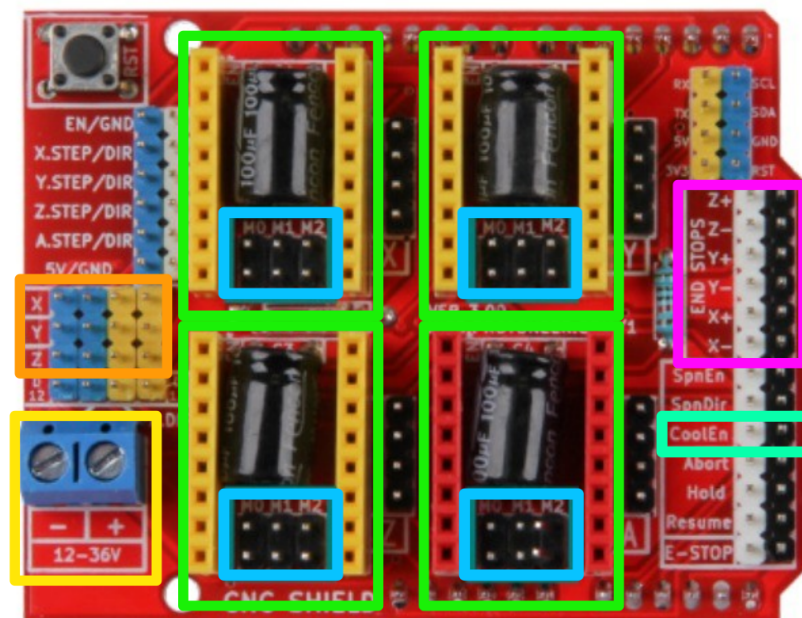
8.4. Spezifisches CNC Shield V3

Im Projekt wird ein CNC Shield V3 für Arduino verwendet. Das Shield wird auf einen Arduino Uno gesteckt und dient als Verbindungsebene zwischen Mikrocontroller, Schrittmotortreibern und Schrittmotoren. Es besitzt Steckplätze für bis zu vier Motortreiber und kann dadurch mehrere Achsen eines kleinen CNC-Systems ansteuern [Joy22].

Für das Projekt werden insbesondere die Achsausgänge des Shields genutzt. Die Achsen bewegen den Heißdraht beziehungsweise den Schneidrahmen entlang der programmierten Bahn. Die Bewegung muss gleichmäßig erfolgen, damit der Draht das Material sauber schneiden kann.

Das Shield stellt außerdem Anschlüsse für Endschalter bereit. Diese können für Referenzfahrten oder zur Begrenzung der Achsbewegungen verwendet werden. Dadurch kann das System sicherer betrieben und eine definierte Ausgangsposition angefahren werden.

Zusätzlich besitzt das Board Anschlüsse für Zusatzfunktionen wie Spindel, Kühlung, Reset, Endschalter und Not-Aus. Im praktischen Aufbau werden davon nur die für den CNC-Hot-Wire-Foam-Cutter benötigten Anschlüsse verwendet [Joy23].



Motoranschlüsse Versorgung 12-36V
 Treibersteckplätze Microstepping-Jumper
 Endstop-Anschlüsse Mosfet-Anschluss

Abbildung 8.1.: Anschlussübersicht des CNC Shield V3

8.5. Spezifikation

Die technischen Daten wurden dem Datenblatt und der Anleitung des Herstellers entnommen [Joy22][Joy23].

- kompatibel mit Arduino Uno und kompatiblen Arduino-Boards
- geeignet für GRBL-basierte CNC-Anwendungen

- Steckplätze für bis zu vier A4988-Schrittmotortreiber
- Achsausgänge für X, Y, Z und A
- Anschlüsse für Schrittmotoren, Endschalter, Reset, Not-Aus und Zusatzfunktionen
- Einstellung der Mikroschrittauflösung über Jumper
- Anschluss für externe Motorspannung
- Eingangsspannung: 12 bis 36 V
- maximaler Phasenstrom: 2 A pro Phase
- Abmessungen des Boards: 53 x 68 x 20 mm
- Abmessungen der A4988-Treiberboards: 15 x 20 x 16 mm
- Gesamtabmessung mit Treibern: 53 x 68 x 28 mm
- Gewicht: 43 g

Das CNC Shield erzeugt selbst keine Motorleistung. Die Leistung für die Schrittmotoren wird über die eingesetzten Treibermodule bereitgestellt. Die Motorspannung wird über die Schraubklemmen des Shields eingespeist und an die Motortreiber verteilt.

8.6. Bibliothek und Steuerung

Für das Board wird keine separate Bibliothek im klassischen Sinn benötigt. Die Steuerung erfolgt stattdessen über GRBL auf dem Arduino Uno. Dadurch können Standardfunktionen der CNC-Bewegungssteuerung genutzt werden, ohne dass eine eigene komplexe Firmware geschrieben werden muss [Joy23].

Die GRBL-Softwareumgebung übernimmt die Bewegungssteuerung und die Signalausgabe an die Motortreiber. Sie verarbeitet G-Code-Befehle und erzeugt daraus die erforderlichen STEP- und DIR-Signale für die Achsen. Das vereinfacht die Inbetriebnahme und ermöglicht den Betrieb mit einem etablierten CNC-Standard.

Zur direkten Steuerung einzelner Testprogramme können die Funktionen `pinMode()`, `digitalWrite()`, `delay()` und `delayMicroseconds()` verwendet werden. Mit `pinMode()` werden die verwendeten Pins als Ausgänge konfiguriert. Mit `digitalWrite()` werden die Zustände der STEP-, DIR- und ENABLE-Pins gesetzt. Die Funktion `delayMicroseconds()` bestimmt die Zeit zwischen den Schrittimpulsen und damit die Drehgeschwindigkeit des Motors.

8.6.1. Daten und Signale

Für die Ansteuerung des Shields werden verschiedene Signalarten verwendet. Dazu gehören Step-/Dir-Signale, Enable-Signale sowie Schaltausgänge für Zusatzfunktionen wie Spindel und Kühlung [Joy23].

- **Datenarten:** Step-/Dir-Signale, Enable-Signale und Schaltausgänge für Zusatzfunktionen.
- **Datenstruktur:** Arduino-Uno-kompatibles Shield-System mit vier Steckplätzen für A4988-Motortreiber.
- **Datenmenge:** Steuerung von bis zu vier Achsen.
- **Datenqualität:** GRBL-kompatibel und damit für eine standardisierte CNC-Steuerung ausgelegt.

8.7. Installation und Montage

Das CNC-Erweiterungsboard wird auf den Arduino Uno aufgesteckt. Danach werden die A4988-Motortreiber in der richtigen Ausrichtung eingesetzt und die externe Versorgungsspannung angeschlossen. Für den Betrieb wird eine Spannung von 12 bis 36 V verwendet [Joy22][Joy23].

Dabei muss auf die korrekte Ausrichtung der Treibermodule geachtet werden, da eine falsche Orientierung den Treiber beschädigen kann. Der Schrittmotor wird an den jeweiligen Motoranschluss der Achse angeschlossen. Vor dem Einschalten muss die Strombegrenzung des eingesetzten Motortreibers auf den verwendeten Motor abgestimmt werden.

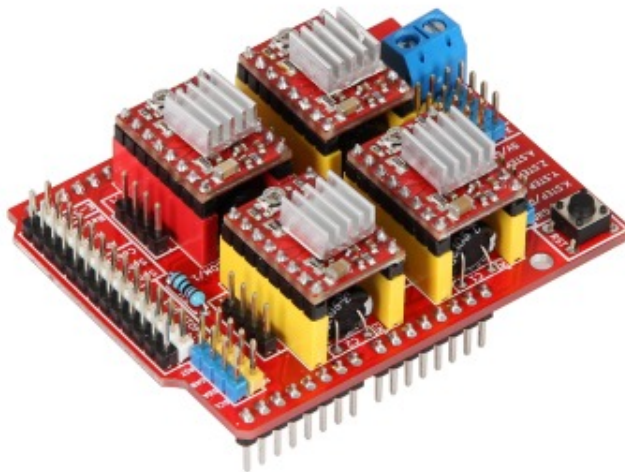


Abbildung 8.2.: CNC Shield V3 mit eingesetzten A4988-Schrittmotortreibern

8.8. Beispiel

Ein einfaches Anwendungsbeispiel ist eine kleine 4-Achs-CNC-Fräse, bei der der Arduino Uno GRBL-Befehle verarbeitet und das CNC Shield V3 die Signale an die A4988-Treiber weitergibt. So lassen sich X-, Y-, Z- und A-Achse getrennt ansteuern [Joy23].

Im Projekt dient das Beispiel zur Prüfung der grundlegenden Funktion des Shields, der eingesetzten Motortreiber und der angeschlossenen Schrittmotoren. Dabei kann kontrolliert werden, ob die Achsen reagieren, ob die Drehrichtung korrekt ist und ob die Treibermodule nicht übermäßig erwärmen.

8.8.1. Anleitung

Zuerst wird das CNC Shield auf den Arduino Uno aufgesteckt. Danach werden die A4988-Treiber so eingesetzt, dass ihre Orientierung korrekt ist. Anschließend wird die externe Spannungsversorgung im Bereich von 12 bis 36 V angeschlossen. Vor dem ersten Start wird jeder Motortreiber auf den verwendeten Motor abgestimmt, damit der Strom passend eingestellt ist. Danach können die Achsen über die GRBL-Steuerung oder mit einem einfachen Testprogramm verwendet werden [Joy22][Joy23].

8.8.2. Erklärung des Beispiel-Codes

Im Beispiel werden die Pins `X_DIR`, `X_STEP`, `Y_DIR`, `Y_STEP`, `Z_DIR`, `Z_STEP`, `A_DIR` und `A_STEP` verwendet. Zusätzlich wird der Enable-Pin aktiviert, damit die Schrittmotoren freigeschaltet werden.

Mit dem Beispielcode kann der Arduino die vier Achsen nacheinander ansteuern und nach einer festen Schrittzahl die Richtung wechseln. Der Code eignet sich als Funktionstest für das CNC Shield und zeigt die Grundsteuerung der Motoren [Joy23].

8.8.3. Code

Der folgende Code prüft die Grundfunktion des CNC Shield V3. Die Achsen werden kontinuierlich bewegt und nach einer festgelegten Schrittzahl wird die Drehrichtung gewechselt. Dadurch können die Motorspannung, die Treiberorientierung, der Motoranschluss und das STEP-Signal überprüft werden.

Listing 8.1.: Funktionstest des CNC Shield V3

```

1000 /**
1001  * @file CNCShieldFunctionTest.ino
1002  * @brief Function test for the CNC Shield V3.
1003
1004  * @date 2026-05-23
1005  */
1006
1007 const uint8_t STEP_PIN = 2;
1008 const uint8_t DIR_PIN = 5;
1009 const uint8_t EN_PIN = 8;
1010
1011 const unsigned int STEPS = 300;
1012
1013 /**
1014  * @brief Moves the motor for a defined number of steps.
1015  *
1016  * @param delayUs Delay between step edges in microseconds.
1017  */
1018 void moveMotor(unsigned int delayUs)
1019 {
1020     for (unsigned int i = 0; i < STEPS; i++)
1021     {
1022         digitalWrite(STEP_PIN, HIGH);
1023         delayMicroseconds(delayUs);
1024         digitalWrite(STEP_PIN, LOW);
1025         delayMicroseconds(delayUs);
1026     }
1027 }
1028
1029 /**
1030  * @brief Initializes the CNC Shield control pins.
1031  */
1032 void setup()
1033 {
1034     pinMode(STEP_PIN, OUTPUT);
1035     pinMode(DIR_PIN, OUTPUT);
1036     pinMode(EN_PIN, OUTPUT);
1037
1038     digitalWrite(EN_PIN, LOW);
1039 }
1040
1041 /**
1042  * @brief Repeats the complete function test.
1043  */
1044 void loop()
1045 {
1046     digitalWrite(DIR_PIN, HIGH);
1047     moveMotor(1200);

```

```

1048     delay(1000);
1050     digitalWrite(DIR_PIN, LOW);
1052     moveMotor(1200);
1054     delay(1000);
1056     digitalWrite(DIR_PIN, HIGH);
1058     moveMotor(600);
1060     delay(1000);
1062     digitalWrite(EN_PIN, HIGH);
1064     delay(1500);
1066     digitalWrite(EN_PIN, LOW);
1068     delay(1000);
1070 }

```

../Code/CNCShieldFunctionTest/CNCShieldFunctionTest.ino

8.9. Kalibrierung

Vor dem Betrieb des CNC Shield V3 müssen die eingesetzten Schrittmotortreiber eingestellt werden. Besonders wichtig ist dabei die Strombegrenzung des A4988-Treibers. Eine zu hohe Einstellung kann zu starker Erwärmung oder zur thermischen Abschaltung führen. Eine zu niedrige Einstellung kann dagegen dazu führen, dass der Motor unter Last Schritte verliert.

Die Einstellung erfolgt über das Potentiometer auf dem jeweiligen Motortreibermodul. Laut Herstelleranleitung wird die Referenzspannung mit einem Multimeter gemessen und über das Potentiometer angepasst [Joy23]. Dabei muss vorsichtig gearbeitet werden, da die Einstellung direkten Einfluss auf den maximalen Motorstrom hat.

Zusätzlich wird die Mikroschrittauflösung über Jumper unterhalb der Motortreiber eingestellt. Für den CNC-Hot-Wire-Foam-Cutter ist eine gleichmäßige Achsbewegung wichtig. Daher ist eine feinere Mikroschrittauflösung sinnvoll, sofern der Motor dabei zuverlässig läuft und keine Schritte verliert.

8.10. Einfache Anwendung

Die einfache Anwendung stellt eine projektbezogene Achsbewegung nach. Dabei bewegen sich X- und Y-Achse langsam in Schneidrichtung und anschließend schneller zurück. Das Beispiel dient zur Demonstration einer kontrollierten Vorschubbewegung, wie sie beim CNC-Hot-Wire-Foam-Cutter benötigt wird.

Im Programm werden die Funktionen `moveAxis()` und `moveXY()` verwendet. Die Funktion `moveAxis()` erzeugt Schritimpulse für eine einzelne Achse. Mit `moveXY()` werden zwei Achsen gemeinsam bewegt, sodass eine einfache zweiachsige Bewegung simuliert werden kann.

Listing 8.2.: Einfache Achsbewegung mit dem CNC Shield V3

```

1000 /**
1001  * @file CNCShieldAxisDemo.ino
1002  * @brief Simple project-related axis movement for the CNC hot wire foam
1003  *       cutter.
1004  * @date 2026-05-25
1005  */
1006
1007 const uint8_t ENABLE_PIN = 8; /**< Enable pin for all stepper drivers,
1008     active LOW. */
1009
1010 const uint8_t X_STEP_PIN = 2; /**< STEP pin for the X-axis driver. */

```

```

1010 const uint8_t X_DIR_PIN = 5; /**< DIR pin for the X-axis driver. */
1011 const uint8_t Y_STEP_PIN = 3; /**< STEP pin for the Y-axis driver. */
1012 const uint8_t Y_DIR_PIN = 6; /**< DIR pin for the Y-axis driver. */

1014 const unsigned long TRAVEL_STEPS = 1200;          /**< Number of steps for
    one travel movement. */
1015 const unsigned int CUTTING_DELAY_US = 1600;      /**< Slow feed movement
    delay. */
1016 const unsigned int RETURN_DELAY_US = 750;        /**< Faster return
    movement delay. */

1018 /**
    * @brief Moves the X- and Y-axis together.
1020 *
    * @param steps Number of step pulses.
1022 * @param delayUs Delay between HIGH and LOW level in microseconds.
    */
1024 void moveXY(unsigned long steps, unsigned int delayUs)
    {
1026     for (unsigned long i = 0; i < steps; i++)
        {
1028         digitalWrite(X_STEP_PIN, HIGH);
            digitalWrite(Y_STEP_PIN, HIGH);
1030         delayMicroseconds(delayUs);

1032         digitalWrite(X_STEP_PIN, LOW);
            digitalWrite(Y_STEP_PIN, LOW);
1034         delayMicroseconds(delayUs);
        }
1036     }

1038 /**
    * @brief Initializes the CNC Shield pins.
    */
1040 void setup()
1042 {
    pinMode(ENABLE_PIN, OUTPUT);
1044     pinMode(X_STEP_PIN, OUTPUT);
        pinMode(X_DIR_PIN, OUTPUT);
1046     pinMode(Y_STEP_PIN, OUTPUT);
        pinMode(Y_DIR_PIN, OUTPUT);

1048     digitalWrite(ENABLE_PIN, LOW);
1050 }

1052 /**
    * @brief Simulates a simple cutting movement and return movement.
    */
1054 void loop()
1056 {
    digitalWrite(X_DIR_PIN, HIGH);
1058     digitalWrite(Y_DIR_PIN, HIGH);
        moveXY(TRAVEL_STEPS, CUTTING_DELAY_US);
1060     delay(1500);

1062     digitalWrite(X_DIR_PIN, LOW);
        digitalWrite(Y_DIR_PIN, LOW);
1064     moveXY(TRAVEL_STEPS, RETURN_DELAY_US);
        delay(1500);
1066 }

```

../Code/CNCShieldAxisDemo/CNCShieldAxisDemo.ino

8.11. Tests

8.11.1. Einfacher Funktionstest

Beim einfachen Funktionstest wird geprüft, ob eine einzelne Achse grundsätzlich reagiert. Dabei werden die Verdrahtung, die Versorgungsspannung, der verwendete Motortreiber und die Drehrichtung kontrolliert.

Listing 8.3.: Einfacher Funktionstest des CNC Shield V3

```

1000 /**
1001  * @file CNCShieldSimpleStep.ino
1002  * @brief Basic function test for one axis on a CNC Shield V3.
1003
1004  * @date 2026-05-25
1005  */
1006
1007 const uint8_t ENABLE_PIN = 8;      /**< Enable pin for all stepper
1008     drivers, active LOW. */
1009 const uint8_t X_STEP_PIN = 2;      /**< STEP pin for the X-axis driver.
1010     */
1011 const uint8_t X_DIR_PIN = 5;      /**< DIR pin for the X-axis driver. */
1012
1013 const unsigned int STEP_DELAY_US = 1000; /**< Delay between step edges
1014     in microseconds. */
1015
1016 /**
1017  * @brief Initializes the CNC Shield pins and enables the stepper
1018  * drivers.
1019  */
1020 void setup()
1021 {
1022     pinMode(ENABLE_PIN, OUTPUT);
1023     pinMode(X_STEP_PIN, OUTPUT);
1024     pinMode(X_DIR_PIN, OUTPUT);
1025
1026     digitalWrite(ENABLE_PIN, LOW);
1027     digitalWrite(X_DIR_PIN, HIGH);
1028 }
1029
1030 /**
1031  * @brief Generates continuous step pulses on the X-axis.
1032  */
1033 void loop()
1034 {
1035     digitalWrite(X_STEP_PIN, HIGH);
1036     delayMicroseconds(STEP_DELAY_US);
1037     digitalWrite(X_STEP_PIN, LOW);
1038     delayMicroseconds(STEP_DELAY_US);
1039 }

```

../Code/CNCShieldSimpleStep/CNCShieldSimpleStep.ino

8.11.2. Test aller Funktionen

Beim vollständigen Test werden alle vier Achsausgänge des CNC Shields geprüft. Die Motoren werden nacheinander beziehungsweise gemeinsam über die STEP-Pins angesteuert. Nach einer festgelegten Schrittzahl wird die Drehrichtung gewechselt. Zusätzlich wird das ENABLE-Signal getestet.

Im Testprogramm werden die Pins **X_STEP**, **Y_STEP**, **Z_STEP** und **A_STEP** verwendet. Die Richtung wird über **X_DIR**, **Y_DIR**, **Z_DIR** und **A_DIR** festgelegt. Mit **digitalWrite()** werden die Achsen aktiviert und mit **delay()** beziehungsweise **delayMicroseconds()** zeitlich gesteuert.

Während des Tests wird kontrolliert, ob die Achsen korrekt reagieren, ob sich die Richtung ändert und ob sich die Motortreiber nicht übermäßig erwärmen. Der Test ersetzt keine vollständige GRBL-Inbetriebnahme, ermöglicht aber eine grundlegende Prüfung der Hardware.

8.12. Dateien

Für die Nutzung sind das Datenblatt und die Anleitung relevant. Das Datenblatt enthält die technischen Kenndaten, die Anleitung liefert Angaben zum Aufbau, zur Pinbelegung, zur Treibereinstellung und zum Beispielpogramm [Joy22][Joy23].

8.13. Weiterführende Literatur

- Hehenberger, Peter: *Computerunterstützte Produktion*. Springer Vieweg, 2020. [Heh20]
- Joy-IT / SIMAC Electronics GmbH: *ARD-CNC-KIT1 Datenblatt*. 2022. [Joy22]
- Joy-IT / SIMAC Electronics GmbH: *ARD-CNC-KIT1 Anleitung*. 2023. [Joy23]

9. Sensor: Endschalter

9.1. Einleitung: Positionssensoren

In der industriellen Fertigungssteuerung nimmt die Erfassung von Zuständen eine zentrale Rolle ein. Gemäß der Fachliteratur wandelt ein Sensor (lat. sensus, der Sinn) „eine physikalische Größe (z. B. Kraft oder Temperatur) mit Hilfe eines physikalischen Effekts in ein elektrisches Signal um“ [Her+12, S. 433]. Bei der Steuerung wird zwischen zwei grundlegenden Messverfahren unterschieden: Der „Positionserfassung durch Schalter“ und der „Positionserfassung durch Wegbeobachtung“. [Her+12, S. 435] Bevor der Fokus auf kontaktbehaftete Schalter gelegt wird, sind folgende berührungslose Sensoren (Sensorschalter) zu nennen:

- Induktive Sensoren: Erfassen metallische Elemente „ohne Kontakt zwischen der Bewegung und dem Sensor“ [Her+12, S. 436].
- Kapazitive Sensoren: Nutzen die Beeinflussung eines elektrischen Feldes zur Kapazitätsänderung [Her+12, S. 438].
- Optische Sensoren: Arbeiten mit Infrarot-Lichtstrahlen zur Objekterkennung (z. B. Lichttaster oder Lichtschranken) [Her+12, S. 440].
- Ultraschall-Sensoren: Basieren auf der Messung der Laufzeit von Schallimpulsen [Her+12, S. 443].

9.2. Definition und Funktion: Elektromechanischer Endschalter

Ein elektromechanischer Endschalter ist ein kontaktbehafteter Positionssensor. Ein solcher Schalter „hat die Aufgabe, das Erreichen einer bestimmten Position (Endlage) zu melden oder einen Schaltvorgang auszulösen“ [Her+12, S. 435]. Dabei wird durch das Betätigen eines mechanischen Elements, wie beispielsweise eines Hebels oder Tasters, ein elektrischer Kontakt geöffnet oder geschlossen [Her+12, S. 120–122].

Technische Spezifikationen (Beispiel Creality): Bei dem vorliegenden Bauteil handelt es sich um einen „originalen Endschalter von Creality 3D, der bei fast allen FDM Druckern von Creality 3D verbaut ist“ [3dj].

9.3. Specification

- Kompatibilität: Geeignet für die X-, Y- oder Z-Achse von Modellen wie der Ender-3 oder CR-10 Serie [3dj][Bas].
- Hersteller: Creality
- Hersteller SKU: 4004180004
- Gewicht: 7.3g
- Abmaße: 28mm x 20mm x 9mm
- Auslöseweg: 4mm

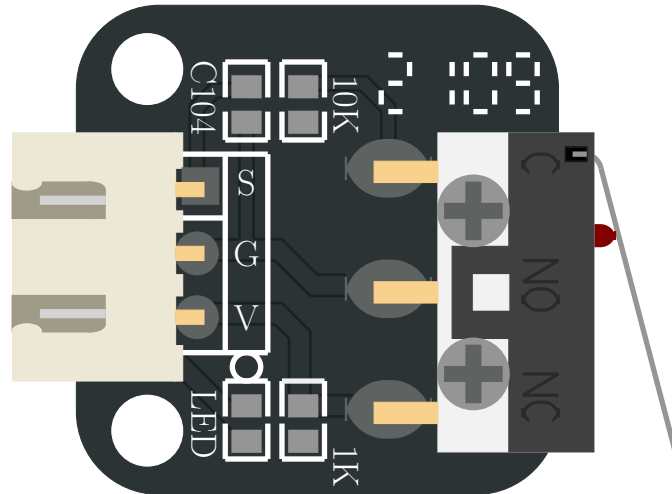


Abbildung 9.1.: Darstellung des im Projekt verwendeten Endschalters

9.4. Verwendung und Schaltfunktionen

Mechanische Endschalter werden zur Überwachung von Bewegungsabläufen eingesetzt. Dabei können verschiedene logische Funktionen realisiert werden:

- Schließer: Der Kontakt wird bei Betätigung geschlossen.
- Öffner: Der Kontakt wird bei Betätigung unterbrochen.
- Antivalenter Sensor: Dieser verfügt über zwei Ausgänge, wobei „stets ein Kontakt geschlossen und der andere offen“ ist [Her+12, S. 435]. Dies ermöglicht neben der reinen Schaltfunktion auch eine „Fehlererkennung und Diagnose durch die SPS“ [Her+12, S. 435–436].

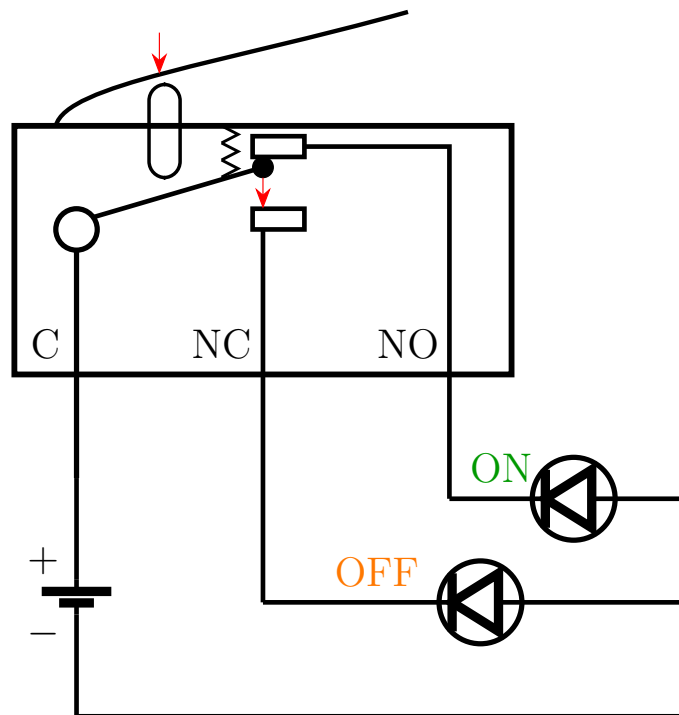


Abbildung 9.2.: Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO

Haupteinsatzgebiete:

- Endstopp-Erkennung: Schutz der Maschine vor dem Überfahren der mechanischen Grenzen.
- Referenzschalter (Homing): Der Schalter dient als „Referenzschalter“, um den Nullpunkt einer Achse (z. B. beim 3D-Drucker) zu kalibrieren [Bas].

9.5. Analyse: Vor- und Nachteile

Vorteile:

- Wirtschaftlichkeit: Sehr kostengünstig (ca. 4-5 € pro Stück) [3DJake, Bastelgarage].
- Einfachheit: Direkte Erzeugung eines binären Signals ohne aufwändige Auswerteelektronik.
- Robustheit gegen EMV: Unempfindlich gegenüber elektromagnetischen Störungen, da kein Oszillator wie bei induktiven Sensoren benötigt wird.

Nachteile:

- Mechanischer Verschleiß: Durch den physischen Kontakt unterliegen die Bauteile einer Abnutzung.
- Geschwindigkeit: Mechanische Kontakte können „prellen“ und sind langsamer als elektronische Lösungen [Her+12, S. 444].
- Technologische Verbreitung: Obwohl immer noch sehr viele mechanische Endschalter eingesetzt werden, ist der berührungslose Endschalter (Sensorschalter) in nahezu allen Bereichen als Standard-Bauelement zu finden [Her+12, S. 435].

9.6. Ausblick: Intelligente und vernetzte Sensorik

Die Zukunft der Positionserfassung liegt in der bidirektionalen Kommunikation. Ein moderner Standard ist hierbei IO-Link, welcher eine „feldbusunabhängige Kommunikation zwischen Sensoren und Aktoren und der Maschinensteuerung“ beschreibt [Her+12, S. 510].

Während klassische Endschalter nur „1 Bit“ an Informationen liefern (Ein/Aus), können intelligente Systeme über IO-Link zusätzlich Servicedaten übertragen, „die zur Einstellung oder Diagnose eines Devices führen“ [Her+12, S. 512].

Dies ermöglicht eine „vorbeugende Instandhaltung“, indem beispielsweise Verschleiß gemeldet wird, bevor es zum Maschinenausfall kommt [Her+12, S. 471, 512].

9.7. Library

Für die Verwendung des Endschalters werden keine zusätzlichen Bibliotheken benötigt. Es werden Funktionen aus der Arduino-Standardbibliothek verwendet:

- `pinMode()`: Konfiguriert einen Pin als Ein- oder Ausgang.
- `digitalRead()`: Liest den aktuellen Zustand eines digitalen Eingangs.
- `digitalWrite()`: Setzt einen digitalen Ausgang auf HIGH oder LOW.
- `Serial.begin()`: Initialisiert die serielle Kommunikation.
- `Serial.print()` / `Serial.println()`: Gibt Daten im Serial Monitor aus.
- `delay()`: Erzeugt eine zeitliche Verzögerung im Programmablauf.

9.8. Beispiel

Dieses Beispiel zeigt, wie der Endschalter über einen digitalen Eingang ausgelesen werden kann. Dabei wird der interne Pull-Up-Widerstand verwendet. Der Zustand des Schalters wird im Serial Monitor angezeigt und ändert sich beim Betätigen von HIGH auf LOW.

9.8.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND.

9.8.2. Erklärung des Beispiel-Codes

Nach der Initialisierung durch die Funktion `setup()` wird das Programm durch die Funktion `loop()` kontinuierlich ausgeführt. Diese Funktion stellt eine Endlosschleife dar, in der der Zustand des Endschalters bei jedem Durchlauf überprüft wird. [Ardj; Ardg]

Der verwendete Pin wird mit der Funktion `pinMode()` als Eingang oder Ausgang konfiguriert. [Ardi] Dabei wird der Modus `INPUT_PULLUP` verwendet, um den internen Pull-Up-Widerstand zu aktivieren. [Arda] Der interne Pull-Up-Widerstand sorgt dafür, dass am Eingang im Ruhezustand ein definiertes HIGH-Signal anliegt. Innerhalb der `loop()`-Funktion wird der Zustand des Endschalters mit der Funktion `digitalRead()` ausgelesen. Der gelesene Wert wird in einer Variablen gespeichert und über die serielle Schnittstelle ausgegeben. [Arde] Die Funktionen `Serial.print()` und `Serial.println()` werden verwendet, um den aktuellen Zustand des Eingangs im Serial

Monitor darzustellen.[Ardc] Durch die Verwendung des internen Pull-Up-Widerstands ergibt sich eine invertierte Logik. Im nicht betätigten Zustand des Endschalters liegt ein HIGH-Signal am Eingang an. Wird der Endschalter betätigt liegt ein LOW-Signal an. Zur besseren Lesbarkeit der Ausgabe wird eine Verzögerung mit der Funktion `delay()` erzeugt.[Ardd]

Listing 9.1.: Einfaches Beispiel zum Lesen eines mechanischen Endschalters

```

1000 /**
1001  * @file MicroSwitch_BasicRead.ino
1002
1003  * @brief Basic example for reading a micro switch.
1004  */
1005 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1006     switch. */
1007
1008 /**
1009  * @brief Initializes the serial interface and the input pin.
1010  */
1011 void setup()
1012 {
1013     Serial.begin(115200);
1014     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1015 }
1016
1017 /**
1018  * @brief Reads the switch state and prints it to the serial monitor.
1019  */
1020 void loop()
1021 {
1022     int switchState = digitalRead(MICRO_SWITCH_PIN);
1023
1024     Serial.print("Micro switch state: ");
1025     Serial.println(switchState);
1026
1027     delay(200);
1028 }

```

../Code/MicroSwitchBasicRead/MicroSwitchBasicRead.ino

9.9. Justage

Die Justage (auch „Justierung“) ist eine „Tätigkeit, welche ein Messgerät in einen gebrauchstauglichen Betriebszustand versetzt. Sie kann manuell, halb automatisch oder automatisch erfolgen.“ [Hel21, S. 17] Sie kann auch als das „abgleichen des Nullpunktes und der Kennliniensteigung“ [Ber24, S. 17] verstanden werden. Da der verwendete Endschalter nur die analogen Werte OPEN = 0 und CLOSED = 1 detektieren kann, sodass keine Kennlinie im herkömmlichen Sinne vorliegt, liegt die anwendungsspezifische Justage-Aufgabe darin, den gesamten Endschalter physisch in seiner Position am Rahmengestell so auszurichten, dass er die entsprechende bewegliche Achse im Rahmen einer Referenzfahrt in ihrer Bewegung eingrenzt, aber gleichzeitig im Normalbetrieb den erforderlichen Bauraum nicht künstlich einschränkt.

9.10. Tests

9.10.1. Einfacher Funktions-Test

Im einfachsten Anwendungstest wird der Endschalter verwendet, um eine LED zu steuern. Hierbei wird der Zustand des Schalters ausgewertet und auf einen Ausgang

übertragen. Wird der Endschalter betätigt, schaltet sich die LED ein. Im nicht betätigten Zustand bleibt die LED ausgeschaltet. Durch diesen Test kann die Funktion des Sensors sowie die Verarbeitung des Signals im System überprüft werden.

9.10.2. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND. Die LED wird an einen digitalen Ausgang (z. B. D13) angeschlossen. Beim Betätigen des Endschalters wird die LED eingeschaltet, im nicht betätigten Zustand bleibt sie ausgeschaltet.

9.10.3. Einfacher Test-Code

Die Funktion `loop()` läuft dauerhaft und liest den Zustand des Mikroschalters ein. Da `INPUT_PULLUP` verwendet wird, bedeutet `LOW` = gedrückt. Ist der Schalter gedrückt, wird die LED eingeschaltet, sonst ausgeschaltet.

Listing 9.2.: An- und Ausschalten einer LED mittels Endschalter

```

1000 /**
1001  * @file MicroSwitch_LEDExample.ino
1002  * @brief Simple application example using a micro switch and an LED.
1003  */
1004
1005 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1006     switch. */
1007 const int LED_PIN = 13;          /**< Digital output pin for the built-in
1008     LED. */
1009
1010 /**
1011  * @brief Initializes the input and output pins.
1012  */
1013 void setup()
1014 {
1015     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1016     pinMode(LED_PIN, OUTPUT);
1017 }
1018
1019 /**
1020  * @brief Switches the LED depending on the micro switch state.
1021  */
1022 void loop()
1023 {
1024     int switchState = digitalRead(MICRO_SWITCH_PIN);
1025
1026     if (switchState == LOW)
1027     {
1028         digitalWrite(LED_PIN, HIGH);
1029     }
1030     else
1031     {
1032         digitalWrite(LED_PIN, LOW);
1033     }
1034 }

```

../Code/MicroSwitchLEDExample/MicroSwitchLEDExample.ino

9.11. Einfache Anwendung

Das Programm führt eine Referenzfahrt mit einem Schrittmotor aus.

Im `setup()` werden die Pins für den Treiber und den Endschalter eingerichtet. Danach wird der Motortreiber aktiviert und die Referenzfahrt gestartet.

Die Funktion `oneStep()` erzeugt einen einzelnen Schritt des Motors.

Mit `moveSteps()` kann der Motor eine festgelegte Anzahl an Schritten in eine bestimmte Richtung fahren.

In `homeAxis()` fährt der Motor so lange, bis der Endschalter gedrückt wird. Danach wartet das Programm kurz und fährt den Motor einige Schritte zurück.

9.11.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters wird an D2 angeschlossen, der zweite Anschluss liegt auf GND. Der Schrittmotortreiber wird über die Steuerpins STEP und DIR an digitale Ausgänge des Mikrocontrollers angeschlossen (z. B. STEP an D3 und DIR an D4).

Listing 9.3.: Referenzfahrt einer Achse mit Endschalter

```

1000 /**
1001  * @file referenzfahrt.ino
1002  * @brief Homing routine using a stepper motor, A4988 driver, and
1003  *       endstop.
1004  */
1005 /** @brief STEP pin of the driver */
1006 const int stepPin = 2;
1007
1008 /** @brief Direction pin of the driver */
1009 const int dirPin = 5;
1010
1011 /** @brief Enable pin of the driver */
1012 const int enPin = 8;
1013
1014 /** @brief Endstop input pin */
1015 const int endstopPin = 9;
1016
1017 /** @brief Step delay in microseconds */
1018 const int stepDelayUs = 1000;
1019
1020 /** @brief Number of steps for the backoff movement */
1021 const int backoffSteps = 50;
1022
1023 /** @brief Initializes the hardware and starts the homing routine */
1024 void setup() {
1025     pinMode(stepPin, OUTPUT);
1026     pinMode(dirPin, OUTPUT);
1027     pinMode(enPin, OUTPUT);
1028     pinMode(endstopPin, INPUT_PULLUP);
1029
1030     Serial.begin(115200);
1031
1032     digitalWrite(enPin, LOW);
1033     digitalWrite(dirPin, LOW);
1034
1035     homeAxis();
1036 }
1037
1038 /** @brief Main loop (unused in this example) */
1039 void loop() {
1040 }
1041
1042 /** @brief Executes a single motor step */
1043 void oneStep() {
1044     digitalWrite(stepPin, HIGH);
1045     delayMicroseconds(stepDelayUs);
1046     digitalWrite(stepPin, LOW);
1047     delayMicroseconds(stepDelayUs);

```

```

1048 }
1050 /**
1051  * @brief Moves the motor by a defined number of steps
1052  * @param steps Number of steps to execute
1053  * @param dir Direction of movement (LOW/HIGH)
1054  */
1055 void moveSteps(int steps, bool dir) {
1056     digitalWrite(dirPin, dir);
1057     for (int i = 0; i < steps; i++) {
1058         oneStep();
1059     }
1060 }
1062 /** @brief Performs the homing sequence and backoff movement */
1063 void homeAxis() {
1064     while (digitalRead(endstopPin) == HIGH) {
1065         oneStep();
1066     }
1068     delay(200);
1069     moveSteps(backoffSteps, HIGH);
1070 }

```

../..Code/referenzfahrt/referenzfahrt.ino

9.12. Weiterführende Literatur

- Zielke, Dirk: *Mikrosysteme*. Springer, 2025. [Zie25] Das Buch behandelt die Grundlagen und Anwendungen von Mikrosystemen und bietet einen Überblick über Aufbau, Funktion und Einsatz moderner Sensorsysteme.
- Tr
 - glqq ankler, Hans-Rolf und Reindl, Leo: *Sensortechnik*. Springer, 2014. [TR14] Das Buch behandelt grundlegende Prinzipien der Sensortechnik und beschreibt verschiedene Sensorarten sowie deren Einsatz in technischen Systemen.

10. Druckknopf

10.1. General

Ein elektrischer Druckknopf, auch Drucktaster genannt, ist ein elektromechanisches Bedienelement, mit dem ein Stromkreis durch Drücken geöffnet oder geschlossen wird. Je nach Ausführung kann er als Schließer oder Öffner arbeiten. Nach dem Loslassen kehrt ein Taster wieder in seine Ausgangsstellung zurück. [Rs2]

10.2. Joy IT

Beim Joy-IT Button-Micro handelt es sich um einen kompakten Micro-Arcadebutton mit Mikroschalter. Er besteht aus Nylon, ist in mehreren Farben erhältlich und für eine Belastungsgrenze von 12 V ausgelegt.[Joya]

10.3. Spezifikationen

- Abmessungen: Ø 32.8 x 29.9 mm
- Belastungsgrenze: 10 mA / 12 V DC [Joya]
- Schaltkreis

10.4. Beispiel Code

10.4.1. Manual

Der Taster wird mit einem Anschluss an GND und mit dem anderen an Pin 2 angeschlossen. Die LED bekommt einen 220 Ohm oder 330 Ohm Vorwiderstand. Der lange LED-Anschluss ist die Anode (+) und kommt über den Widerstand an Pin 13. Der kurze LED-Anschluss ist die Kathode (-) und kommt an GND. Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()` legt fest, ob ein Pin als Eingang oder Ausgang verwendet wird. [Ardi]
- `INPUT PULLUP` aktiviert den interne Pull-up-Widerstand des Arduino. [Arda]
- `digitalRead()` liest den Zustand des Druckknopfes aus. [Arde]
- `digitalWrite()` wird die LED ein- oder ausgeschaltet. [Ardf]

10.4.2. Code

Listing 10.1.: Led mit Druckknopf

```
1000 /**
      * @file ledmitdruckknopf.ino
1002  * @brief Turns an LED on while a push button is pressed.
      *
1004  * The push button is connected between pin 2 and GND.
```

```

1006  * The LED is connected to pin 13 with a 220 ohm or 330 ohm series
      resistor.
1007  * The long LED leg is the anode (+), and the short LED leg is the
      cathode (-).
1008  */
1009  const int druckknopfPin = 2; ///< Pin where the push button is connected
1010  const int ledPin = 13;      ///< Pin where the LED is connected.
1011
1012  /**
1013   * @brief Runs once when the Arduino starts.
1014   *
1015   * The pins are configured as input or output here.
1016   */
1017  void setup() {
1018      pinMode(druckknopfPin, INPUT_PULLUP); ///< Push button input with
      internal pull-up.
1019      pinMode(ledPin, OUTPUT);              ///< LED pin as output.
1020  }
1021
1022  /**
1023   * @brief Main program that runs repeatedly.
1024   *
1025   * If the push button is pressed, the LED is turned on.
1026   * If the push button is released, the LED is turned off.
1027   */
1028  void loop() {
1029      int druckknopfStatus = digitalRead(druckknopfPin); ///< Current state
      of the push button.
1030
1031      if (druckknopfStatus == LOW) {
1032          digitalWrite(ledPin, HIGH); ///< Turns the LED on.
1033      } else {
1034          digitalWrite(ledPin, LOW);  ///< Turns the LED off.
1035      }
1036  }

```

../Code/ledmitdruckknopf/ledmitdruckknopf.ino

10.5. Einfacher Testcode

10.5.1. Manual

Der Taster wird mit einem Anschluss an GND und mit dem anderen an Pin 2 angeschlossen. Im seriellen Monitor der Arduino IDE wird dann angezeigt, ob der Druckknopf gedrückt ist oder nicht.

Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()`
- `INPUT_PULLUP`
- `digitalRead()`
- `Serial.begin()` Startet die serielle Verbindung zum Computer. [Ardb]
- `Serial.println()` Gibt den Wert 1 oder 0 im seriellen Monitor aus. [Ardc]
- `delay()` Wartet kurz, damit die Ausgabe lesbar bleibt. [Ardd]

10.5.2. Code

Listing 10.2.: Druckknopf Test

```

1000 /**
1001  * @file druckknopftest.ino
1002  * @brief Tests a push button and prints 0 or 1 to the serial monitor.
1003  *
1004  * The push button is connected between pin 2 and GND.
1005  * The internal pull-up resistor is used.
1006  */
1007
1008 const int druckknopfPin = 2; ///< Pin where the push button is connected
1009
1010 /**
1011  * @brief Runs once when the Arduino starts.
1012  *
1013  * The push button pin is set as an input with the internal pull-up
1014  * resistor.
1015  * The serial output is started.
1016  */
1017 void setup() {
1018     pinMode(druckknopfPin, INPUT_PULLUP); ///< Sets the pin as input with
1019     Serial.begin(115200);                ///< Starts the serial
1020     connection.
1021 }
1022
1023 /**
1024  * @brief Main program that runs repeatedly.
1025  *
1026  * Prints 1 if the push button is pressed.
1027  * Prints 0 if the push button is not pressed.
1028  */
1029 void loop() {
1030     int druckknopfStatus = digitalRead(druckknopfPin); ///< Reads the
1031     state of the push button.
1032
1033     if (druckknopfStatus == LOW) {
1034         Serial.println(1); ///< Push button is pressed.
1035     } else {
1036         Serial.println(0); ///< Push button is not pressed.
1037     }
1038
1039     delay(200); ///< Short pause for readable output.
1040 }

```

../Code/druckknopftest/druckknopftest.ino

10.6. Einfache Anwendung

10.6.1. Manual

Erklärung: In diesem Programm werden drei Druckknöpfe verwendet: Start, Pause und Stop. Der Start-Druckknopf startet eine LED-Abfolge, bei der fünf LEDs nacheinander aufleuchten. Der Pause-Druckknopf hält die Abfolge an und setzt sie beim erneuten Drücken fort. Der Stop-Druckknopf beendet die Abfolge und schaltet alle LEDs aus. Anschluss: Die drei Druckknöpfe werden jeweils mit einem Anschluss an GND angeschlossen. Die anderen Anschlüsse kommen an Pin 2, 3 und 4. Die LEDs werden jeweils mit 220 Ohm oder 330 Ohm Vorwiderstand an Pin 8 bis 12 angeschlossen. Der lange Anschluss der LED kommt zum Arduino-Pin, der kurze Anschluss zu GND. Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()`
- `digitalRead()`
- `digitalWrite()`
- `digitalWrite()`
- `millis()` Liefert die Zeit seit dem Start des Arduino in Millisekunden. [Ardh]
- `delay()`

10.6.2. Code

Listing 10.3.: Druckknöpfe Led Abfolge

```

1000 /**
1001  * @file druckknoepfedabfolge.ino
1002  * @brief Controls an LED sequence with three push buttons.
1003  *
1004  * The start push button starts the LED sequence.
1005  * The pause push button pauses or continues the sequence.
1006  * The stop push button stops the sequence and turns all LEDs off.
1007  */
1008
1009 const int startdruckknopf = 2; ///< Input pin for the start push button.
1010 const int pausedruckknopf = 3; ///< Input pin for the pause push button.
1011 const int stopdruckknopf = 4;  ///< Input pin for the stop push button.
1012
1013 const int ledPins[] = {8, 9, 10, 11, 12}; ///< Output pins for the LEDs.
1014 const int anzahlLeds = 5;                ///< Number of connected LEDs.
1015
1016 bool programmLaeuft = false; ///< Shows whether the LED sequence is
1017                               active.
1018 bool programmPause = false;  ///< Shows whether the LED sequence is
1019                               paused.
1020
1021 int aktuelleLed = 0;          ///< Number of the currently active
1022                               LED.
1023 unsigned long letzteZeit = 0; ///< Time of the last LED change.
1024 const unsigned long intervall = 500; ///< Time between LED changes in
1025                                     milliseconds.
1026
1027 bool letzterPauseStatus = HIGH; ///< Previous state of the pause push
1028                               button.
1029
1030 /**
1031  * @brief Runs once when the Arduino starts.
1032  *
1033  * The push buttons are set as inputs with internal pull-up resistors.
1034  * The LED pins are set as outputs.
1035  */
1036 void setup() {
1037     pinMode(startdruckknopf, INPUT_PULLUP); ///< Start push button as
1038     input.
1039     pinMode(pausedruckknopf, INPUT_PULLUP); ///< Pause push button as
1040     input.
1041     pinMode(stopdruckknopf, INPUT_PULLUP);  ///< Stop push button as input
1042     .
1043
1044     for (int i = 0; i < anzahlLeds; i++) {
1045         pinMode(ledPins[i], OUTPUT); ///< Sets the LED pin as output.
1046     }
1047
1048     alleLedsAus();
1049 }

```

```

1044 /**
1045  * @brief Main program that runs repeatedly.
1046  *
1047  * It checks the three push buttons and controls the LED sequence
1048  * depending on the input.
1049  */
1050 void loop() {
1051     if (digitalRead(startdruckknopf) == LOW) {
1052         programmLaeuft = true;
1053         programmPause = false;
1054     }
1055
1056     if (digitalRead(stopdruckknopf) == LOW) {
1057         programmLaeuft = false;
1058         programmPause = false;
1059         aktuelleLed = 0;
1060         alleLedsAus();
1061     }
1062
1063     bool aktuellerPauseStatus = digitalRead(pausedruckknopf); ///< Current
1064     state of the pause push button.
1065
1066     if (letzterPauseStatus == HIGH && aktuellerPauseStatus == LOW) {
1067         programmPause = !programmPause;
1068         delay(200); ///< Simple debounce for the push button.
1069     }
1070
1071     letzterPauseStatus = aktuellerPauseStatus;
1072
1073     if (programmLaeuft && !programmPause) {
1074         ledAbfolge();
1075     }
1076 }
1077
1078 /**
1079  * @brief Turns all LEDs off.
1080  *
1081  * All LED pins are set to LOW.
1082  */
1083 void alleLedsAus() {
1084     for (int i = 0; i < anzahlLeds; i++) {
1085         digitalWrite(ledPins[i], LOW);
1086     }
1087 }
1088
1089 /**
1090  * @brief Makes the LEDs light up one after another.
1091  *
1092  * After the defined interval has passed, the current LED is turned off
1093  * and the next LED is turned on.
1094  */
1095 void ledAbfolge() {
1096     unsigned long aktuelleZeit = millis(); ///< Current runtime of the
1097     Arduino in milliseconds.
1098
1099     if (aktuelleZeit - letzteZeit >= intervall) {
1100         letzteZeit = aktuelleZeit;
1101
1102         alleLedsAus();
1103         digitalWrite(ledPins[aktuelleLed], HIGH);
1104
1105         aktuelleLed++;
1106
1107         if (aktuelleLed >= anzahlLeds) {
1108             aktuelleLed = 0;
1109         }
1110     }
1111 }

```

```
../..Code/druckknoepfeledabfolge/druckknoepfeledabfolge.ino
```

11. MOSFET PWM-Schaltmodul

11.1. Einleitung

Das MOSFET PWM-Schaltmodul dient zur Steuerung elektrischer Lasten mit Hilfe eines Mikrocontrollers. MOSFET-Module werden in der Leistungselektronik eingesetzt, da sie hohe Ströme bei geringem Steuerstrom schalten können. Dadurch eignen sie sich zur direkten Ansteuerung durch Mikrocontroller [All24].

Durch die Ansteuerung mit Pulsweitenmodulation (PWM) kann die mittlere Leistung eines Verbrauchers geregelt werden. Dadurch lassen sich Motoren, Heizsysteme oder LED-Schaltungen gezielt steuern [TSG23].

11.2. Grundlegende Informationen

MOSFETs (Metal-Oxide-Semiconductor Field-Effect Transistors) gehören zur Gruppe der spannungsgesteuerten Halbleiterbauelemente. Sie werden als elektronische Schalter oder zur Leistungsregelung eingesetzt [All24].

Ein MOSFET besitzt die drei Anschlüsse Gate, Drain und Source. Der Source-Anschluss dient als Bezugspotential, der Drain-Anschluss ist mit der zu schaltenden Last verbunden. Über den Gate-Anschluss wird die Leitfähigkeit zwischen Drain und Source gesteuert. Liegt eine ausreichende Gate-Source-Spannung an, entsteht ein leitfähiger Kanal, durch den Strom fließen kann [TSG23].

Zur Leistungsregelung wird die Pulsweitenmodulation verwendet. Dabei wird ein Verbraucher periodisch ein- und ausgeschaltet. Durch die Veränderung des Tastverhältnisses lässt sich die mittlere Ausgangsleistung beeinflussen [TSG23].

Für Schaltanwendungen wird der MOSFET möglichst im ohmschen Bereich betrieben. In diesem Bereich verhält sich der MOSFET näherungsweise wie ein kleiner Widerstand zwischen Drain und Source [All24].

11.3. Spezifisches MOSFET-Modul

Verwendet wird ein JOY-IT MOSFET PWM-Schaltmodul. Das Modul dient zur Schaltung und Regelung von Gleichstromverbrauchern über ein PWM-Signal.

Auf dem Modul ist ein Leistungs-MOSFET vom Typ IRF9540N verbaut. Durch den geringen Durchlasswiderstand können höhere Ströme mit vergleichsweise geringer Verlustleistung geschaltet werden.

Das MOSFET-Modul wird mit einem Mikrocontroller angesteuert und eignet sich zur Steuerung von Lasten mit höherer Stromaufnahme [Joyb].

11.4. Spezifikation

Die technischen Daten wurden dem Datenblatt des Herstellers entnommen [Joyb].

11.4.1. Eigenschaften des IRF9540N

Der verwendete IRF9540N besitzt folgende Eigenschaften:

Der MOSFET wird auf dem Modul zur Schaltung der angeschlossenen Last verwendet.

Tabelle 11.1.: Technische Daten des MOSFET PWM-Schaltmoduls

Eigenschaft	Wert
Eingangsspannung	DC 5 V bis 36 V
Steuerspannung	3,3 V bis 20 V
PWM-Frequenzbereich	0 kHz bis 20 kHz
Maximale Ausgangsleistung	400 W
Maximaler Dauerstrom	15 A
Betriebstemperatur	-40 °C bis +85 °C
Abmessungen	34 mm × 17 mm × 12 mm

Tabelle 11.2.: Eigenschaften des verwendeten IRF9540N

Eigenschaft	Beschreibung
MOSFET-Typ	p-Kanal-Leistungs-MOSFET
Funktion	elektronischer Schalter
Eingangsverhalten	hoher Eingangswiderstand
Leitverhalten	geringer Drain-Source-Durchlasswiderstand
Anwendung	geeignet für PWM-Anwendungen

11.4.2. Anschlüsse

Das Modul besitzt folgende Anschlüsse:

- **SIG**
- **VCC**
- **GND**
- **VIN**
- **VOUT**

Das PWM-Steuersignal wird über den Anschluss **SIG** eingespeist. Die Versorgung der Last erfolgt über die Schraubklemmen **VIN** und **VOUT** [Joyb].

11.5. Bibliothek

11.5.1. Beschreibung

Für das MOSFET PWM-Schaltmodul wird keine zusätzliche Bibliothek benötigt. Die Ansteuerung erfolgt über die Standardfunktionen der Arduino-IDE.

11.5.2. Funktionen

Zur Steuerung des Moduls werden folgende Arduino-Funktionen verwendet:

- **pinMode()**: Konfiguriert einen Pin als Ein- oder Ausgang.
- **digitalWrite()**: Setzt einen digitalen Ausgang auf **HIGH** oder **LOW**.
- **analogWrite()**: Gibt ein PWM-Signal an einem PWM-fähigen Pin aus.
- **delay()**: Erzeugt eine zeitliche Verzögerung im Programmablauf.

Mit **digitalWrite()** wird die Last ein- oder ausgeschaltet. Mit **analogWrite()** wird ein PWM-Signal erzeugt, wodurch sich die mittlere Ausgangsleistung regeln lässt.

11.6. Beispiel

In diesem Beispiel wird eine elektrische Last über ein PWM-Signal angesteuert.

11.6.1. Versuchsaufbau

Das PWM-Signal des Mikrocontrollers wird mit dem Eingang **SIG** des MOSFET-Moduls verbunden. Der Anschluss **GND** des Moduls muss mit **GND** des Mikrocontrollers verbunden werden, damit beide Schaltungen ein gemeinsames Bezugspotential besitzen. Die Last wird über die Schraubklemmen **VIN** und **VOUT** mit einer externen Spannungsversorgung verbunden. Die externe Versorgung muss zur angeschlossenen Last passen und den erforderlichen Strom liefern können.

Als Testlast kann zunächst eine LED mit Vorwiderstand oder eine kleine Gleichstromlampe verwendet werden. Eine leistungsstarke Last, beispielsweise ein Heizdraht, sollte erst nach erfolgreichem Funktionstest angeschlossen werden.

11.6.2. Funktionsweise des Beispiels

Der Mikrocontroller erzeugt mit der Funktion `analogWrite()` ein PWM-Signal. Das MOSFET-Modul schaltet dadurch den Stromfluss periodisch ein und aus. Durch die Veränderung des Tastverhältnisses wird die mittlere Leistung der Last beeinflusst.

11.6.3. Einfacher Code

Der folgende Code dient als einfacher Funktionstest des MOSFET PWM-Schaltmoduls. Dabei wird der Steuereingang des Moduls über einen digitalen Ausgang des Mikrocontrollers geschaltet.

Für den Test wird **SIG** mit dem im Code verwendeten Ausgangspin verbunden. **GND** des MOSFET-Moduls wird mit **GND** des Mikrocontrollers verbunden. Die Testlast wird über die Lastseite des Moduls angeschlossen. Die externe Versorgung wird über **VIN** eingespeist und die Last über **VOUT** geschaltet.

Listing 11.1.: Einfacher Funktionstest des MOSFET PWM-Schaltmoduls

```

1000 /**
1001  * @file MosfetPWMTTest.ino
1002  * @brief Tests different PWM output values for a MOSFET PWM switching
1003  *       module.
1004  */
1005
1006 /// PWM-capable output pin connected to the MOSFET module signal input.
1007 const int MOSFET_PIN = 9;
1008
1009 /// PWM values used for the test sequence.
1010 const int PWM_VALUES[] = {0, 64, 128, 192, 255};
1011
1012 /// Number of PWM values in the test sequence.
1013 const int NUMBER_OF_VALUES = sizeof(PWM_VALUES) / sizeof(PWM_VALUES[0]);
1014
1015 /// Time in milliseconds for each PWM value.
1016 const unsigned long PWM_STEP_DELAY_MS = 5000;
1017
1018 /**
1019  * @brief Initializes the MOSFET control pin.
1020  */
1021 void setup() {
1022     pinMode(MOSFET_PIN, OUTPUT);
1023     analogWrite(MOSFET_PIN, 0);
1024 }
1025
1026 /**
1027  * @brief Cycles through predefined PWM values.
1028  */
1029 void loop() {
1030     for (int i = 0; i < NUMBER_OF_VALUES; i++) {
1031         analogWrite(MOSFET_PIN, PWM_VALUES[i]);
1032         delay(PWM_STEP_DELAY_MS);
1033     }
1034     analogWrite(MOSFET_PIN, 0);
1035     delay(PWM_STEP_DELAY_MS);
1036 }

```

../Code/MosfetSimpleSwitch/MosfetSimpleSwitch.ino

11.7. Kalibrierung

Für die Ansteuerung eines Heißdrahts muss ein geeigneter PWM-Wert experimentell bestimmt werden.

Dazu wird die PWM-Stufe schrittweise erhöht, bis die gewünschte Temperatur erreicht wird. Dabei dürfen die zulässige Stromaufnahme des Moduls und die Belastbarkeit der Spannungsversorgung nicht überschritten werden.

11.8. Einfache Anwendung

Das MOSFET PWM-Schaltmodul kann im Projekt zur Regelung eines Gleichstromverbrauchers eingesetzt werden. Eine mögliche Anwendung ist die Leistungsregelung eines Heißdrahts über PWM. Dadurch wird die Drahttemperatur an den verwendeten Werkstoff und die gewünschte Schnittgeschwindigkeit angepasst.

Der folgende Code gibt ein PWM-Signal an das MOSFET-Modul aus. Der Steuereingang **SIG** wird dazu mit einem PWM-fähigen Ausgang des Mikrocontrollers verbunden. **GND** des Moduls und **GND** des Mikrocontrollers müssen verbunden sein. Die Lastver-

sorgung wird an **VIN** angeschlossen, während der Verbraucher über **VOUT** geschaltet wird.

Listing 11.2.: PWM-Test des MOSFET PWM-Schaltmoduls

```

1000 /**
1001  * @file MosfetHotwireTest.ino
1002  * @brief PWM test for controlling a hot wire with a MOSFET module.
1003  */
1004
1005 /// PWM-capable output pin connected to the MOSFET module signal input.
1006 const int MOSFET_PIN = 9;
1007
1008 /// Conservative PWM values for a careful hot wire test.
1009 const int PWM_VALUES[] = {0, 30, 60, 90, 120, 150};
1010
1011 /// Number of PWM values in the test sequence.
1012 const int NUMBER_OF_VALUES = sizeof(PWM_VALUES) / sizeof(PWM_VALUES[0]);
1013
1014 /// Time in milliseconds for each PWM test step.
1015 const unsigned long PWM_STEP_DELAY_MS = 8000;
1016
1017 /// Cooling pause in milliseconds after each full test cycle.
1018 const unsigned long COOLING_DELAY_MS = 10000;
1019
1020 /**
1021  * @brief Initializes the MOSFET control pin and keeps the output off.
1022  */
1023 void setup() {
1024     pinMode(MOSFET_PIN, OUTPUT);
1025     analogWrite(MOSFET_PIN, 0);
1026 }
1027
1028 /**
1029  * @brief Applies increasing PWM values and then switches the output off
1030  */
1031 void loop() {
1032     for (int i = 0; i < NUMBER_OF_VALUES; i++) {
1033         analogWrite(MOSFET_PIN, PWM_VALUES[i]);
1034         delay(PWM_STEP_DELAY_MS);
1035     }
1036
1037     analogWrite(MOSFET_PIN, 0);
1038     delay(COOLING_DELAY_MS);
1039 }
1040

```

../Code/MosfetPWMTTest/MosfetPWMTTest.ino

11.9. Tests

11.9.1. Einfacher Funktionstest

Beim einfachen Funktionstest wird eine Testlast angeschlossen. Das Modul wird für mehrere Sekunden eingeschaltet und anschließend wieder ausgeschaltet.

Dabei wird überprüft, ob die Last korrekt schaltet, ob die Verdrahtung passend ausgeführt wurde und ob sich das Modul während des kurzen Tests auffällig erwärmt. Der Test gilt als bestanden, wenn die Last eindeutig ein- und ausgeschaltet wird und keine unzulässige Erwärmung an Modul, Leitungen oder Anschlüssen auftritt.

11.9.2. Test aller Funktionen

Beim vollständigen Test werden mehrere PWM-Werte nacheinander ausgegeben. Dabei wird überprüft, ob sich die Leistung der angeschlossenen Last abhängig vom Tastverhältnis verändert. Bei kleinen PWM-Werten muss die Last nur mit geringer Leistung betrieben werden, während bei höheren PWM-Werten eine deutlich stärkere Wirkung erkennbar sein muss.

Zusätzlich wird kontrolliert, ob das Modul bei einem digitalen Schaltsignal zuverlässig ein- und ausschaltet. Dazu wird die Last zunächst vollständig ausgeschaltet und anschließend für eine definierte Zeit eingeschaltet. Während des Tests werden die Erwärmung des MOSFET-Moduls, die Erwärmung der Anschlussleitungen und die Stromaufnahme der Last beobachtet.

Der folgende Code testet die Heißdrahtansteuerung mit dem MOSFET PWM-Schaltmodul. Dabei werden ein PWM-fähiger Ausgang des Mikrocontrollers, der Steuereingang **SIG**, die gemeinsame Masseverbindung und die Lastseite des Moduls geprüft.

Der Heißdraht wird über die Lastseite des Moduls geschaltet. **SIG** wird mit dem verwendeten PWM-Ausgang des Mikrocontrollers verbunden. Zusätzlich wird **GND** des Moduls mit **GND** des Mikrocontrollers verbunden.

Listing 11.3.: Testprogramm für die Heißdrahtansteuerung

```

1000 /**
1001  * @file A4988SimpleStep.ino
1002  * @brief Basic function test for an A4988 stepper motor driver.
1003  */
1004
1005 const uint8_t STEP_PIN = 2;
1006 const uint8_t DIR_PIN  = 5;
1007 const uint8_t EN_PIN   = 8;
1008
1009 const unsigned int STEP_DELAY_US = 1000;
1010
1011 /**
1012  * @brief Initializes the output pins and enables the driver.
1013  */
1014 void setup()
1015 {
1016     pinMode(STEP_PIN, OUTPUT);
1017     pinMode(DIR_PIN, OUTPUT);
1018     pinMode(EN_PIN, OUTPUT);
1019
1020     digitalWrite(DIR_PIN, HIGH);
1021     digitalWrite(EN_PIN, LOW);
1022 }
1023
1024 /**
1025  * @brief Generates continuous step pulses.
1026  */
1027 void loop()
1028 {
1029     digitalWrite(STEP_PIN, HIGH);
1030     delayMicroseconds(STEP_DELAY_US);
1031
1032     digitalWrite(STEP_PIN, LOW);
1033     delayMicroseconds(STEP_DELAY_US);
1034 }

```

../Code/MosfetHotwireTest/MosfetHotwireTest.ino

11.10. Weiterführende Literatur

- Alles, Martin: *Einführung in die elektronische Schaltungstechnik*. Springer Vieweg, 2024.

12. A4988-Schrittmotortreiber

12.1. Einleitung

Für die Ansteuerung der Schrittmotoren im Projekt wird der Schrittmotortreiber A4988 verwendet. Der A4988 ist ein integrierter DMOS-Microstepping-Treiber für bipolare Schrittmotoren und wird in Positioniersystemen, CNC-Systemen und vergleichbaren Anwendungen eingesetzt [All22].

Im Projekt übernimmt der A4988 die Leistungsansteuerung der Achsmotoren. Dadurch können die Bewegungen des Heißdrahts definiert und wiederholbar ausgeführt werden.

12.2. Grundlagen

Schrittmotoren gehören zu den elektrischen Antrieben, bei denen die Drehbewegung in einzelne Schritte unterteilt wird. Dadurch können Positionen gezielt angefahren werden, ohne dass zwingend ein zusätzliches Positionsmesssystem erforderlich ist [Ise08].

Für die Ansteuerung eines bipolaren Schrittmotors muss die Stromrichtung in den Motorwicklungen umgeschaltet werden. Diese Aufgabe wird durch leistungselektronische Schaltungen übernommen [Zac22]. Der A4988 enthält hierfür integrierte H-Brücken und eine interne Stromregelung.

Die Ansteuerung erfolgt über die Signale **STEP** und **DIR**. Mit jeder steigenden Flanke am Eingang **STEP** bewegt der Treiber den Motor um einen Schritt beziehungsweise Mikroschritt weiter. Das Signal **DIR** bestimmt die Drehrichtung [All22].

12.3. Spezifischer Treiber

Im Projekt wird ein A4988-kompatibles Treibermodul verwendet. Der eigentliche Treiber-IC stammt von Allegro MicroSystems. Laut Herstellerdatenblatt unterstützt der A4988 Voll-, Halb-, Viertel-, Achtel- und Sechzehntelschrittbetrieb [All22]. Durch Microstepping kann die Bewegung der Achsen gleichmäßiger ausgeführt werden.

Der A4988 besitzt eine integrierte PWM-Stromregelung. Dadurch wird der Strom durch die Motorwicklungen begrenzt. Dies ist wichtig, da der Motor mit einer höheren Versorgungsspannung betrieben werden kann, ohne die Wicklungen zu überlasten.

Zusätzlich verfügt der Treiber über Schutzfunktionen gegen Überstrom, Kurzschluss, Unterspannung und Übertemperatur [All22].

12.3.1. Installation

Der A4988 wird direkt auf das CNC-Shield V3 gesteckt. Bei Verwendung des CNC-Shields sind die Anschlüsse für **STEP**, **DIR** und **ENABLE** bereits den jeweiligen Achsen zugeordnet.

Vor dem Anschluss des Motors muss die Strombegrenzung eingestellt werden. Außerdem müssen die Masse des Mikrocontrollers und die Masse der externen Motorspannungversorgung miteinander verbunden werden.

12.4. Spezifikation

Die technischen Daten des verwendeten A4988 wurden dem Herstellerdatenblatt entnommen [All22].

Tabelle 12.1.: Technische Daten des A4988-Schrittmotortreibers

Eigenschaft	Wert
Motorspannung	8 V bis 35 V
Logikspannung	3 V bis 5,5 V
Maximaler Ausgangsstrom	bis ± 2 A
Schrittmodi	Voll-, Halb-, Viertel-, Achtel- und Sechzehntelschritt
Steuersignale	STEP, DIR, ENABLE
Stromregelung	integrierte PWM-Stromregelung
Schutzfunktionen	Übertemperatur, Unterspannung und Überstrom
Motortyp	bipolarer Schrittmotor

Der maximale Motorstrom ergibt sich aus Gleichung 12.1.

$$I_{TripMAX} = \frac{V_{REF}}{8 \cdot R_S} \quad (12.1)$$

Dabei beschreibt R_S den Sense-Widerstand und V_{REF} die Referenzspannung [All22]. Die Referenzspannung wird am Potentiometer des Treibermoduls eingestellt. Der konkrete Wert von R_S hängt vom verwendeten A4988-Modul ab und muss am jeweiligen Treiberboard geprüft werden.

12.5. Bibliothek

12.5.1. Beschreibung

Für den A4988 wird keine zusätzliche Arduino-Bibliothek benötigt. Die Ansteuerung erfolgt direkt über digitale Ausgänge des Mikrocontrollers. Im späteren Betrieb des Projekts erzeugt die GRBL-Firmware die notwendigen STEP- und DIR-Signale.

12.5.2. Funktionen

Zur direkten Steuerung des A4988 werden Standardfunktionen der Arduino-IDE verwendet. Mit `pinMode()` werden die verwendeten Pins als Ausgänge konfiguriert. Mit `digitalWrite()` werden STEP-, DIR- und ENABLE-Signale ausgegeben. Die Funktion `delayMicroseconds()` bestimmt die Zeit zwischen den Schritimpulsen und damit die Drehgeschwindigkeit.

- `pinMode()`: Konfiguriert einen Pin als Ein- oder Ausgang.
- `digitalWrite()`: Setzt einen digitalen Ausgang auf HIGH oder LOW.
- `delayMicroseconds()`: Erzeugt eine Verzögerung im Mikrosekundenbereich.
- `delay()`: Erzeugt eine Verzögerung im Millisekundenbereich.

12.6. Beispiel

In diesem Abschnitt wird ein einfacher Funktionstest des A4988 beschrieben. Der Mikrocontroller erzeugt STEP-Impulse, die vom Treiber in eine Motorbewegung umgesetzt werden. Das Beispiel prüft damit die grundlegende Verbindung zwischen Mikrocontroller, Treiber und Schrittmotor.

12.6.1. Funktionsweise des Beispiels

Im Beispiel wird die Drehrichtung über das Signal **DIR** festgelegt. Anschließend werden am Eingang **STEP** periodische Impulse ausgegeben. Jeder Impuls bewegt den Motor um einen Schritt oder Mikroschritt weiter.

Die Drehgeschwindigkeit hängt von der Frequenz der **STEP**-Impulse ab. Je kürzer die Wartezeit zwischen den Impulsen ist, desto schneller dreht sich der Motor. Zu kurze Wartezeiten können jedoch dazu führen, dass der Motor Schritte verliert oder nicht zuverlässig anläuft.

12.6.2. Einfacher Code

Für diesen Test wird der Eingang **STEP** des Treibers mit Arduino-Pin D2 verbunden. Der Eingang **DIR** wird mit Arduino-Pin D5 verbunden. Der Eingang **ENABLE** wird mit Arduino-Pin D8 verbunden.

Die Logikversorgung des A4988 erfolgt über den 5 V-Anschluss des Arduino Uno. Die Motorspannung wird über ein externes Netzteil bereitgestellt. Die Masse des Arduino Uno und die Masse der externen Spannungsversorgung müssen miteinander verbunden werden.

Listing 12.1.: Einfacher Funktionstest des A4988-Schrittmotortreibers

```

1000 /**
1001  * @file A4988SimpleStep.ino
1002  * @brief Basic function test for an A4988 stepper motor driver.
1003  */
1004
1005 const uint8_t STEP_PIN = 2;
1006 const uint8_t DIR_PIN  = 5;
1007 const uint8_t EN_PIN   = 8;
1008
1009 const unsigned int STEP_DELAY_US = 1000;
1010
1011 /**
1012  * @brief Initializes the output pins and enables the driver.
1013  */
1014 void setup()
1015 {
1016     pinMode(STEP_PIN, OUTPUT);
1017     pinMode(DIR_PIN, OUTPUT);
1018     pinMode(EN_PIN, OUTPUT);
1019
1020     digitalWrite(DIR_PIN, HIGH);
1021     digitalWrite(EN_PIN, LOW);
1022 }
1023
1024 /**
1025  * @brief Generates continuous step pulses.
1026  */
1027 void loop()
1028 {
1029     digitalWrite(STEP_PIN, HIGH);
1030     delayMicroseconds(STEP_DELAY_US);
1031
1032     digitalWrite(STEP_PIN, LOW);
1033     delayMicroseconds(STEP_DELAY_US);
1034 }

```

../Code/A4988SimpleStep/A4988SimpleStep.ino

12.7. Kalibrierung

Vor dem Betrieb muss die Strombegrenzung des A4988 eingestellt werden. Die Einstellung erfolgt über ein Potentiometer auf dem Treibermodul.

Eine zu hohe Strombegrenzung führt zu starker Erwärmung und kann eine thermische Abschaltung verursachen. Eine zu niedrige Strombegrenzung kann Schrittverluste verursachen. Für das Projekt wird die Strombegrenzung so eingestellt, dass der Motor zuverlässig läuft und der Treiber nicht übermäßig heiß wird.

Die Einstellung der Strombegrenzung erfolgt über die Referenzspannung V_{REF} . Dazu wird die gewünschte Stromgrenze mit Gleichung 12.1 berechnet. Anschließend wird die Referenzspannung am Treibermodul mit einem Multimeter kontrolliert und über das Potentiometer angepasst.

12.8. Einfache Anwendung

Die einfache Anwendung simuliert eine Achsbewegung im Projekt. Die Achse bewegt sich langsam in Arbeitsrichtung und anschließend schneller zurück. Dadurch wird der praktische Einsatz des A4988 im Projekt demonstriert.

Der folgende Code erzeugt eine einfache Vor- und Rückbewegung einer Achse. Der Treiber wird dazu über **STEP** und **DIR** angesteuert. Der Schrittmotor wird an die Motorausgänge des A4988 angeschlossen. Bei Nutzung des CNC-Shields erfolgt die Zuordnung der Signale über den jeweiligen Achssteckplatz. Im gezeigten Aufbau wird ein CNC-Shield V3 verwendet. Der A4988 wird dabei auf den Steckplatz der X-Achse gesteckt.

Für die X-Achse verwendet das CNC-Shield den Arduino-Pin D2 für **STEP**, den Arduino-Pin D5 für **DIR** sowie den Arduino-Pin D8 für **ENABLE**.

Die Logikversorgung erfolgt über den Arduino Uno. Die Motorspannung wird über ein externes Netzteil bereitgestellt. Die Masse des Arduino und die Masse der externen Spannungsversorgung müssen miteinander verbunden werden.

Listing 12.2.: Einfache Achsbewegung im Projekt

```

1000 /**
1001  * @file A4988FoamCutterAxis.ino
1002  * @brief Simple CNC axis movement for a hot wire foam cutter.
1003  */
1004
1005
1006 const uint8_t STEP_PIN = 2;
1007 const uint8_t DIR_PIN  = 5;
1008 const uint8_t EN_PIN   = 8;
1009
1010 const unsigned long AXIS_TRAVEL_STEPS = 1600;
1011
1012 const unsigned int CUTTING_FEED_DELAY_US = 1500;
1013 const unsigned int RETURN_FEED_DELAY_US  = 700;
1014
1015 /**
1016  * @brief Moves the axis with a defined speed.
1017  *
1018  * @param steps Number of steps.
1019  * @param delayUs Delay between step edges in microseconds.
1020  */
1021 void moveAxis(unsigned long steps, unsigned int delayUs)
1022 {
1023     for (unsigned long i = 0; i < steps; i++)
1024     {
1025         digitalWrite(STEP_PIN, HIGH);
1026         delayMicroseconds(delayUs);
1027
1028         digitalWrite(STEP_PIN, LOW);
1029         delayMicroseconds(delayUs);
1030     }
1031 }
1032
1033 /**
1034  * @brief Initializes the driver pins.
1035  */
1036 void setup()
1037 {
1038     pinMode(STEP_PIN, OUTPUT);
1039     pinMode(DIR_PIN, OUTPUT);
1040     pinMode(EN_PIN, OUTPUT);
1041
1042     digitalWrite(EN_PIN, LOW);
1043 }
1044
1045 /**
1046  * @brief Simulates a simple CNC axis movement.
1047  */
1048 void loop()
1049 {
1050     digitalWrite(DIR_PIN, HIGH);
1051     moveAxis(AXIS_TRAVEL_STEPS, CUTTING_FEED_DELAY_US);
1052
1053     delay(1500);
1054
1055     digitalWrite(DIR_PIN, LOW);
1056     moveAxis(AXIS_TRAVEL_STEPS, RETURN_FEED_DELAY_US);
1057
1058     delay(1500);
1059 }

```

../Code/A4988FoamCutterAxis/A4988FoamCutterAxis.ino

12.9. Tests

12.9.1. Test aller Funktionen

Beim vollständigen Test wird überprüft, ob der Motor bei **STEP**-Impulsen reproduzierbar läuft, ob sich die Drehrichtung durch **DIR** umkehrt und ob der Treiber über **ENABLE** deaktiviert und wieder aktiviert werden kann. Zusätzlich wird beobachtet, ob sich Treiber und Motor während mehrerer Bewegungszyklen unauffällig verhalten oder sich übermäßig erwärmen.

Die Geschwindigkeitsänderung wird durch unterschiedliche Impulsabstände geprüft. Bei kürzeren Abständen zwischen den **STEP**-Impulsen muss sich die Drehzahl erhöhen, bei längeren Abständen muss sie sinken. Damit wird kontrolliert, ob der Treiber die Schritimpulse zuverlässig verarbeitet und ob der Motor auch bei wechselnder Ansteuerfrequenz stabil läuft.

Im Testaufbau wird ein CNC-Shield V3 mit einem A4988-Treiber auf der X-Achse verwendet. Für die X-Achse verwendet das CNC-Shield den Arduino-Pin D2 für **STEP**, den Arduino-Pin D5 für **DIR** sowie den Arduino-Pin D8 für **ENABLE**. Die Motorspannung wird über ein externes Netzteil bereitgestellt. Die Masse des Arduino Uno und die Masse der externen Spannungsversorgung müssen miteinander verbunden werden.

Der folgende Code testet die Schritimpulserzeugung über **STEP**, die Drehrichtungsumschaltung über **DIR**, das Aktivieren und Deaktivieren des Treibers über **ENABLE** sowie das Verhalten des Motors bei unterschiedlichen Impulsabständen.

Listing 12.3.: Testprogramm für den A4988-Schrittmotortreiber

```

1000 /**
1001  * @file A4988DriverTest.ino
1002  * @brief Test program for the A4988 stepper motor driver.
1003  */
1004
1005 const uint8_t STEP_PIN = 2;
1006 const uint8_t DIR_PIN  = 5;
1007 const uint8_t EN_PIN   = 8;
1008
1009 const unsigned int STEPS_PER_TEST = 400;
1010
1011 /**
1012  * @brief Generates a defined number of step pulses.
1013  *
1014  * @param steps Number of step pulses.
1015  * @param delayUs Delay between HIGH and LOW level in microseconds.
1016  */
1017 void moveSteps(unsigned int steps, unsigned int delayUs)
1018 {
1019     for (unsigned int i = 0; i < steps; i++)
1020     {
1021         digitalWrite(STEP_PIN, HIGH);
1022         delayMicroseconds(delayUs);
1023
1024         digitalWrite(STEP_PIN, LOW);
1025         delayMicroseconds(delayUs);
1026     }
1027 }
1028
1029 /**
1030  * @brief Initializes the output pins.
1031  */
1032 void setup()
1033 {
1034     pinMode(STEP_PIN, OUTPUT);
1035     pinMode(DIR_PIN, OUTPUT);
1036     pinMode(EN_PIN, OUTPUT);
1037
1038     digitalWrite(EN_PIN, LOW);
1039 }
1040
1041 /**
1042  * @brief Executes a complete driver function test.
1043  */
1044 void loop()
1045 {
1046     digitalWrite(DIR_PIN, HIGH);
1047     moveSteps(STEPS_PER_TEST, 1200);
1048
1049     delay(1000);
1050
1051     digitalWrite(DIR_PIN, LOW);
1052     moveSteps(STEPS_PER_TEST, 1200);
1053
1054     delay(1000);
1055
1056     digitalWrite(DIR_PIN, HIGH);
1057     moveSteps(STEPS_PER_TEST, 600);
1058
1059     delay(1000);
1060
1061     digitalWrite(EN_PIN, HIGH);
1062
1063     delay(2000);
1064
1065     digitalWrite(EN_PIN, LOW);
1066
1067     delay(1000);
1068 }

```

../Code/A4988DriverTest/A4988DriverTest.ino

12.10. Weiterführende Literatur

- Isermann, Rolf: *Mechatronische Systeme*. Springer, 2008. [Ise08]
- Zach, Franz: *Leistungselektronik*. Springer Vieweg, 2022. [Zac22]
- Allegro MicroSystems: *A4988 DMOS Microstepping Driver with Translator and Overcurrent Protection*. Datenblatt, 2022. [All22]

13. Schrittmotor

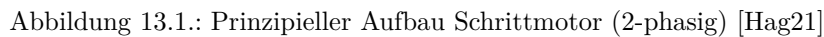
In diesem Kapitel folgt eine grundsätzliche Beschreibung eines Schrittmotors. Darauf folgt die Erläuterungen zum Aufbau und Funktionsweise eines Schrittmotors sowie die Aufzählung verschiedener Grundbauarten. Nachfolgend werden Ursachen und Lösungen zum Verhindern von Schrittfehlern und der verwendete Schrittmotor genauer erläutert.

13.1. Beschreibung eines Schrittmotors

Ein Schrittmotor ist ein Elektromotor, der sich für präzise Positionierungsaufgaben eignet. Im Gegenteil zu anderen Elektromotoren wird bei einem Schrittmotor keine Positionsmessung oder Positionsregelung benötigt. Andere mechanische Antriebssysteme benötigen einen geschlossenen Regelkreis und mechanische Bremsen, um Drehzahl und Position einzuhalten. Der Motor führt durch die Rotation des Rotors diskrete Schritte aus, wobei jeder Steuerimpuls eine Verschiebung um einen konstanten Winkel bewirkt. Mit diesem Motor ist eine erreichbare Positioniergenauigkeit von $0,1^\circ$ möglich. Diese Art von Elektromotor wird beispielsweise in Druckern oder Scanner, aber auch im Kraftfahrzeugbereich verwendet. Im Kraftfahrzeugbereich werden die Schrittmotoren zur Spiegelverstellung sowie der Sitzverstellung verwendet. Das maximal erreichbare Drehmoment eines Schrittmotors liegt bei zwei Newtonmeter und die maximal erreichbare Drehzahl bei ca. 2000 Umdrehung pro Minute. Der Vorteil eines Schrittmotors ist die Wartungsfreiheit, da der Rotor keine Wicklungen hat. [Bab23; Hag21; Ber18; SK21]

13.1.1. Aufbau und Funktionsweise eines Schrittmotors

Der Schrittmotor besteht aus einem Stator, einem Rotor ohne Wicklungen und einer Steuerelektronik. Diese Steuerelektronik setzt sich zusammen aus einer Treiberstufe und der eigentlichen Steuerung. Bei dem Stator handelt es sich um den feststehenden äußeren Teil und bei dem Rotor um den beweglichen inneren Teil, wie in Abbildung 13.1 zu erkennen ist. Im Stator sind Spulen verbaut, die von einem Strom durchflossen werden. Hierdurch entsteht ein magnetisches Feld. Da der Rotor magnetisch ist, folgt er dem Magnetfeld des Stators. Soll eine Bewegung hervorgerufen werden, werden einzelne Wicklungsstränge ein- aus- oder umgeschaltet. Durch diesen Vorgang wird ein rotierendes Magnetfeld erzeugt. Das erzeugte Magnetfeld zieht den Rotor an. Der Rotor wird bei jedem Puls (Takt) um einen Winkelschritt weitergeschaltet. Die Anzahl der Polpaare im Stator geben die Anzahl der Schritte vor. Die Drehzahl und die Drehrichtung hängt von der Reihenfolge und der Häufigkeit der Stromimpulse ab. Es gibt drei verschiedene Betriebsarten, die abhängig von der Genauigkeit und der Drehzahl sind. Im Vollschrittbetrieb werden alle Polpaare bestromt. In dem Halbschrittbetrieb wird die Schrittzahl des Motors verdoppelt, wodurch sich die Positionsauflösung im Gegensatz zum Vollschrittbetrieb verdoppelt. Allerdings wird in diesem Betrieb das Drehmoment reduziert. In dem Mikroschrittbetrieb bewegt sich der Rotor in sehr kleinen Schritten. Hierdurch wird eine hohe Positioniergenauigkeit und ein ruhiger Lauf erreicht, da die Ströme und das Drehmoment in kleineren Schritten verändert werden. Bei einer zu hohen Schrittfrequenz kann ein Schrittverlust entstehen. Bei einem Schrittverlust überspringt der Motor einzelne Winkelschritte und landet in einer vorherigen oder nächsten Position gleicher Phase. Durch die offene Steuerkette



Wie bei anderen Elektromotoren gibt es auch bei dem Schrittmotor verschiedene Grundbauarten.

- Der **permanentmagnetische Schrittmotor** hat einen Permanentmagneten in dem Rotor verbaut. Hierbei stellt sich der permanentmagnetische Rotor immer so, dass der Nordpol des Rotors dem Nordpol des Statorfeldes gegenüber liegt und der Südpol des Rotors dem Südpol des Statorfeldes. In dieser Ausrichtung ziehen sich die Pole gegenseitig an. Die Drehrichtung des Rotors hängt von der Fließrichtung des Stromes ab. Der permanentmagnetische Schrittmotor entwickelt im ausgeschalteten Zustand ein Drehmoment zur Selbsthaltung. Dies ist aufgrund des permanentmagnetischen Rotors möglich. Bei diesem Drehmoment handelt es sich um das höchste Drehmoment, das auf die Welle des Motors übertragen werden kann, ohne dass diese sich in eine rotierende Bewegung versetzt. Zu dieser Art von Schrittmotoren gehören beispielsweise der Klauenpol-Schrittmotor und der Scheibenmagnet-Schrittmotor. Bei der zweiten Bauart handelt es sich um den **Reluktanzschrittmotor**. Bei dieser Bauart besteht der Rotor aus einem weichmagnetischen Material und besitzt eine gezahnte Form. So lange der Schrittmotor von keinem Strom durchflossen wird, entsteht kein Magnetfeld. Sobald der Motor in Betrieb genommen wird, entsteht ein magnetischer Fluss innerhalb des Rotors. Wird nun eine Wicklung erregt, wird der nächste Zahn des Rotors angezogen. Dadurch, dass die Rotorzähne ungleich der Polteilung sind, kann das System unendlich lange fortgesetzt werden. Die Anzahl der Schritte und die Genauigkeit des Reluktanzschrittmotors ist abhängig von der Anzahl der Zähne auf dem Rotor. Aus

technischer Sicht sind mit dieser Bauart Schrittwinkel unter 1° möglich. Damit die Drehrichtung verändert werden kann, sind mindestens zwei Strangwicklungen nötig. Bei den **Hybridschrittmotoren**, auch bekannt als Hybridschrittmotoren handelt es sich um eine Kombination aus dem Reluktanzschrittmotor und dem Permanentmagnet-erregter-Schrittmotor. Durch diese Kombination aus den beiden Schrittmotoren werden die Vorteile aus der kleinen Schrittweite, dem hohen Drehmoment und dem Selbsthaltungsmoment genutzt. Bei dem Hybridschrittmotor besteht der Rotor aus zwei um eine halbe Zahnteilung versetzten weichmagnetischen Polrädern, die eine zahnförmige Form haben. Bei den beiden Polrädern bildet das eine Polrad den Nordpol und das zweite Polrad den Südpol. Zwischen den beiden Polrädern befindet sich ein Permanentmagnet. Anders als bei anderen Schrittmotoren wird der Rotor bei dieser Bauart axial magnetisiert. Damit ein kleiner Schrittwinkel möglich ist, haben die Statorpole ebenfalls eine zahnförmige Form. Die Ausrichtung des Rotors ist abhängig von der Stromrichtung und wird durch den minimalen Widerstand bestimmt, der sich aus dem Stromfluss durch die einzelnen Stränge ergibt. Wird ein besonders kleiner Schrittwinkel benötigt, kann dies durch Erhöhung der Zähnezahl erreicht werden. [SK21; Hag21; Bab23]

13.1.3. Betriebsarten unipolar und bipolar

Neben den oben bereits genannten Betriebsarten, kann außerdem zwischen dem Unipolarbetrieb und dem Bipolarbetrieb unterschieden werden. Der große Unterschied zwischen den beiden Betriebsarten besteht darin, dass in dem Unipolarbetrieb der Strom in eine Richtung fließt. Bei dem Bipolarbetrieb hingegen fließt der Strom in beide Richtungen. Dies ist möglich, da jeder Wicklungsstrang über eine Vollbrücke gespeist wird. Ein weiterer Unterschied besteht in der Schaltung der Zweige, durch die ein Gleichstrom fließt. In dem Unipolarbetrieb werden die beiden Zweige in Reihe geschaltet. Jeder Wicklungsstrang wird mit zwei Drähten parallel gewickelt. Sind die beiden Wicklungsstränge in dem Bipolarbetrieb parallel gewickelt, müssen die Zweige parallelgeschaltet werden. Im Bipolarbetrieb kann ein höherer Wirkungsgrad erzielt werden, wo hingegen der Unipolarbetrieb eine deutlich einfachere Schaltung aufweist. [SK21]

13.1.4. Mikroschrittverfahren

Mit dem Mikroschrittverfahren können Zwischenschritte und Mikroschritte erzeugt werden. Es bietet die Fähigkeit, Geräusche und Vibrationen zu minimieren sowie die Auflösung der Umdrehung zu erhöhen. Um Zwischenschritte zu erreichen, müssen beide Wicklungen eines Schrittmotors gleichzeitig mit Strom versorgt werden. Mit dem Sinus/Cosinus-Mikroschrittverfahren können die Mikroschrittverfahren realisiert werden, indem der Strom in jeder Phase des Motors sinusförmig variiert. Durch die Anwendung sinusförmiger Ströme auf die Motorwicklungen wird eine feinere und gleichmäßigere Bewegung erzielt, was zur einer präziseren Steuerung und reduzierten mechanischen Vibrationen führt. [McC24c]

Ein Mikroschritt-Treiber unterteilt die Motorschrittwinkel in vier oder mehr Teil. Es handelt sich um einen Controller, der die Methode der Stromreglung verwendet, um die Mikroschritte zu erzeugen. Der Controller bietet außerdem den Vorteil einer erhöhten Flexibilität bei der Integration der Strombegrenzung und der Anpassung der Antriebscharakteristik als Reaktion auf Änderungen der Systemdynamik. Die Anzahl der Unterteilungen ist einstellbar, was eine präzise Steuerung und Anpassung an spezifische Anforderungen ermöglicht. [Bab23]

13.2. Schrittverluste verhindern

Um mögliche Schrittverlusten einzugrenzen und oder sie zu verhindern, wird in diesem Kapitel beschrieben, wie die Ursachen für Schrittverluste oder Stillstand methodisch zu ermitteln sind. [Dr.20]

13.2.1. Auswahl des Schrittmotors

Zunächst muss ein passender Motor für die Anwendung gewählt werden. Dabei sollten folgende grundlegende Regeln erfüllt sein:

- Motorauswahl durch Höchstwerte für Drehmoment und Drehzahl (Worst-Case-Szenario)
- Verwendung eines Sicherheitsaufschlag von 30 % auf die Drehmoment-Drehzahl-Kennlinie(Kippmoment)
- Sicherstellen, dass externe Ereignisse die Anwendung nicht blockieren können

Sind die geforderten Drehmomente bei den jeweiligen Drehzahlen, den Motorspezifikation entsprechend, dann sind keine Probleme zu erwarten. Ist der Motor zu schwach gewählt und die Anwendung fordert mehr Leistung als der Motor abgeben kann, so bleibt der Motor stehen. Der nächste Schritt ist die Durchführung von Testdurchläufen. Es soll im Betrieb überprüft werden, ob Schrittverluste auftreten. Schrittmotoren verlieren konstruktionsbedingt nicht nur einen einzigen Schritt. Bei geringen Drehzahlen verliert der Motor ein Vielfaches von vier Schritten.[Dr.20]

13.2.2. Betriebsart

In diesem Abschnitt werden je nach Betriebsart mögliche Ursachen erläutert, falls der Schrittmotor bei den Tests versagt.

Start-Stopp-Betrieb

Der Motor ist mit der Last fest verbunden und wird mit konstanter Drehzahl betrieben. Innerhalb des ersten Schrittes muss der Motor auf die vorgegebene Frequenz beschleunigen wie in Abbildung 13.2 zu sehen ist.[Dr.20]

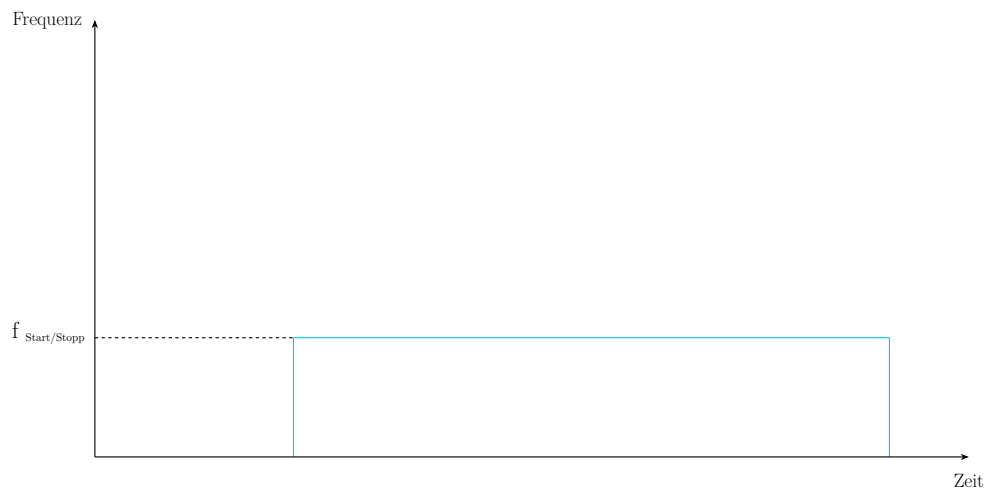


Abbildung 13.2.: Start-Stopp Frequenz [Dr.20]

Fehlerbild: Motor läuft nicht an (siehe Ursachen und Lösungen aus Tabelle 13.1)

Ursachen	Lösungen
Last zu Hoch	Falscher Motor, größeren Motor wählen
Frequenz zu hoch	Frequenz reduzieren
Pendelt der Motor von Links nach Rechts	Es könnte eine Phase unterbrochen oder nicht angeschlossen sein, dies muss repariert werden
Phasenstrom passt nicht	Phasenstrom erhöhen

Tabelle 13.1.: Ursachen und Lösungen: Motor läuft nicht an [Dr.20]

Beschleunigung und Rampenprofil (Trapezförmig)

Der Motor kann mit einer im Controller vorgegebenen Beschleunigungsrate bis auf die Maximalfrequenz beschleunigen wie in Abbildung 13.3 zu sehen ist.[Dr.20]

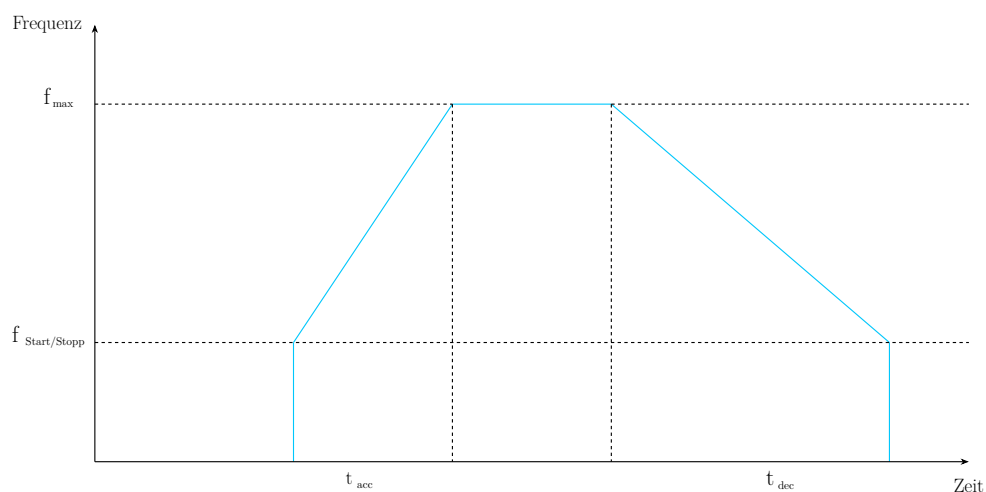


Abbildung 13.3.: Trapezförmiges Geschwindigkeitsprofil [Dr.20]

Fehlerbild: Motor beendet die Beschleunigungsrampe nicht (siehe Ursachen und Lösungen aus Tabelle 13.2)

Ursachen	Lösungen
Motor bleibt bei der Resonanzfrequenz hängen	• Beschleunigung erhöhen, um die Resonanzfrequenz schneller zu durchlaufen • Start-Stopp Frequenz über dem Resonanzpunkt wählen • Halbschritt- oder Mikroschrittbetrieb verwenden • Mechanische Dämpfung vorsehen
Falsche Einstellung von Versorgungsspannung oder Strom zu gering	• Spannung oder Strom erhöhen • Motor mit geringerer Impedanz testen • Stromregelung verwenden
Maximaldrehzahl zu hoch	• Maximaldrehzahl reduzieren • Beschleunigungsrampe abflachen
Schlechte Vorgabe der Beschleunigungsrampe durch Elektronik	• Anderen Controller verwenden

Tabelle 13.2.: Ursachen und Lösungen: Motor beendet die Beschleunigungsrampe nicht [Dr.20]

Fehlerbild: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist (siehe Ursachen und Lösungen aus Tabelle 13.3)

Ursachen	Lösungen
Motor wird an Leistungsgrenze betrieben und bleibt stehen aufgrund zu hoher Beschleunigung	• Ruckeln verringern durch geringere Beschleunigungsrate oder durch unterschiedliche Beschleunigungsrampen, erst steil dann flacher • Drehmoment erhöhen • Motor im Mikroschrittbetrieb betreiben • Mechanische Dämpfung vorsehen

Tabelle 13.3.: Ursachen und Lösungen: Motor beschleunigt bis zur Enddrehzahl und bleibt stehen, sobald eine konstante Drehzahl erreicht ist. [Dr.20]

13.2.3. Externe Ereignisse

Lastrückkopplung

„Manchmal wird der vom Motor angetriebene Mechanismus/Last während der Bewegung „aufgezogen“ und gibt diese Energie wieder an den Motor zurück, wenn die Ströme ausgeschaltet werden. Der Mechanismus könnte z.B. ein Untersetzungsgetriebe sein.“ [Dr.20] Wird diese Energie an den Motor zurück geleitet, kann es passieren, dass der Motor sich um einen Winkel verdreht, der mehr als einem Schritt entspricht. Dabei kann es passieren, dass der Motor nicht ausreichend Drehmoment entwickelt und nicht oder erst nach 4 Vollschritten anläuft. [Dr.20]

Lösungen:

- Die Kommutierung so programmieren, dass Wert und Polarität vor dem Abschalten gespeichert und beim Wiedereinschalten verwendet werden kann.
- Nicht vollständig den Strom abschalten, sondern bei Motorstillstand einen reduzierten *Stand-By* Strom aufrechterhalten

Erhöhung der Nutzlast mit der Zeit

„Manchmal läuft der Motor für eine lange Zeit störungsfrei und viel später treten die ersten Schrittlverluste auf. In diesem Fall ist es sehr wahrscheinlich, dass die Last, die der Motor „sieht“, sich geändert hat. Das kann auf Verschleiß der Motorlager oder ein externes Ereignis zurückzuführen sein.“[Dr.20]

Lösungen:

- Prüfen ob ein externes Ereignis durch Veränderung des Mechanismus vorliegt.
- Prüfen ob Lagerverschleiß vorhanden ist. Verwendung von Kugellager erhöhen die Lebensdauer des Motors.
- Verwendung von Schmiermittel um Reibung zu verhindern.

13.3. Beschreibung des verwendeten Schrittmotors

Für das Automatisierungsprojekt wird ein NEMA-17-Schrittmotor der Firma Creativity3D verwendet. Dieser Schrittmotor wird im 3D-Druck sowie in CNC-Maschinen eingesetzt. Der Motor arbeitet im Bipolarbetrieb und es sind zwei Spulen verbaut. Er hat einen Schrittwinkel von $1,8^\circ$ und benötigt 200 Schritte für eine volle Umdrehung. Die Wellenlänge beträgt 20 mm und der Wellendurchmesser 5 mm. Der ausgewählte Schrittmotor arbeitet mit einer Nennspannung von 12 Volt. Der Motor ist nicht für große Drehmomente ausgelegt, sondern für schnelle und präzise Bewegungen. Dies wird qualitativ in der Abbildung ?? für den Schrittmotor GM42BYG015 dargestellt. Hier ist zu erkennen, dass der Schrittmotor ein Drehmoment von ca. 0,8 kg·cm erreicht. Auf der x-Achse wird die Geschwindigkeit in Pulses per Second (PPS) angegeben, d.h. die Anzahl der Steuerimpulse pro Sekunde. Der Schrittmotor erzielt aber eine gute Positioniergenauigkeit durch den kleinen Schrittwinkel und die Microstep-Funktion. Des weiteren arbeitet der Motor mit 0,84 Ampere pro Phase. Weitere Vorteile des Schrittmotors sind die kompakte Bauform mit den Abmessungen $42 \times 42 \times 34$ mm sowie die geringe Masse mit 220 Gramm. Betrieben werden kann der Schrittmotor in einer Umgebungstemperatur von -10°C bis $+50^\circ\text{C}$. [Glob; Gloa]

13.4. Bibliotheken

Für die Erstellung des Programms 13.1 werden Bibliotheken verwendet, um die Programmierung zu vereinfachen.

13.4.1. AccelStepper

Die Arduino AccelStepper Bibliothek bietet eine objektorientierte Schnittstelle für die Steuerung von Schrittmotoren und Motortreibern über 2, 3 oder 4 Pins. Diese Bibliothek stellt eine wesentliche Verbesserung gegenüber der standardmäßigen Arduino Stepper Bibliothek dar und bietet zahlreiche erweiterte Funktionen, die für anspruchsvollere Anwendungen erforderlich sind. Sie wird in diesem Projekt in der Version 1.64 verwendet. Zu den herausragenden Merkmalen der AccelStepper Bibliothek gehört die Unterstützung von Beschleunigungs- und Verzögerungsprozessen. Dies ermöglicht eine feinere Steuerung der Motorbewegungen, was insbesondere bei präzisen Anwendungen

von Vorteil ist. Darüber hinaus unterstützt die Bibliothek die gleichzeitige Steuerung mehrerer Schrittmotoren, wobei jeder Motor unabhängig voneinander gesteuert werden kann. Dies ist besonders nützlich in Anwendungen, die eine synchronisierte Bewegung mehrerer Motoren erfordern. Ein weiterer Vorteil der AccelStepper Bibliothek ist, dass die meisten API-Funktionen nicht blockieren oder verzögern, es sei denn, dies ist ausdrücklich angegeben. Dies trägt dazu bei, dass der Hauptprogrammfluss nicht unterbrochen wird, was die Effizienz und Reaktionsfähigkeit des Systems verbessert. Die Bibliothek unterstützt sowohl 2-, 3- als auch 4-Draht-Schrittmotoren sowie 3- und 4-Draht-Halbschrittmotoren. Darüber hinaus können alternative Schrittverfahren verwendet werden, um beispielsweise den AFMotor zu unterstützen. Die Theorie hinter der AccelStepper Bibliothek basiert auf den Geschwindigkeitsberechnungen, die in dem Artikel „Generate stepper-motor speed profiles in real time“ von David Austin beschrieben sind. Die Bibliothek verwendet Schritte pro Sekunde, anstelle von Radianen pro Sekunde, da der Schrittwinkel des Motors nicht bekannt ist. Ein anfängliches Schrittingintervall wird basierend auf der gewünschten Beschleunigung berechnet, und bei nachfolgenden Schritten werden kürzere Schrittingintervalle basierend auf dem vorherigen Schritt berechnet, bis die maximale Geschwindigkeit erreicht ist. Die AccelStepper Bibliothek wird unter einer freien GPL-Lizenz angeboten, was bedeutet, dass sie kostenlos genutzt und verändert werden kann, solange der Quellcode offengelegt wird. Für kommerzielle Anwendungen, bei denen der Quellcode nicht offengelegt werden soll, steht eine kommerzielle Lizenz zur Verfügung. Zusammenfassend bietet die Arduino AccelStepper Bibliothek eine leistungsstarke und flexible Lösung für die Steuerung von Schrittmotoren. Mit ihrer Unterstützung für Beschleunigung und Verzögerung, der gleichzeitigen Steuerung mehrerer Motoren und ihrer nicht blockierenden API ist sie eine ausgezeichnete Wahl für anspruchsvolle Anwendungen.[McC24b; McC24a]

13.4.2. Wire

Die Wire Bibliothek ermöglicht die Kommunikation mit I2C-Geräten auf allen Arduino-Plattformen und ist ein weit verbreitetes Protokoll zum Lesen und Senden von Daten zu und von externen I2C-Komponenten. Die I2C-Pins variieren je nach Hardware-Design und Architektur der verschiedenen Arduino-Boards, wobei die Standard-I2C-Pins für Nano Boards die Pins A4 (SDA) und A5 (SDL) sind.

13.4.3. Adafruit GFX

Die Adafruit GFX Bibliothek ist eine Kern-Grafikbibliothek, die grundlegende Grafikprimitiven wie Punkte, Linien und Kreise für Adafruit-Displays bereitstellt. Sie ist unerlässlich für die Darstellung von Grafiken auf verschiedenen Hardware-Displays und erfordert für jedes Display eine spezifische Hardware-Bibliothek zur Verwaltung der niedrigeren Funktionen. Zusätzlich unterstützt sie verschiedene Bitmap-Schriftarten und bietet Werkzeuge wie Image2Code zur Konvertierung von BMP-Dateien in Code-Arrays für die Anzeige. Die Bibliothek legt besonderen Wert auf die Rückwärtskompatibilität mit bestehenden Arduino-Sketches und bietet umfassende Möglichkeiten zur Optimierung und Anpassung von Grafik- und Textdarstellungen auf den Displays. Sie wird in diesem Projekt in der Version 1.11.9 verwendet.[Aada]

13.4.4. Adafruit SSD1306

Die Adafruit SSD1306 Bibliothek ist speziell für monochrome OLED-Displays entwickelt, die den SSD1306 Treiber verwenden. Diese Displays kommunizieren über I2C oder SPI und erfordern 2 bis 5 Pins für die Verbindung. Die Bibliothek bietet grundlegende Funktionen zur Steuerung der Displays wie das Initialisieren der Kommunikation, das Zeichnen von Grafikelementen wie Linien und Kreisen sowie das Anzeigen von Text. Sie ermöglicht die einfache Anpassung der Displaygröße über den

Konstruktor und unterstützt sowohl SPI-Transaktionen als auch die Spezifikation der SPI-Bitrate ab Arduino 1.6 oder höher. Die Installation erfolgt idealerweise über den Arduino IDE Bibliotheksmanager oder durch manuelles Herunterladen und Entpacken des Quellcodes von GitHub. Sie wird in diesem Projekt in der Version 2.5.10 verwendet.[Adab]

13.4.5. Encoder

Die Encoder Library ermöglicht das Zählen von Impulsen aus bestimmten Signalen, die häufig von Drehknöpfen, Motor- oder Wellensensoren sowie anderen Positionsgebern stammen. Diese Bibliothek ist für Arduino und Teensy Boards optimiert und bietet verschiedene Zählmodi, darunter 4X counting für präzise Positionserfassung. Die Verwendung erfolgt durch die Initialisierung eines Encoder-Objekts mit zwei Pins, wobei der erste Pin idealerweise über Interruptfähigkeit verfügt für optimale Leistung. Die Bibliothek unterstützt sowohl I2C- als auch SPI-Schnittstellen und bietet verschiedene Hardware- und Softwareoptionen zur Anpassung an die Anforderungen des Benutzers. Sie wird in diesem Projekt in der Version 1.4.4 verwendet.[Sto24]

13.5. Beschreibung der Anwendung

13.5.1. Aufgabe des Programms

Das Programm steuert einen Schrittmotor und zeigt Informationen auf einem OLED-Display an. Es nutzt einen Drehwinkel-Encoder, um verschiedene Betriebsstufen des Motors auszuwählen.

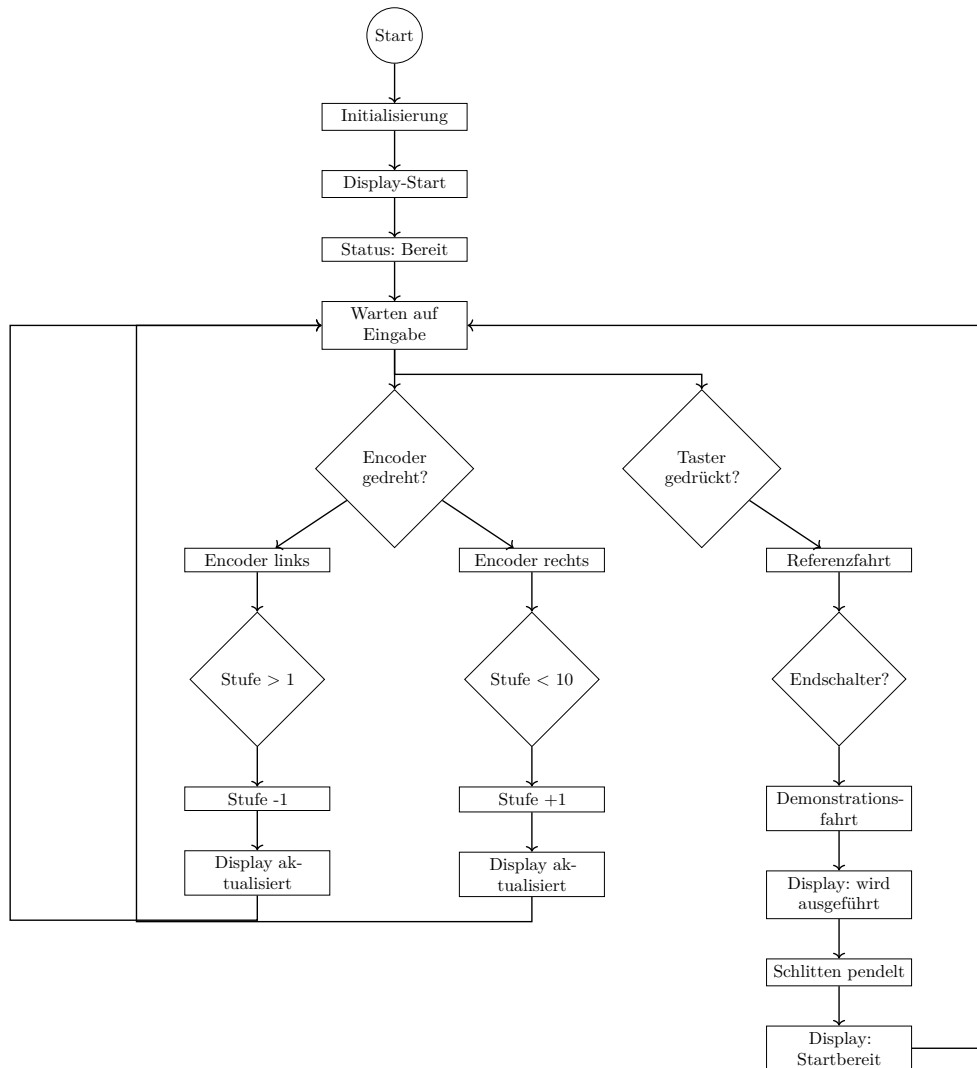


Abbildung 13.4.: Flussdiagramm des Programms

Beim Start des Programms werden alle Komponenten, einschließlich des Schrittmotors, des Displays, der LEDs und des Encoders, initialisiert. Nach einer kurzen Startanimation auf dem Display wechselt das System in den Bereitschaftsmodus. Der Benutzer kann den Drehwinkel-Encoder drehen, um eine der zehn verfügbaren Stufen auszuwählen. Jede Stufe hat spezifische Geschwindigkeitswerte für den Schrittmotor. Die aktuell ausgewählte Stufe wird auf dem OLED-Display angezeigt, sodass der Benutzer immer über die gewählte Einstellung informiert ist. Ein Taster ermöglicht es dem Benutzer, den Startprozess des Schrittmotors zu initiieren. Nach dem Drücken des Tasters führt der Schrittmotor Bewegungen aus, die der ausgewählten Stufe entsprechen. Diese Bewegungen beinhalten das Fahren zu einem festgelegten Punkt und das anschließende Zurückfahren zur Ausgangsposition. Zusammengefasst ermöglicht das Programm dem Benutzer, durch Drehen des Encoders verschiedene Betriebsstufen auszuwählen und den Schrittmotor zu steuern. Die Stufenanzeige auf dem Display und die LED-Statusanzeigen bieten dabei eine klare und leicht verständliche Rückmeldung über den aktuellen Zustand des Systems. Der Ablauf des Programms ist in Abbildung 13.4 dargestellt.

13.5.2. Erklärung des Codes

Das Programm zur Steuerung des Demonstrators wurde zunächst aus Teilen der Testprogramme für jede Hardware-Komponente basierend auf der gewünschten Funktion zusammengefügt. Nachdem die gewünschte Funktionalität erreicht war, wurde der Code mit Hilfe des Large-Language-Models Chat GPT in der Version 4.0 des US-Unternehmens OpenAI Inc. bezüglich der Lesbarkeit und Vereinheitlichung Benennung von Variablen. Zuerst werden die notwendigen Bibliotheken eingebunden, die für die Steuerung des Schrittmotors, das Display und den Encoder erforderlich sind. Die define-Anweisungen legen Konstanten fest, wie die Art des Motorinterfaces, die Abmessungen des Displays und die Pin-Nummern für den Stepper-Motor, die LEDs, den Taster, den Endschalter und den Encoder. Nun werden die Objekte für den Stepper-Motor, das Display und den Encoder erstellt. Die globalen Variablen speichern den Status des Tasters, die Zeit der letzten Zustandsänderung, die aktuelle Stufe sowie die Geschwindigkeiten und Beschleunigungen für jede Stufe.

Funktionen

- In `setup` werden die Startwerte für die maximale Geschwindigkeit und Beschleunigung des Schrittmotors gesetzt, die Pins für LEDs, Taster und Endschalter initialisiert, die serielle Kommunikation gestartet und das Display initialisiert. Ein Interrupt für den Encoder-Taster wird festgelegt. Das Display zeigt eine Startanimation, und die LEDs werden auf ihre Startzustände gesetzt.
- In `loop` wird kontinuierlich ausgeführt. Sie liest die Position des Encoders aus und zeigt bei einer Änderung die entsprechende Stufe auf dem Display an. Der Zustand des Tasters wird abgefragt und bei Betätigung die Referenzfahrt des Motors gestartet. Nach der Referenzfahrt führt der Motor Bewegungen gemäß den eingestellten Stufen und Geschwindigkeiten aus.
- `updateLEDs` aktualisiert den Zustand der LEDs basierend auf den übergebenen Parametern.
- `updateDisplay` aktualisiert das Display mit den übergebenen Nachrichten.
- `findReference` führt die Referenzfahrt aus und überprüft den Endschalter. Wenn der Endschalter gedrückt ist, stoppt der Motor und die Funktion gibt true zurück.
- `handleInterrupt` wird aufgerufen, wenn der Encoder-Taster gedrückt wird. Sie stoppt den Motor und aktualisiert das Display mit einer Abbruchnachricht.

13.5.3. Code

Listing 13.1.: Steuerung eines Schrittmotors mit OLED-Display, Encoder, Taster und Endschalter.

```

1000 /**
      * @file StepperOLEDEncoder.ino
1002 * @brief Steuerung eines Schrittmotors mit OLED-Display, Encoder,
      *       Taster und Endschalter.
      *
1004 * Das Programm steuert einen Schrittmotor ueber einen Motortreiber.
      * Ueber einen Drehencoder kann eine Stufe von 1 bis 10 ausgewaehlt
      * werden.
1006 * Die aktuelle Stufe und der Programmstatus werden auf einem OLED-
      * Display angezeigt.
      *
1008 * Mit dem Starttaster wird zuerst eine Referenzfahrt ausgefuehrt.
  
```

```

    * Danach bewegt sich der Schlitten entsprechend der ausgewaehlten Stufe
    *
1010 * Ein Endschalter dient zur Referenzierung.
1012 * Ein Encoder-Taster kann zum Abbrechen verwendet werden.
    */
1014
#include "AccelStepper.h"
1016 #include <Wire.h>
#include <Adafruit_GFX.h>
1018 #include <Adafruit_SSD1306.h>
#include <Encoder.h>
1020
/**
1022 * @brief Interface-Typ fuer den Schrittmotortreiber.
    *
1024 * Der Wert 1 steht bei AccelStepper fuer einen Treiber mit STEP- und
    DIR-Pin.
    */
1026 #define MOTOR_INTERFACE_TYPE 1

/** @brief Breite des OLED-Displays in Pixeln. */
1028 #define SCREEN_WIDTH 128
1030
/** @brief Hoehe des OLED-Displays in Pixeln. */
1032 #define SCREEN_HEIGHT 64

/** @brief Reset-Pin des OLED-Displays. -1 bedeutet, dass kein Reset-Pin
    verwendet wird. */
1034 #define OLED_RESET -1
1036
/** @brief I2C-Adresse des OLED-Displays. */
1038 #define SCREEN_ADDRESS 0x3C

/** @brief Richtungspin des Schrittmotortreibers. */
1040 #define PIN_STEP_DIR 4
1042
/** @brief STEP-Pin des Schrittmotortreibers. */
1044 #define PIN_STEP_STEP 5

/** @brief Pin der roten LED. */
1046 #define PIN_LED_RED 6
1048
/** @brief Pin der gruenen LED. */
1050 #define PIN_LED_GREEN 7

/** @brief Pin der blauen LED. */
1052 #define PIN_LED_BLUE 8
1054
/** @brief Pin des Starttasters. */
1056 #define PIN_BUTTON 11

/** @brief Pin des Endschalters fuer die Referenzfahrt. */
1058 #define PIN_ENDSTOP 12
1060
/** @brief CLK-Pin des Drehencoders. */
1062 #define PIN_CLK 9

/** @brief DT-Pin des Drehencoders. */
1064 #define PIN_DT 2
1066
/** @brief SW-Pin des Encoder-Tasters. */
1068 #define PIN_SW 3

/**
1070 * @brief Schrittmotor-Objekt.
    *
1072 * Das Objekt steuert den Schrittmotor ueber STEP und DIR.
    */
1074

```

```

AccelStepper stepper(MOTOR_INTERFACE_TYPE, PIN_STEP_STEP, PIN_STEP_DIR);
1076
/**
1078  * @brief OLED-Display-Objekt.
1080  * Das Display wird ueber I2C mit der Wire-Bibliothek angesteuert.
1082  */
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET)
;

1084 /**
1086  * @brief Encoder-Objekt.
1088  * Der Encoder wird verwendet, um die Betriebsstufe auszuwaehlen.
1090  */
Encoder encoder(PIN_DT, PIN_CLK);

1092 /** @brief Aktueller Zustand des Starttasters. */
int buttonStatus = HIGH;

1094 /** @brief Vorheriger Zustand des Starttasters. */
int lastButtonStatus = HIGH;

1096 /** @brief Zeitpunkt der letzten Zustandsaenderung des Tasters. */
1098 unsigned long lastDebounceTime = 0;

1100 /** @brief Entprellzeit fuer Taster und Endschalter in Millisekunden. */
unsigned long debounceDelay = 50;

1102
1104 /** @brief Aktueller Zustand des Endschalters. */
int endstopStatus = HIGH;

1106 /** @brief Vorheriger Zustand des Endschalters. */
int lastEndstopStatus = HIGH;

1108 /** @brief Zeitpunkt der letzten Zustandsaenderung des Endschalters. */
1110 unsigned long lastEndstopDebounceTime = 0;

1112 /** @brief Aktuell ausgewaehlte Stufe. */
int stage = 1;

1114
1116 /**
1118  * @brief Geschwindigkeiten fuer die einzelnen Stufen.
1120  * Index 0 wird fuer die Referenzfahrt bzw. Startwerte verwendet.
1122  * Die Stufen 1 bis 10 verwenden die Indizes 1 bis 10.
1124  */
int speeds[] = {
1126     1000,
1128     500,
1130     750,
1132     1000,
1134     1250,
1136     1500,
1138     1750,
1140     2000,
1142     2250,
1144     2500,
1146     1750
};

1148 /**
1150  * @brief Beschleunigungswerte fuer die einzelnen Stufen.
1152  * Index 0 wird fuer die Referenzfahrt bzw. Startwerte verwendet.
1154  * Die Stufen 1 bis 10 verwenden die Indizes 1 bis 10.
1156  */
int accelerations[] = {
1158     1000,
1160

```

```

1144     6000,
1146     6000,
1148     6000,
1150     6000,
1152     6000
};

1154 /** @brief Alte Encoderposition zur Erkennung einer Aenderung. */
1156 long oldPosition = -999;

1158 /** @brief Sicherheitsweg nach der Referenzfahrt in Schritten. */
1160 const int safeSteps = 70;

1162 /** @brief Laenge der Fahrstrecke in Schritten. */
1164 const int trackLength = 1750;

1166 /**
 * @brief Texte fuer die Anzeige der Stufen.
 */
1168 const char* stageMessages[] = {
1170     "Keine Stufe",
1172     "Stufe 1",
1174     "Stufe 2",
1176     "Stufe 3",
1178     "Stufe 4",
1180     "Stufe 5",
1182     "Stufe 6",
1184     "Stufe 7",
1186     "Stufe 8",
1188     "Stufe 9",
1190     "Stufe 10"
};

1192 /**
 * @brief Allgemeine Statusmeldungen fuer das OLED-Display.
 */
1194 const char* messages[] = {
1196     "Start...",
1198     "Startbereit",
1200     "Programm laeuft...",
1202     "Abbruch",
1204     ""
};

1206 /**
 * @brief Initialisiert Hardware, Display, LEDs, Taster, Endschafter und
 * Encoder.
 * Diese Funktion wird beim Start des Arduino einmal ausgefuehrt.
 */
1208 void setup() {
1210     stepper.setMaxSpeed(1000);
    stepper.setAcceleration(1000);

    pinMode(PIN_LED_RED, OUTPUT);
    pinMode(PIN_LED_GREEN, OUTPUT);
    pinMode(PIN_LED_BLUE, OUTPUT);

    pinMode(PIN_BUTTON, INPUT_PULLUP);
    pinMode(PIN_ENDSTOP, INPUT_PULLUP);
    pinMode(PIN_SW, INPUT_PULLUP);

    Serial.begin(9600);

```

```

1212   if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
        Serial.println(F("SSD1306 allocation failed"));
1214
        while (true) {
            // Programm bleibt hier stehen, falls das Display nicht gefunden
            // wird.
1216        }
    }
1218
    attachInterrupt(digitalPinToInterrupt(PIN_SW), handleInterrupt, CHANGE);
1220
    display.display();
1222    delay(2000);

    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
1224
    updateDisplay(messages[0], messages[4]);
1226    delay(1000);
1228

    updateDisplay(messages[1], stageMessages[stage]);
    updateLEDs(false, true, false);
1230
1232 }

1234 /**
    * @brief Hauptprogramm des Arduino.
1236    *
    * Die Funktion wird dauerhaft wiederholt.
1238    * Sie liest den Encoder, aktualisiert das Display und startet bei
    * Tastendruck
    * die Referenzfahrt sowie die Demonstrationsfahrt.
1240    */
    void loop() {
1242        long newPosition = encoder.read();

1244        if (newPosition != oldPosition) {
            stage = constrain(newPosition / 4, 1, 10);
            updateDisplay(messages[1], stageMessages[stage]);
            oldPosition = newPosition;
1246        }
1248
1250        int buttonReading = digitalRead(PIN_BUTTON);

1252        if (buttonReading != lastButtonStatus) {
            lastDebounceTime = millis();
1254        }

1256        if ((millis() - lastDebounceTime) > debounceDelay) {
            if (buttonReading != buttonStatus) {
1258                buttonStatus = buttonReading;

1260                if (buttonStatus == LOW) {
                    startProgram();
1262                }
            }
1264        }

1266        lastButtonStatus = buttonReading;
    }
1268

1270 /**
    * @brief Startet den Ablauf aus Referenzfahrt und Bewegungsfahrt.
    *
1272    * Zuerst wird die Referenzfahrt ausgefuehrt.
    * Danach faehrt der Schlitten entsprechend der aktuell eingestellten
    * Stufe.
1274    */
    void startProgram() {

```

```

1276 stepper.setAcceleration(accelerations[0]);
1278 while (!findReference()) {
1280     stepper.setSpeed(-200);
1281     stepper.runSpeed();
1282 }
1283
1284 int currentStage = stage;
1285
1286 updateDisplay(messages[2], stageMessages[currentStage]);
1287 updateLEDs(false, false, true);
1288
1289 for (int i = 0; i < 2; i++) {
1290     stepper.setMaxSpeed(speeds[currentStage]);
1291     stepper.setAcceleration(accelerations[currentStage]);
1292
1293     if (i == 0) {
1294         stepper.move(trackLength + safeSteps);
1295     } else {
1296         stepper.move(trackLength);
1297     }
1298
1299     while (stepper.distanceToGo() != 0) {
1300         stepper.run();
1301     }
1302
1303     stepper.move(-trackLength);
1304
1305     while (stepper.distanceToGo() != 0) {
1306         stepper.run();
1307     }
1308 }
1309
1310 updateDisplay(messages[1], stageMessages[currentStage]);
1311 updateLEDs(false, true, false);
1312 }
1313
1314 /**
1315  * @brief Aktualisiert die drei Status-LEDs.
1316  *
1317  * @param red Zustand der roten LED.
1318  * @param green Zustand der gruenen LED.
1319  * @param blue Zustand der blauen LED.
1320  */
1321 void updateLEDs(bool red, bool green, bool blue) {
1322     digitalWrite(PIN_LED_RED, red ? HIGH : LOW);
1323     digitalWrite(PIN_LED_GREEN, green ? HIGH : LOW);
1324     digitalWrite(PIN_LED_BLUE, blue ? HIGH : LOW);
1325 }
1326
1327 /**
1328  * @brief Aktualisiert den Text auf dem OLED-Display.
1329  *
1330  * @param line1 Erste Zeile auf dem Display.
1331  * @param line2 Zweite Zeile auf dem Display.
1332  */
1333 void updateDisplay(const char* line1, const char* line2) {
1334     display.clearDisplay();
1335     display.setCursor(0, 0);
1336
1337     display.println(line1);
1338
1339     if (line2[0] != '\0') {
1340         display.println(line2);
1341     }
1342
1343     display.display();
1344 }

```

```

1346 /**
1347  * @brief Fuehrt die Referenzfahrt aus und prueft den Endschalter.
1348  *
1349  * Der Motor faehrt so lange in Richtung Endschalter ,
1350  * bis der Endschalter gedrueckt wird.
1351  *
1352  * @return true, wenn der Endschalter erreicht wurde.
1353  * @return false, wenn der Endschalter noch nicht erreicht wurde.
1354  */
1355 bool findReference() {
1356     int endstopReading = digitalRead(PIN_ENDSTOP);
1357
1358     if (endstopReading != lastEndstopStatus) {
1359         lastEndstopDebounceTime = millis();
1360     }
1361
1362     if ((millis() - lastEndstopDebounceTime) > debounceDelay) {
1363         if (endstopReading != endstopStatus) {
1364             endstopStatus = endstopReading;
1365
1366             if (endstopStatus == LOW) {
1367                 stepper.stop();
1368                 return true;
1369             }
1370         }
1371     }
1372
1373     lastEndstopStatus = endstopReading;
1374     return false;
1375 }
1376
1377 /**
1378  * @brief Interrupt-Funktion fuer den Encoder-Taster.
1379  *
1380  * Diese Funktion wird aufgerufen, wenn sich der Zustand des Encoder-
1381  * Tasters aendert.
1382  * Der Motor wird gestoppt und eine Abbruchmeldung wird angezeigt.
1383  */
1384 void handleInterrupt() {
1385     stepper.stop();
1386     updateDisplay(messages[3], messages[4]);
1387     updateLEDs(true, false, false);
1388 }

```

../Code/StepperOLEDEncoder/StepperOLEDEncoder.ino

13.6. Einfacher Code

Für den einfachen Code wird der Treiber Code: „Einfacher Funktionstest des A4988-Schrittmotortreibers“ 12.1 genutzt.

13.7. Test

Für die Test wird der Treiber Test „Testprogramm für den A4988-Schrittmotortreiber“ 12.3 verwendet.

14. DC-DC-Abwärtswandler

In elektronischen Systemen werden unterschiedliche Spannungsbereiche benötigt. Mikrocontroller, Sensoren und weitere elektronische Baugruppen arbeiten mit niedrigen Gleichspannungen wie 3,3 V oder 5 V, während andere Komponenten höhere Versorgungsspannungen benötigen. Spannungswandler ermöglichen die Anpassung vorhandener Versorgungsspannungen an die jeweiligen Verbraucher.

Für die Versorgung von Mikrocontrollern und Logikschaltungen wird ein DC-DC-Abwärtswandler eingesetzt. Dieser reduziert eine höhere Eingangsspannung auf eine stabile Ausgangsspannung. Im Projekt wird hierfür ein Buck-Converter-Modul auf Basis des LM2596 beziehungsweise MP2307 verwendet.

Der Arduino Uno kann laut Datenblatt über den VIN-Anschluss mit einer Eingangsspannung von 6 V bis 20 V betrieben werden. Für den direkten Betrieb der Logikschaltungen wird jedoch eine stabile Versorgungsspannung von 5 V benötigt [Ard26b]. Der eingesetzte DC-DC-Abwärtswandler übernimmt daher die Spannungsanpassung zwischen Netzteil und Mikrocontroller.

Im Gegensatz zu linearen Spannungsreglern arbeitet ein Abwärtswandler im Schaltbetrieb. Dadurch entstehen geringere Verlustleistungen und eine reduzierte Wärmeentwicklung. Gleichzeitig werden hohe Wirkungsgrade erreicht [Sch20].

14.1. Eingesetzter Spannungswandler

Für diese Anwendung wird ein einstellbarer DC-DC-Abwärtswandler verwendet, der auf dem LM2596 beziehungsweise MP2307 basiert. Das Modul reduziert die Eingangsspannung des Netzteils auf eine stabile Ausgangsspannung zur Versorgung des Arduino Uno sowie weiterer elektronischer Baugruppen.

Der eingesetzte Wandler besitzt eine kompakte Bauform, eine hohe Effizienz und lässt sich über Schraubklemmen in die Spannungsversorgung des Projekts integrieren. Aufgrund der Strombelastbarkeit eignet sich das Modul für Mikrocontroller-Anwendungen und kleinere elektronische Verbraucher.

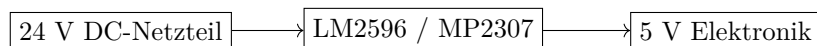


Abbildung 14.1.: Blockschaltbild der Spannungsversorgung

14.2. Spezifikation

Das verwendete Modul besitzt folgende technische Eigenschaften [Mp2]:

Tabelle 14.1.: Technische Spezifikationen des LM2596-/MP2307-Abwärtswandlers

Eigenschaft	Wert
Eingangsspannung	4,75 V bis 24 V DC
Ausgangsspannung	0,93 V bis 18 V DC
Ausgangsstrom	bis 2,5 A dauerhaft
Spitzenstrom	bis 4 A
Wirkungsgrad	bis 98 %
Schaltfrequenz	ca. 150 kHz
Abmessungen	ca. 45 mm × 32 mm

Der Arduino Uno unterstützt laut Datenblatt Eingangsspannungen von 6 V bis 20 V am Anschluss [VIN](#) [Ard26b]. Die Versorgung über einen externen DC-DC-Wandler reduziert die Belastung des integrierten Spannungsreglers und verbessert die Energieeffizienz des Gesamtsystems.

14.3. Funktionsprinzip

Ein DC-DC-Abwärtswandler, auch Buck Converter genannt, reduziert eine Gleichspannung auf eine niedrigere Ausgangsspannung. Die Spannungsregelung erfolgt mithilfe eines schnell schaltenden MOSFETs. Durch das periodische Ein- und Ausschalten entsteht ein pulsierender Spannungsverlauf, der durch Induktivitäten und Kondensatoren geglättet wird [Sch20].

Der Betrieb des Abwärtswandlers erfolgt in mehreren Schritten. Zunächst schaltet der MOSFET die Eingangsspannung mit hoher Frequenz ein und aus. Während der Einschaltphase wird Energie in einer Induktivität gespeichert. In der Ausschaltphase gibt die Spule die gespeicherte Energie an den Ausgang ab. Kondensatoren glätten die entstehenden Spannungsschwankungen und stabilisieren die Ausgangsspannung.

Die Regelung der Ausgangsspannung erfolgt über das Tastverhältnis der Pulsweitenmodulation. Für den idealen Abwärtswandler gilt:

$$\frac{U_a}{U_e} = v_T$$

Die Ausgangsspannung U_a ergibt sich aus der Eingangsspannung U_e und dem Tastverhältnis v_T der PWM. Die Ausgangsspannung hängt somit direkt vom Tastverhältnis ab. Durch die hohe Schaltfrequenz kann eine kompakte und effiziente Spannungsregelung realisiert werden [Sch20].

14.3.1. Wirkungsgrad

Der Wirkungsgrad beschreibt das Verhältnis zwischen abgegebener und aufgenommener Leistung:

$$\eta = \frac{P_{ab}}{P_{zu}} \cdot 100$$

Dabei entspricht P_{ab} der abgegebenen Leistung und P_{zu} der aufgenommenen Leistung. Durch den Schaltbetrieb erreicht der Abwärtswandler höhere Wirkungsgrade als lineare Spannungsregler. Beim verwendeten Modul sind Wirkungsgrade von bis zu 98 % möglich [Mp2].

14.3.2. Verlustleistungen

Trotz des hohen Wirkungsgrades entstehen innerhalb des Wandlers verschiedene Verlustleistungen. Dazu gehören Schaltverluste des MOSFETs, ohmsche Leitungsverluste sowie Verluste der Induktivität und der Kondensatoren.

Die Verlustleistung wird überwiegend in Wärme umgewandelt und führt zu einer Erwärmung der elektronischen Bauteile. Besonders bei höheren Lastströmen müssen diese Verluste berücksichtigt werden [Sch20].

14.4. Schutzvorrichtungen

Der eingesetzte DC-DC-Wandler verfügt über Schutzmechanismen zum Schutz der Elektronik.

14.4.1. Überstromschutz

Bei zu hohen Ausgangsströmen wird die Strombegrenzung aktiviert, um Schäden am Wandler und den angeschlossenen Komponenten zu vermeiden.

14.4.2. Kurzschlusschutz

Im Falle eines Kurzschlusses schützt sich der Wandler gegen zu hohe Ströme am Ausgang.

14.4.3. Thermischer Schutz

Bei zu hoher Temperatur reduziert der Wandler seine Leistung oder schaltet sich zum Schutz vor Überhitzung ab.

14.5. Montage

Der DC-DC-Abwärtswandler wird über Schraubklemmen mit der Spannungsversorgung und den elektronischen Verbrauchern verbunden.

Die Anschlüsse des Moduls sind:

- **IN+**: positive Eingangsspannung
- **IN-**: Masse der Eingangsspannung
- **OUT+**: positive Ausgangsspannung
- **OUT-**: Masse der Ausgangsspannung

Die Eingangsspannung des Netzteils wird an **IN+** und **IN-** angeschlossen. Die reduzierte Ausgangsspannung liegt an **OUT+** und **OUT-** an. Vor dem Anschluss an den Arduino Uno und weitere elektronische Baugruppen wird die Ausgangsspannung mit einem Multimeter geprüft und über das Potentiometer des Wandlers auf 5 V eingestellt.

Anschließend kann die Ausgangsspannung mit dem Arduino Uno sowie weiteren elektronischen Baugruppen verbunden werden. Dabei ist auf eine gemeinsame Masseverbindung der beteiligten Baugruppen zu achten.

14.6. Weiterführende Literatur

- Schlenz, Ulrich: *Schaltnetzteile und ihre Peripherie*. Springer Vieweg, 2020. [Sch20]
- Erickson, Robert W. und Maksimovic, Dragan: *Fundamentals of Power Electronics*. Springer, 2020.
- Arduino: *Arduino Uno Rev3 Datasheet*. Technisches Datenblatt, 2025. [Ard26b]
- MP2307 / LM2596: *DC-DC Converter Buck Module*. Technisches Datenblatt. [Mp2]

15. Komplettes System

15.1. Beschreibung des Gesamtsystems

Das Gesamtsystem besteht aus einer mechanischen Achskonstruktion, einer elektrischen Leistungsversorgung, einer Mikrocontrollersteuerung und einem thermischen Schneidwerkzeug.

Die zentrale Funktion des Systems besteht darin, einen elektrisch erwärmten Schneiddraht entlang einer definierten Bahn durch Schaumstoff zu führen. Die Bahnbewegung wird durch Schrittmotoren erzeugt. Die Ansteuerung der Schrittmotoren erfolgt über den Arduino Uno, das CNC Shield V3 und die eingesetzten A4988-Schrittmotortreiber. Der Heißdraht wird getrennt von der Motorsteuerung über ein MOSFET-PWM-Schaltmodul geschaltet.

Die Maschine besitzt damit zwei wesentliche Wirkbereiche. Der erste Bereich ist die Bewegungssteuerung der Achsen. Der zweite Bereich ist die thermische Trennung des Werkstoffs durch den Heißdraht. Beide Bereiche müssen aufeinander abgestimmt werden, da die Schnittqualität sowohl von der Achsbewegung als auch von der Heizleistung abhängt.

Abbildung 15.1 zeigt den CNC-Heißdrahtschneider als Automatisierungssystem. Die Darstellung trennt Steuerung, Maschine, Signalfluss und Leistungsfluss.

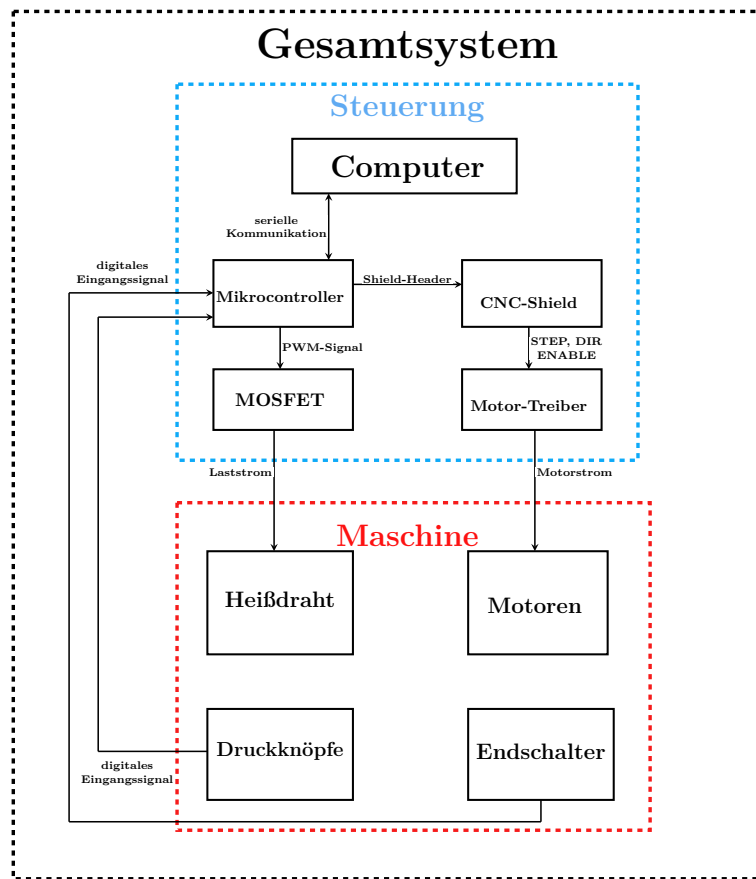


Abbildung 15.1.: Schematische Darstellung des CNC-Heißdrahtschneiders als Automatisierungssystem

15.2. Design und Aufbau

Der Aufbau ist modular ausgeführt. Die mechanische Baugruppe trägt die Achsen und den Schneiddraht. Die elektrische Baugruppe enthält die Steuer- und Leistungskomponenten. Die Software erzeugt die Steuerbefehle für die Bewegung und stellt die Bedienbeziehungsweise Testfunktionen bereit.

Der Arduino Uno dient als zentrale Steuereinheit. Auf ihm befindet sich das CNC Shield V3, das die Steuerpins des Arduino auf die Anschlüsse der Schrittmotortreiber, Motoren und Endschalter verteilt. Die A4988-Schrittmotortreiber übernehmen die Leistungsansteuerung der Schrittmotoren. Die Motoren bewegen die Achsen der Maschine und führen dadurch den Heißdraht entlang der gewünschten Kontur.

Der Heißdraht wird nicht direkt über den Arduino versorgt. Stattdessen wird ein MOSFET-PWM-Schaltmodul verwendet. Dieses Modul schaltet den Laststrom des Heißdrahts. Der Arduino liefert nur das Steuersignal. Dadurch werden Logikteil und Lastteil funktional getrennt.

Die Spannungsversorgung ist ebenfalls aufgeteilt. Die Motoren und der Heißdraht werden mit einer Nennspannung von 24 V versorgt. Der Arduino und die Logiksignale arbeiten mit niedrigeren Spannungen. Für zusätzliche Verbraucher oder Hilfsschaltungen kann ein DC-DC-Abwärtswandler eingesetzt werden.

15.3. Hardware-Schnittstellen

Die Hardware-Schnittstellen verbinden die einzelnen Baugruppen elektrisch und mechanisch miteinander. Die Signalleitungen übertragen Steuerinformationen. Die Leistungsleitungen versorgen Motoren und Heißdraht.

Tabelle 15.1.: Hardware-Schnittstellen des Gesamtsystems

Schnittstelle	Verbindung	Funktion
Arduino Uno – CNC Shield V3	Steckverbindung	Übergabe der digitalen Steuerpins an das Shield
CNC Shield – A4988	Treibersteckplätze	Übergabe von STEP-, DIR- und ENABLE-Signalen
A4988 – Schrittmotor	Motoranschlüsse	Leistungsansteuerung der Motorwicklungen
CNC Shield – Endschalter	Digitale Eingänge	Erfassung von Referenz- oder Endpositionen
Arduino/CNC Shield – MOSFET-Modul	PWM-Signal und GND	Ansteuerung der Heißdraht-Leistungstufe
MOSFET-Modul – Heißdraht	Lastanschluss	Schalten des Heißdrahtstroms
Netzteil – CNC Shield	Versorgungsklemme	Motorspannung für die Schrittmotortreiber
Netzteil – MOSFET-Modul	Versorgungsklemme	Versorgung des Heißdrahtkreises
DC-DC-Wandler – Zusatzmodule	Versorgungsausgang	Bereitstellung angepasster Hilfsspannungen

Für alle Signale, die zwischen Steuerung und Leistungsmodulen übertragen werden, ist ein gemeinsamer Massebezug erforderlich.

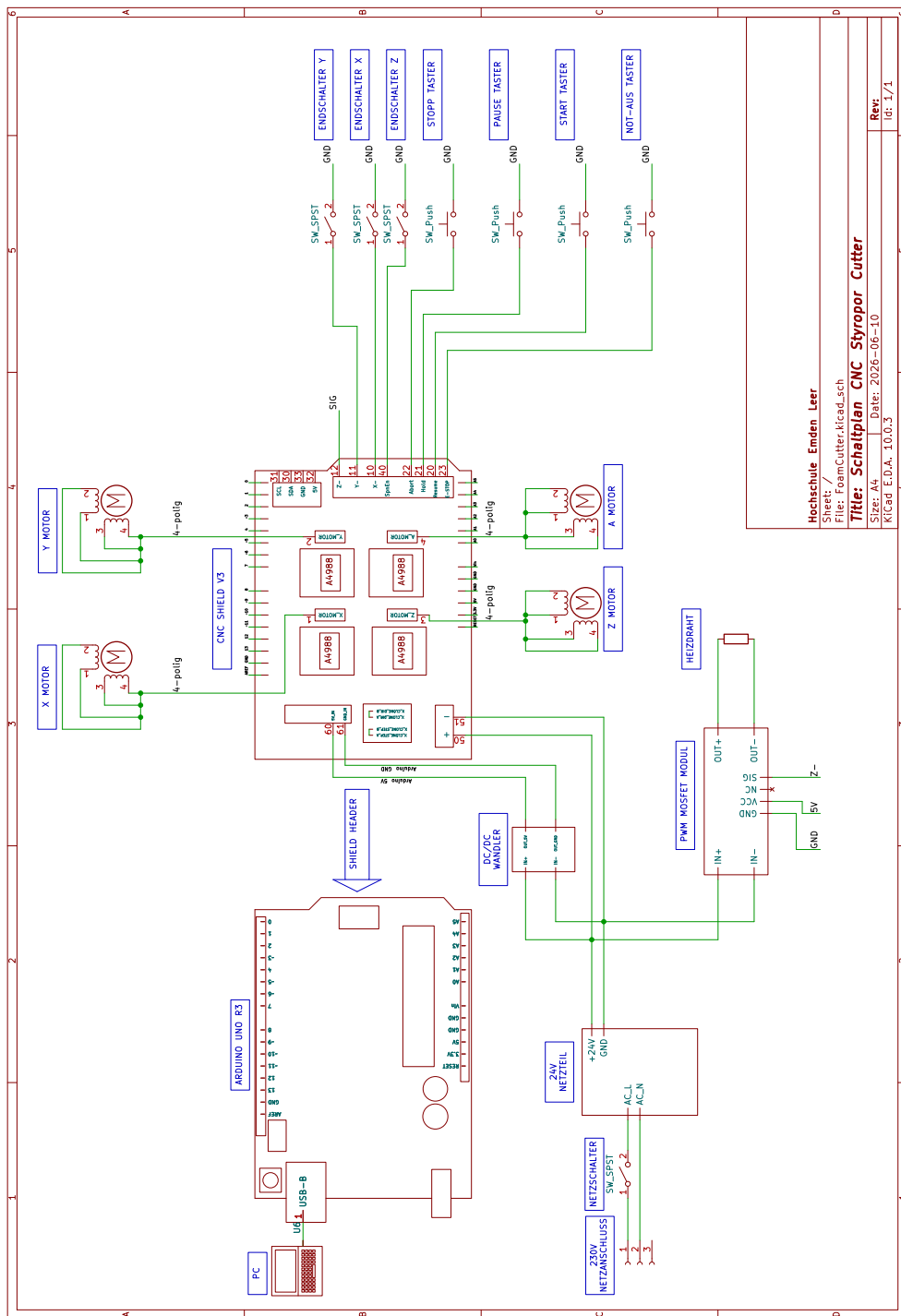


Abbildung 15.2.: Schaltplan des CNC-Heißdrahtschneiders

15.4. Eigenschaften des Gesamtsystems

Das Gesamtsystem besitzt vier mechanische Achsen. Davon werden drei Achsen unabhängig angesteuert. Die A-Achse ist als Y-Klon ausgeführt und bewegt sich synchron zur Y-Achse. Dadurch kann der Heißdraht auf beiden Seiten geführt werden, ohne dass für die zweite Y-Seite eine eigene Bewegungsberechnung notwendig ist.

Tabelle 15.2.: Technische Kenndaten des Gesamtsystems

Kenngroße	Wert	Bemerkung
Steuerung	Arduino Uno R3	zentrale Mikrocontrollersteuerung
Firmware	GRBL 1.1h	CNC-Firmware auf dem Arduino Uno R3
Erweiterungsplatine	CNC Shield V3	Verteilung der Achs- und Endschaltersignale
Motortreiber	A4988	Schrittmotortreiber für die Achsantriebe
Anzahl Achsen	4	X, Y, Z und A
Unabhängig gesteuerte Achsen	3	A-Achse als Y-Klon
Antrieb X/Z	Zahnriemen	Riemensteigung 2 mm
Antrieb Y/A	Trapezgewindespindel	Steigung ca. 2 mm/U
Riemenscheibe X/Z	20 Zähne	ergibt 40 mm Verfahrweg pro Motorumdrehung
Versorgungsspannung	24 V DC	gemeinsame Nennspannung der Leistungsverorgung
Motorspannung	24 V	Nennwert
Heißdrahtspannung	24 V	Nennwert
Heißdrahtsteuerung	PWM-MOSFET-Modul	Schalten des Heißdrahtkreises
Drahtdurchmesser	0,4 mm	Herstellerwert
PWM-Bereich Heißdraht	0–15 %	verwendeter Einstellbereich
Schnittfugenbreite	ca. 1–2 mm	abhängig von Vorschubgeschwindigkeit und Heizleistung

Das Gesamtsystem besitzt folgende funktionale Eigenschaften:

- rechnergestützte Bewegung des Heißdrahts über Schrittmotorachsen,
- getrennte Ansteuerung von Bewegungs- und Heißdrahtkreis,
- Verwendung von STEP-, DIR- und ENABLE-Signalen für die Achsantriebe,
- Mikroschrittauflösung von 1/2 an allen vier Achsen,
- Leistungsbeeinflussung des Heißdrahts über ein PWM-MOSFET-Modul,
- Referenzfahrt über Endschalter an X-, Y- und Z-Achse,
- dauerhafte Motorhaltefunktion über \$1 = 255.

Die Genauigkeit des Systems hängt nicht nur von den elektronischen Komponenten ab. Auch mechanische Faktoren wie Achsspiel, Reibung, Drahtspannung und Rahmensteifigkeit beeinflussen das Ergebnis. Zusätzlich wirken thermische Prozessgrößen wie Heißleistung, Vorschubgeschwindigkeit und Materialdicke auf die Schnittqualität.

15.5. Software-Schnittstellen

Die Programmierung und Inbetriebnahme erfolgen über einen Computer mit Arduino IDE. Der Arduino Uno wird über USB mit dem Computer verbunden. Über diese

Verbindung werden Programme auf den Mikrocontroller übertragen. Zusätzlich kann die serielle Schnittstelle zur Ausgabe von Status- und Testinformationen verwendet werden.

Für den CNC-Betrieb wird GRBL 1.1h verwendet. Die Firmware verarbeitet Bewegungsbefehle und erzeugt daraus die STEP-, DIR- und ENABLE-Signale für die Schrittmotortreiber. Für Funktionstests können zusätzlich einfache Arduino-Sketches eingesetzt werden, mit denen einzelne Baugruppen wie Motoren, Endschalter, OLED-Anzeige oder Heißdraht separat geprüft werden.

Für einen CNC-nahen Betrieb können Bewegungsdaten in Form von G-Code oder vergleichbaren Koordinateninformationen verwendet werden. Dabei beschreibt die Software Zielpositionen, Bewegungsrichtungen und Vorschubgeschwindigkeiten. Die Steuerung muss diese Daten in zeitlich koordinierte Schritimpulse für die Achsen umsetzen.

Die softwareseitigen Schnittstellen:

- USB-Verbindung zwischen Computer und Arduino,
- serielle Kommunikation zur Übertragung von Programm- und Statusdaten,
- GRBL 1.1h als Firmware zur Bewegungssteuerung,
- digitale Ausgangssignale für STEP, DIR und ENABLE,
- PWM-Ausgang zur Ansteuerung des MOSFET-Schaltmoduls,
- digitale Eingangssignale der Endschalter.

15.6. Installation des Gesamtsystems

Die Installation beginnt mit der mechanischen Montage des Rahmens, der Achsen und der Drahtführung. Der Schneiddraht wird mechanisch befestigt und gespannt. Dabei muss geprüft werden, ob der Draht frei durch den Arbeitsbereich geführt werden kann und nicht an Bauteilen schleift.

Anschließend werden Arduino Uno und CNC Shield montiert. Die A4988-Schrittmotortreiber werden in die vorgesehenen Steckplätze des Shields eingesetzt. Dabei muss auf die korrekte Ausrichtung der Treibermodule geachtet werden. Eine falsche Orientierung kann die Treiber beschädigen.

Danach werden die Schrittmotoren an die Motoranschlüsse des CNC Shields angeschlossen. Die Endschalter werden mit den vorgesehenen Eingängen verbunden. Das MOSFET-PWM-Schaltmodul wird mit dem Steuersignal, Masse und dem Heißdrahtkreis verbunden. Abschließend werden die Spannungsversorgung für Motoren, Logik und Heißdraht geprüft.

Vor dem ersten Betrieb dürfen Motor- und Heißdrahtversorgung noch nicht dauerhaft aktiv betrieben werden. Zuerst müssen alle Leitungen, Steckverbindungen, Polaritäten und Masseverbindungen kontrolliert werden.

15.7. Konfiguration

Nach der Installation wird das System konfiguriert. Zuerst wird die Arduino IDE eingerichtet und das verwendete Board ausgewählt. Danach wird der passende Port eingestellt und die Firmware beziehungsweise ein Testsketch übertragen.

Die Schrittmotortreiber müssen vor dem Betrieb eingestellt werden. Besonders wichtig ist die Strombegrenzung. Eine zu hohe Einstellung kann zu starker Erwärmung oder zur Abschaltung des Treibers führen. Eine zu niedrige Einstellung kann Schritverluste verursachen. Die Mikroschrittauflösung ist an allen vier Achsen auf 1/2 eingestellt.

Tabelle 15.3.: GRBL- und Bewegungsparameter des Gesamtsystems

Parameter	Wert	Bemerkung
Firmware	GRBL 1.1h	CNC-Firmware auf dem Arduino Uno R3
Schritte pro Millimeter X	\$100 = 10	Kalibrierwert X-Achse
Schritte pro Millimeter Y	\$101 = 50	Kalibrierwert Y-Achse
Schritte pro Millimeter Z	\$102 = 10	Kalibrierwert Z-Achse
Schritte pro Millimeter A	50	A-Achse als Y-Klon
Maximale Verfahrgeschwindigkeit X	2000 mm/min	GRBL-Wert für X-Achse
Maximale Verfahrgeschwindigkeit Y	2000 mm/min	GRBL-Wert für Y-Achse
Maximale Verfahrgeschwindigkeit Z	2000 mm/min	GRBL-Wert für Z-Achse
Beschleunigung X/Y/Z	30 mm/s ²	eingestellter Beschleunigungswert
Motorhaltefunktion	\$1 = 255	Motoren bleiben dauerhaft bestromt

Die Achsen wurden mit den angegebenen Schritten pro Millimeter kalibriert. Der rechnerische Wert wird durch Messfahrten überprüft. Wenn die gemessene Strecke vom Sollwert abweicht, muss der Schrittwert angepasst werden.

Auch die Heißdrahtleistung muss eingestellt werden. Dazu wird die PWM-Ansteuerung zunächst mit niedriger Leistung getestet. Der verwendete Einstellbereich liegt bei 0–15 %. Danach werden Probeschnitte durchgeführt, um Heizleistung und Vorschubgeschwindigkeit aufeinander abzustimmen.

Die Referenzfahrt ist vorhanden und aktiviert. Es werden drei Endschalter für X-, Y- und Z-Achse eingesetzt. Die Referenzposition liegt oben und hinten.

15.8. Daten, Strukturen und Typen

Im Gesamtsystem werden verschiedene Daten und Parameter verwendet. Sie beschreiben die Achsbewegung, die elektrische Ansteuerung und die Bewertung des Schneidprozesses.

Tabelle 15.4.: Daten und Parameter des Gesamtsystems

Parameter	Wert/Typ	Bedeutung
Schrittzahl	Ganzzahl	Anzahl der ausgegebenen STEP-Impulse
Richtung	digitaler Zustand	Bewegungsrichtung einer Achse über DIR
Mikroschrittauflösung	1/2	Einstellung an allen vier Achsen
Vorschubgeschwindigkeit	2000 mm/min	verwendete beziehungsweise maximale Verfahrgeschwindigkeit
Schritte pro Millimeter	X: 10, Y: 50, Z: 10, A: 50	Umrechnung zwischen Steuerimpulsen und realem Verfahrensweg
PWM-Wert Heißdraht	0–15 %	Tastverhältnis zur Beeinflussung der Heizleistung
Endschalterzustand	digitaler Eingangswert	Erkennung der Referenzposition
Drahtdurchmesser	0,4 mm	Herstellerwert des Schneiddrahts
Schnittfugenbreite	ca. 1–2 mm	Bewertungsgröße für die Schnittqualität
Werkstückabmessung	Messwert	Vergleich zwischen Soll- und Ist-Kontur

15.9. Einschränkungen und offene Punkte

Das System besitzt mehrere Einschränkungen, die beim Betrieb berücksichtigt werden müssen. Die verwendeten Schrittmotoren arbeiten im einfachen Aufbau ohne direkte Positionsrückmeldung. Die Steuerung erkennt daher nicht automatisch, ob ein Motor Schritte verliert. Schrittverluste müssen durch geeignete Parameter, leichtgängige Mechanik und Funktionstests vermieden werden.

Die Heißdrahttemperatur wird nicht gemessen. Die Einstellung erfolgt über die elektrische Ansteuerung und die Bewertung der Probeschnitte. Dadurch ist die Heizleistung material- und erfahrungsabhängig. Unterschiedliche Schaumstoffarten oder Materialdicken können andere Einstellungen erfordern.

Weitere offene Punkte sind:

- Maschinengewicht messen und dokumentieren,
- Leistungsaufnahme im Betrieb messen und dokumentieren,
- endgültige Standardwerte für Vorschub und PWM für verschiedene Materialien festlegen,
- Wiederholgenauigkeit über mehrere Schnitte prüfen,
- direkte Temperatur- oder Strommessung am Heißdraht ergänzen,
- Kabelführung und Zugentlastung weiter verbessern,
- Schutzabdeckung für heiße und bewegte Bereiche ergänzen.

15.10. Abmaße

Die Abmaße des Gesamtsystems ergeben sich aus dem mechanischen Rahmen, dem nutzbaren Arbeitsbereich und der Anordnung der elektrischen Baugruppen. Dabei ist zwischen Außenmaß, Arbeitsbereich und Drahtspannweite zu unterscheiden.

Tabelle 15.5.: Abmaße des Gesamtsystems

Größe	Wert	Bemerkung
Gesamtlänge	650 mm	Außenmaß der Maschine
Gesamtbreite	707 mm	Außenmaß der Maschine
Gesamthöhe	583 mm	Außenmaß einschließlich Drahtführung
Nutzbarer Arbeitsbereich X	345 mm	maximaler Verfahrweg beziehungsweise maximale Werkstückbreite
Nutzbarer Arbeitsbereich Y	390 mm	maximaler Verfahrweg beziehungsweise maximale Werkstückhöhe
Freie Drahtlänge	min. 450 mm, max. 688 mm	freie Länge des gespannten Schneid-drahts
Grundplattenmaß	537 mm × 457 mm	Abmessung der Montage- oder Arbeitsplatte
Maschinengewicht	noch zu messen	offener Messwert
Leistungsaufnahme	noch zu messen	offener Messwert im Betrieb

Die maximale Werkstückbreite beträgt ca. 345 mm. Die maximale Werkstückhöhe beträgt ca. 390 mm. Die maximale Drahtspannweite beträgt 688 mm. Diese Werte begrenzen den nutzbaren Arbeitsraum der Maschine.

15.11. Schlussfolgerung

Das Gesamtsystem verbindet die zuvor beschriebenen Einzelkomponenten zu einem funktionsfähigen CNC-Heißdrahtschneider. Der Arduino Uno übernimmt die Steuerlogik, das CNC Shield verteilt die Achssignale, die A4988-Treiber versorgen die Schrittmotoren und das MOSFET-PWM-Schaltmodul schaltet den Heißdrahtkreis. Die mechanische Baugruppe führt den Draht entlang der gewünschten Kontur.

Ein einsatzfähiger Zustand liegt vor, wenn die Achsen reproduzierbar verfahren, der Heißdraht kontrolliert erwärmt wird, die Endschalter zuverlässig reagieren und Probe-schnitte eine nachvollziehbare Schnittqualität zeigen. Die abschließende Bewertung erfolgt daher nicht nur über elektrische Funktionstests, sondern über das reale Schnei-dergebnis am Werkstück.

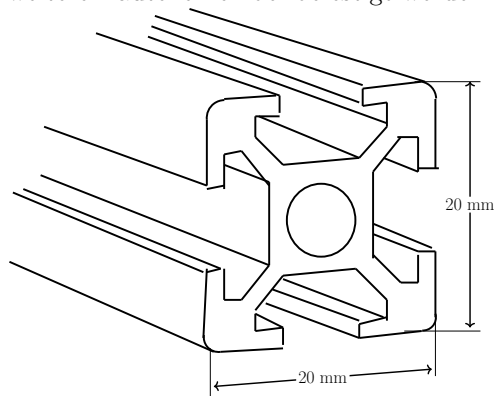
16. Hardware

16.0.1. Grundrahmen

Der Grundrahmen bildet die tragende Struktur von ELMAR. Er wird aus Aluminiumprofilen, Blechen, Eckwinkeln und zusätzlichen Druckteilen aufgebaut.

Creality Aluminium-Extrusionsprofil 20 x 20 mm

Das Aluminiumprofil dient als leichtes Rahmenelement. Durch die Profilhuten können weitere Bauteile flexibel befestigt werden.



Anzahl: 6 Stück

Creality Aluminium-Extrusionsprofil 40 x 20 mm

Das Aluminiumprofil wird für stabilere Bereiche des Rahmens verwendet. Durch den größeren Querschnitt erhöht es die Steifigkeit der Konstruktion.

Spezifikation: Maße: 40 x 20 x 509 mm Material: Aluminium

Anzahl: 4 Stück

Creality Aluminium-Extrusionsprofil 40 x 20 mm, ausgeklinkt

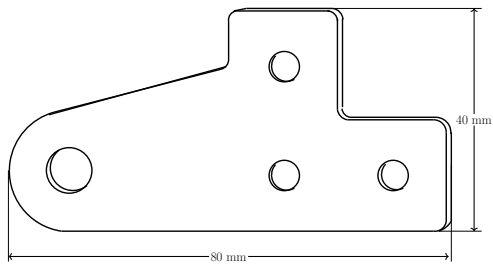
Das ausgeklinkte Aluminiumprofil wird an angepassten Stellen des Rahmens eingesetzt. Die Ausklinkung ermöglicht die Montage in Bereichen mit begrenztem Bauraum.

Spezifikation: Maße: 40 x 20 x 553 mm Material: Aluminium Besonderheit: ausgeklinkt

Anzahl: 2 Stück

Creality Blech 4 mm für Riemenrolle

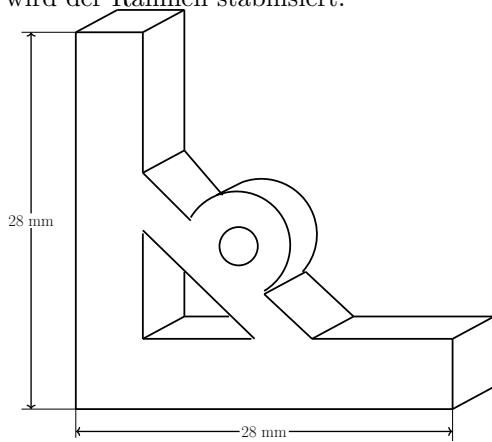
Das Blech dient als Halterung für die Riemenrolle. Es fixiert die Rolle und unterstützt die Kraftübertragung des Riemenantriebs.



Anzahl: 2 Stück **Spezifikation:** Maße: 40 x 80 mm Materialstärke: 4 mm Material: Stahl

Creality Eckwinkel

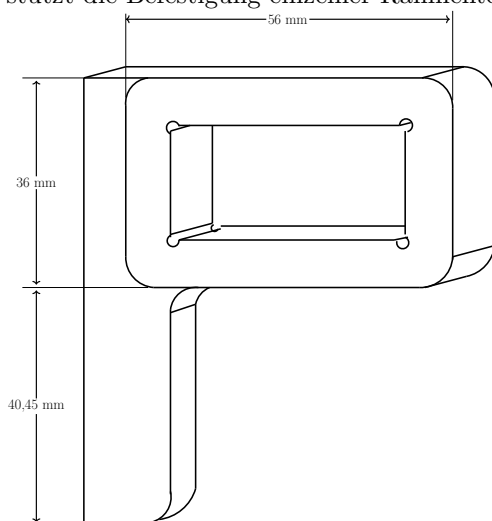
Die Eckwinkel verbinden die Aluminiumprofile rechtwinklig miteinander. Dadurch wird der Rahmen stabilisiert.



Spezifikation: Maße: 28 x 28 mm Materialstärke: 4 mm Material: Aluminium **Anzahl:** 4 Stück

Druckteil Eckhalter 1

Der Eckhalter dient als zusätzliches Verbindungselement am Grundrahmen. Er unterstützt die Befestigung einzelner Rahmenteile.



Spezifikation: Maße: 28 x 28 mm Materialstärke: 4 mm Material: Aluminium **Anzahl:** 1 Stück

Druckteil Eckhalter 2

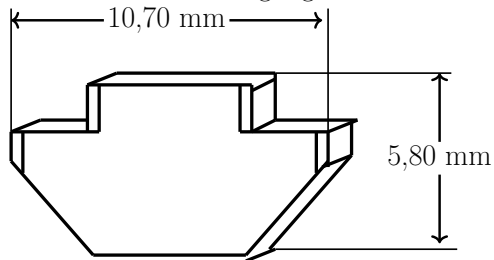
Der Eckhalter dient als weiteres Verbindungselement am Grundrahmen. Er wird zur zusätzlichen Befestigung oder Ausrichtung verwendet.

Spezifikation: Maße: 76,45 x 56 mm Material: PETG

Anzahl: 1 Stück

Creality Nutenstein

Der Nutenstein wird in die Nut der Aluminiumprofile eingesetzt. Er dient als Gewindeelement zur Befestigung von Schrauben.



8 Stück

Spezifikation: Material: PETG **Anzahl:**

Druckteil Nutenstein-Block

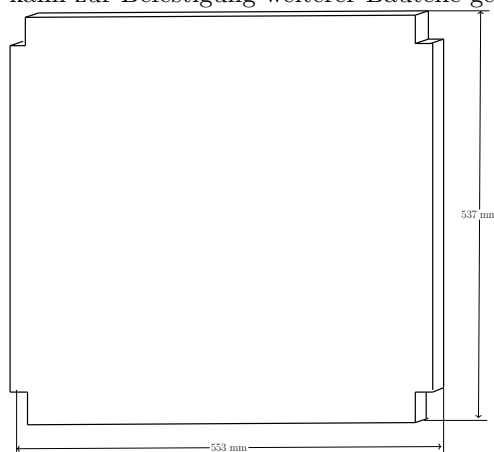
Der Nutenstein-Block dient als zusätzliches Befestigungselement an den Profilenuten. Er ermöglicht die Montage weiterer Bauteile am Rahmen.

Spezifikation: Material: PETG

Anzahl: 4 Stück

Holzplatte

Die Holzplatte dient als Montage- oder Auflagefläche innerhalb der Konstruktion. Sie kann zur Befestigung weiterer Bauteile genutzt werden.



Anzahl: 1 Stück

Teil III.

Domain Knowledge - Tools

17. Softwareseitiges Domänenwissen

Dieses Kapitel beschreibt die softwareseitigen Fachbegriffe und Zusammenhänge, die für die Bewegungserzeugung eines CNC-Heißdrahtschneiders erforderlich sind. Im Mittelpunkt steht die Umwandlung einer geometrischen Kontur in steuerbare Achsbewegungen.

Für die softwareseitige Betrachtung sind Koordinatensystem, Nullpunkt, Schritt-Millimeter-Kalibrierung, Vorschubgeschwindigkeit und Achskoordination relevant. Diese Größen bestimmen, ob eine digitale Kontur maßstäblich und reproduzierbar auf das Werkstück übertragen werden kann.

Die Grundlage bildet die computergestützte Fertigung. Dabei werden digitale Daten genutzt, um Fertigungsinformationen abzuleiten und Maschinenbewegungen vorzubereiten [Heh20]. Für die Bewertung des Ergebnisses sind außerdem Maßhaltigkeit, Prozessparameter und reproduzierbare Bearbeitungsabläufe relevant [FS22].

17.1. Digitale Prozesskette

Der Arbeitsablauf beginnt mit einer vorgegebenen Kontur. Diese Kontur kann als Skizze, technische Zeichnung oder digitale Geometrie vorliegen. Im nächsten Schritt wird sie in eine maschinenverständliche Form gebracht. In CNC-Systemen erfolgt dies typischerweise über Bewegungsbefehle, die Positionen, Richtungen und Vorschubgeschwindigkeiten beschreiben.

Die digitale Prozesskette lässt sich für das Projekt in mehrere Schritte einteilen:

- Festlegen der gewünschten Werkstückgeometrie,
- Überführen der Geometrie in Bewegungsdaten,
- Prüfen von Maßstab, Nullpunkt und Achsrichtung,
- Übertragen der Bewegungsdaten an die Steuerung,
- Ausführen der Achsbewegung an der Maschine,
- Bewerten des erzeugten Schnitts am Werkstück.

Diese Kette entspricht dem Grundgedanken der computerunterstützten Fertigung, bei der digitale Daten zur Steuerung realer Fertigungsprozesse eingesetzt werden [Heh20]. Für das Projekt ist dabei entscheidend, dass die digitale Geometrie nicht nur erzeugt, sondern auch korrekt in reale Achsbewegungen umgesetzt wird.

17.2. CNC-Steuerung und Bewegungsdaten

CNC steht für *Computerized Numerical Control* und bezeichnet eine computergestützte numerische Steuerung von Maschinenbewegungen. Die Bewegung wird dabei nicht manuell geführt, sondern über Zahlenwerte und Steuerbefehle beschrieben. Die Steuerung verarbeitet diese Bewegungsinformationen und setzt sie in Signale für die Achsantriebe um.

In der Praxis werden CNC-Bewegungen häufig mit G-Code beschrieben. Dabei enthalten die Befehle Informationen zu Zielkoordinaten, Bewegungsart und Vorschubgeschwindigkeit. Für den CNC-Heißdrahtschneider ist entscheidend, dass diese Bewegungsdaten zu den realen Achsen, zur eingestellten Schritt-Millimeter-Kalibrierung und zum nutzbaren Arbeitsbereich passen.

17.3. Koordinatensystem, Nullpunkt und Referenzfahrt

Damit eine digitale Kontur richtig auf das reale Werkstück übertragen werden kann, benötigt die Maschine ein definiertes Koordinatensystem. Der Nullpunkt legt fest, von welcher Position aus die Bewegungen berechnet werden. Stimmen digitaler Nullpunkt und reale Startposition nicht überein, verschiebt sich die gesamte Kontur auf dem Werkstück.

Eine Referenzfahrt dient dazu, eine definierte Ausgangsposition der Maschine herzustellen. Dabei fährt eine Achse in Richtung eines Endschalters, bis dieser betätigt wird. Die Steuerung kann diese Position als Referenz verwenden. Dadurch wird die Startposition reproduzierbarer als bei einer rein manuellen Positionierung.

Im Projekt ist die Referenzfahrt besonders relevant, weil Schrittmotoren im einfachen Aufbau keine absolute Positionsrückmeldung liefern. Die Steuerung kennt nur die ausgegebenen Schritte, nicht aber automatisch die tatsächliche mechanische Position. Eine definierte Referenzposition reduziert dieses Problem und erleichtert wiederholbare Schnitte.

17.4. Schritt-Millimeter-Kalibrierung

Die Schrittmotortreiber erhalten Schrittpulse. Jeder Impuls bewegt den Motor um einen Schritt oder Mikroschritt. Damit daraus ein realer Fahrweg entsteht, muss bekannt sein, wie viele Schritte einem Millimeter Achsbewegung entsprechen.

Die Umrechnung hängt von mehreren Faktoren ab:

- Schrittwinkel des Motors,
- eingestellter Mikroschrittbetrieb,
- mechanische Übersetzung der Achse,
- Spindelsteigung oder wirksamer Riemenumfang,
- Schlupf, Spiel und mechanische Toleranzen.

Schrittmotoren bewegen sich in diskreten Winkelschritten und eignen sich dadurch für Positionieraufgaben [Sch17]. Für eine CNC-Anwendung reicht diese Eigenschaft allein jedoch nicht aus. Der rechnerische Schrittwert muss durch reale Messfahrten überprüft werden. Dazu wird eine bekannte Strecke verfahren und der tatsächliche Weg gemessen. Weicht der reale Weg vom Sollwert ab, muss der Schrittwert korrigiert werden.

17.5. Bewegungsalgorithmus

Der Bewegungsalgorithmus legt fest, wie die Maschine von einer Position zur nächsten fährt. Bei einer einzelnen Achse ist dies vergleichsweise einfach, da nur Schrittpulse für diese Achse erzeugt werden müssen. Bei einer zweiachsigen oder mehrachsigen Bewegung müssen die Achsen jedoch zeitlich aufeinander abgestimmt werden, damit eine gerade oder definierte Bahn entsteht.

Für einfache Tests kann eine Bewegung mit festen Schrittzahlen und festen Verzögerungen programmiert werden. Für einen vollständigen CNC-Betrieb ist jedoch eine Steuerung erforderlich, die Bewegungsdaten interpretiert, Achsen koordiniert und Vorschubgeschwindigkeiten umsetzt. Genau darin liegt der Unterschied zwischen einem einfachen Motortest und einer anwendungsbezogenen CNC-Steuerung.

Die Bewegungskoordination wirkt sich direkt auf das Fertigungsergebnis aus. Wenn die Achsen nicht passend zueinander angesteuert werden, entstehen Maßabweichungen oder verformte Konturen. In der Fertigungstechnik werden solche Abweichungen über Maßhaltigkeit und Qualität des Bearbeitungsergebnisses bewertet [FS22].

17.6. Vorschubgeschwindigkeit

Die Vorschubgeschwindigkeit beschreibt, wie schnell der Draht durch das Material bewegt wird. Softwareseitig wird sie über den zeitlichen Abstand der Schrittimpulse beeinflusst. Kürzere Zeitabstände zwischen den Impulsen führen zu einer höheren Geschwindigkeit. Längere Zeitabstände führen zu einer langsameren Bewegung.

Beim Heißdrahtschneiden darf die Vorschubgeschwindigkeit nicht unabhängig von der Heizleistung betrachtet werden. Eine zu hohe Geschwindigkeit kann dazu führen, dass der Draht das Material nicht vollständig trennt. Eine zu geringe Geschwindigkeit kann zu einer zu breiten Schnittfuge führen.

Die Vorschubgeschwindigkeit ist damit eine zentrale Prozessgröße. Allgemein beeinflussen Vorschub, Werkzeugbewegung und weitere Prozessparameter das Bearbeitungsergebnis und die Maßhaltigkeit eines Werkstücks [FS22]. Die Software muss daher eine gleichmäßige und reproduzierbare Vorschubbewegung ermöglichen. Besonders an Richtungswechseln und engen Radien ist dies wichtig, weil der Draht dort länger in einem kleinen Materialbereich wirkt.

17.7. Softwareseitige Fehlerquellen

Typische softwareseitige Fehlerquellen sind ein falscher Nullpunkt, eine falsch eingestellte Achsrichtung, ein fehlerhafter Maßstab oder ein ungeeigneter Vorschub. Auch eine falsche Schritt-Millimeter-Kalibrierung führt direkt zu Maßabweichungen am Werkstück.

Ein weiterer Fehler entsteht, wenn Bewegungsdaten nicht zur realen Mechanik passen. Eine Kontur kann digital korrekt sein, aber trotzdem falsch geschnitten werden, wenn die Maschine eine Achse invertiert, eine andere Skalierung verwendet oder der Startpunkt falsch gesetzt wurde. Deshalb müssen Bewegungsdaten vor dem realen Schnitt mit einfachen Testgeometrien überprüft werden.

Geeignete softwareseitige Tests sind gerade Linien, Rechtecke und einfache Radien. Gerade Linien zeigen, ob die Achsen gleichmäßig fahren. Rechtecke prüfen Maßhaltigkeit und Achsrichtung. Radien zeigen, ob die Bewegung auch bei Richtungsänderungen stabil bleibt. Solche Prüfgeometrien dienen dazu, den Zusammenhang zwischen digitaler Bewegungsbeschreibung, realer Achsbewegung und erzeugtem Fertigungsergebnis zu bewerten [FS22].

Die Ergebnisse dieser Tests bilden die Grundlage für die spätere Bewertung der Anwendung. Sie zeigen, ob die Softwareparameter zur realen Maschine passen und ob die erzeugte Bewegung für reproduzierbare Schnitte geeignet ist.

18. Hardwareseitiges Domänenwissen

18.1. Fachliche Grundlagen des Heißdrahtschneidens

Das Heißdrahtschneiden wird in diesem Kapitel als technischer Prozess betrachtet. Es beschreibt die physikalischen Größen, die den Schnitt beeinflussen: Wärmeeintrag, Drahttemperatur, Drahtspannung, Materialverhalten, Vorschubbewegung und Schnittfuge.

Beim Schneiden wirkt der erhitzte Widerstandsdraht als thermisches Werkzeug. Die Qualität des Schnitts hängt davon ab, ob die eingebrachte Wärme, die Bewegung des Drahts und das Verhalten des Schaumstoffs zueinander passen. Eine gleichmäßige Bahnbewegung allein reicht deshalb nicht aus. Erst durch passende Prozessparameter entsteht eine maßhaltige und reproduzierbare Schnittkante.

Für das Projekt wird der Schneidvorgang deshalb als Fertigungsprozess betrachtet, bei dem Werkzeugbewegung, Prozessparameter und Werkstoffverhalten zusammenwirken. In der Fertigungstechnik werden Maßhaltigkeit, Prozessgrößen und Bearbeitungsqualität als zentrale Bewertungsgrößen technischer Fertigungsprozesse beschrieben [FS22]. Da Schaumstoff ein Kunststoffwerkstoff ist, muss zusätzlich das thermische Verhalten des Materials berücksichtigt werden [Bon25].

18.2. Werkstoff Schaumstoff

Der bearbeitete Werkstoff ist ein Kunststoffschäum. Solche Werkstoffe besitzen eine geringe Dichte und werden häufig für Modelle, Verpackungen, Dämmungen oder leichte Formteile verwendet. Für das Heißdrahtschneiden ist entscheidend, dass der Werkstoff auf Wärme reagiert und dadurch mit einem erhitzten Draht getrennt werden kann.

Kunststoffe unterscheiden sich in Aufbau, thermischem Verhalten und Verarbeitung deutlich von metallischen Werkstoffen. Werkstoffauswahl und Verarbeitungseigenschaften sind daher wichtige Grundlagen der Kunststofftechnik [Bon25]. Bei Schaumstoffen kommt zusätzlich die Zellstruktur hinzu. Sie beeinflusst, wie gleichmäßig das Material beim Schneiden reagiert und wie sauber die Schnittkante wird.

Die optimalen Prozessparameter sind deshalb materialabhängig. Unterschiedliche Plattendicken, Dichten oder Oberflächen können ein unterschiedliches Schneidverhalten verursachen. Aus diesem Grund müssen Probeschnitte durchgeführt werden, bevor endgültige Werkstücke geschnitten werden.

18.3. Thermischer Schneidprozess

Beim thermischen Schneidprozess wird elektrische Energie im Widerstandsdraht in Wärme umgewandelt. Diese Wärme wird an den Schaumstoff abgegeben. Sobald der lokale Wärmeeintrag groß genug ist, wird das Material entlang der Bahn des Drahts getrennt. Dabei entsteht eine Schnittfuge.

Die Schnittfuge ist ein wichtiger Qualitätsparameter. Sie beschreibt den Materialbereich, der durch den thermischen Prozess entfernt oder verformt wird. Eine zu breite Schnittfuge führt zu Maßabweichungen. Eine ungleichmäßige Schnittfuge deutet auf unpassende Prozessparameter oder eine unruhige Drahtbewegung hin.

Der thermische Schneidprozess ist daher nicht nur von der elektrischen Heizleistung abhängig, sondern auch vom Verhalten des Kunststoffs bei Wärmeeinwirkung. Die

Kunststofftechnik beschreibt, dass Kunststoffe je nach Aufbau und Verarbeitungseigenschaften unterschiedlich auf Temperatur reagieren [Bon25]. Für die Bewertung des Schnitts sind dadurch Prozessparameter und Maßhaltigkeit gemeinsam zu betrachten [FS22].

18.4. Widerstandsdraht und Heizleistung

Der Schneiddraht wird als elektrische Last betrieben. Fließt Strom durch den Draht, erwärmt er sich aufgrund seines elektrischen Widerstands. Die Heizleistung kann grundlegend mit $P = U \cdot I$ und für eine ohmsche Last auch mit $P = I^2 \cdot R$ beschrieben werden. Dabei ist P die Leistung, U die Spannung, I der Strom und R der elektrische Widerstand.

Im Projekt darf der Heißdraht nicht direkt über einen Mikrocontroller-Pin betrieben werden. Der Mikrocontroller ist nur für Logiksignale ausgelegt. Der Laststrom des Heißdrahts muss über eine geeignete Leistungsstufe geschaltet werden. Die fachliche Grundlage dafür liegt in der Leistungselektronik, die sich mit dem Schalten, Steuern und Umformen elektrischer Energie beschäftigt [Zac22].

Die Heizleistung muss so gewählt werden, dass der Draht den Schaumstoff sauber trennt. Zu geringe Leistung führt zu einem unvollständigen Schnitt oder zu mechanischer Belastung des Drahts. Zu hohe Leistung kann zu einer breiten Schnittfuge, starker Materialverformung oder Beschädigung des Drahts führen.

18.5. PWM-Ansteuerung des Heißdrahts

Die Heizleistung kann über eine PWM-Ansteuerung beeinflusst werden. PWM steht für Pulsweitenmodulation. Dabei wird eine Last schnell ein- und ausgeschaltet. Durch das Verhältnis zwischen Einschaltzeit und Ausschaltzeit innerhalb einer Schaltperiode verändert sich die mittlere zugeführte Leistung [DDFP24].

Die mittlere Leistung kann über das Tastverhältnis beeinflusst werden. Die eigentliche Last wird dabei durch das MOSFET-Schaltmodul geschaltet. Der Arduino liefert nur das Steuersignal.

Die PWM-Einstellung muss praktisch geprüft werden. Ein bestimmter PWM-Wert ist nicht automatisch für jedes Material geeignet, weil Drahtlänge, Drahtwiderstand, Versorgungsspannung, Materialdicke und Vorschubgeschwindigkeit zusammenwirken. Deshalb gehören Probeschnitte zur Inbetriebnahme des Heißdrahts.

18.6. Drahtspannung und mechanische Führung

Neben der Temperatur ist die mechanische Spannung des Drahts entscheidend. Beim Erwärmen dehnt sich der Draht aus. Wenn diese Längenänderung nicht ausgeglichen wird, kann der Draht durchhängen. Ein durchhängender Draht folgt nicht mehr exakt der programmierten Bahn und verursacht ungenaue Konturen.

Die Drahtspannung muss deshalb mechanisch aufrechterhalten werden. Dies kann beispielsweise über eine Feder oder eine vergleichbare Spannvorrichtung erfolgen. Die Spannvorrichtung soll die thermische Längenänderung ausgleichen und den Draht während des Schnitts möglichst gerade halten.

Die Führung des Drahts muss außerdem so ausgelegt sein, dass keine ungewollten Reibstellen oder Hindernisse entstehen. Der Draht darf während des Schnitts nicht an der Maschine schleifen. Andernfalls können Temperatur, Zugspannung und Schnittlinie beeinflusst werden.

18.7. Achsen, Schrittmotoren und mechanische Belastung

Die Achsen bewegen den Heißdraht entlang der vorgesehenen Kontur. Dabei sollen sie gleichmäßig und wiederholbar verfahren. Die mechanische Belastung ist beim Heißdrahtschneiden geringer als bei spanenden Verfahren, weil der Draht das Material thermisch trennt. Trotzdem müssen Reibung, Massenträgheit und Drahtspannung von den Motoren überwunden werden.

Schrittmotoren bewegen sich in diskreten Winkelschritten und werden deshalb häufig für Positionieraufgaben eingesetzt [Sch17]. Im Projekt ist dies vorteilhaft, weil der Verfahrensweg über Schritimpulse bestimmt werden kann. Gleichzeitig besteht das Risiko von Schrittverlusten, wenn die Last zu hoch oder die Bewegung zu schnell ist.

Mechanische Schwergängigkeit, zu hohe Beschleunigung oder falsch eingestellte Motortreiber können dazu führen, dass die reale Achsposition von der angenommenen Position abweicht. Deshalb müssen Achsen leichtgängig aufgebaut, Treiber korrekt eingestellt und Bewegungen durch Funktionstests geprüft werden.

18.8. Prozessparameter des Schnitts

Die wichtigsten hardwareseitigen Prozessparameter sind Heizleistung, Drahttemperatur, Drahtspannung, Drahtdurchmesser, Materialdicke und mechanische Achsbewegung. Diese Parameter beeinflussen sich gegenseitig. Wird die Vorschubgeschwindigkeit erhöht, bleibt dem Draht weniger Zeit, Wärme in das Material einzubringen. Wird die Heizleistung erhöht, kann die Schnittfuge breiter werden.

Besonders kritisch ist die Abstimmung zwischen Heizleistung und Vorschub. Ein zu kalter Draht oder ein zu schneller Vorschub führt zu unvollständigem Schneiden. Ein zu heißer Draht oder ein zu langsamer Vorschub kann Material übermäßig schmelzen. Damit stehen Prozessparameter, Werkzeugbewegung und Bearbeitungsqualität in einem direkten Zusammenhang [FS22].

Auch die Materialdicke wirkt auf den Prozess. Dickere Schaumstoffplatten benötigen eine längere thermische Einwirkung als dünnere Platten. Deshalb können für unterschiedliche Werkstückdicken unterschiedliche Einstellungen notwendig sein.

18.9. Fehlerbilder und Ursachen

Typische hardwareseitige Fehlerbilder lassen sich an der Schnittkante, an der Schnittfuge und am Maschinenverhalten erkennen. Ein unvollständiger Schnitt deutet auf zu geringe Heizleistung, zu hohen Vorschub oder schlechten Kontakt zwischen Draht und Material hin. Eine zu breite Schnittfuge deutet auf zu hohe Heizleistung oder zu langsamen Vorschub hin.

Eine schräge oder gekrümmte Schnittlinie kann durch einen durchhängenden Draht, eine unzureichende Drahtspannung oder eine fehlerhafte mechanische Führung entstehen. Ruckartige Bewegungen oder Schrittverluste können auf mechanische Schwergängigkeit, zu hohe Geschwindigkeit oder eine ungünstige Treibereinstellung hinweisen.

Auch elektrische Fehler sind möglich. Dazu gehören lose Verbindungen, ungeeignete Leitungsquerschnitte, unzureichende Masseverbindungen oder eine Überlastung des MOSFET-Moduls. Da der Heißdraht eine stromführende und heiße Last ist, müssen diese Fehler besonders sorgfältig vermieden werden.

18.10. Bewertung der Schnittqualität

Die Schnittqualität wird anhand mehrerer Merkmale bewertet. Dazu gehören Maßhaltigkeit, Gleichmäßigkeit der Schnittkante, Breite der Schnittfuge und Wiederholbarkeit.

Ein guter Schnitt liegt vor, wenn die Kontur vollständig getrennt wird, die Kante gleichmäßig bleibt und die Abmessungen zur Sollgeometrie passen.

Zur Bewertung eignen sich einfache Testgeometrien. Gerade Linien zeigen, ob Draht und Achsen gleichmäßig arbeiten. Rechtecke prüfen Maßhaltigkeit und Rechtwinkligkeit. Radien zeigen, ob die Maschine auch bei Richtungsänderungen gleichmäßig schneidet. Die Bewertung der Schnittqualität orientiert sich damit an typischen Kriterien der Fertigungstechnik. Maßhaltigkeit, Wiederholbarkeit und die Wirkung der Prozessparameter auf das Bearbeitungsergebnis sind zentrale Merkmale zur Beurteilung eines Fertigungsprozesses [FS22].

Zusätzlich zur Schnittkante müssen die Baugruppen beobachtet werden. Motoren, Motortreiber, MOSFET-Modul, Leitungen und Spannungsversorgung dürfen sich im Betrieb nicht unzulässig erwärmen. Die Ergebnisse der Probeschnitte werden dokumentiert und dienen zur Festlegung geeigneter Betriebsparameter.

Teil IV.

Nutzung

19. Von der Idee zum Werkstück

In diesem Kapitel wird der gesamte Ablauf des Heißdraht Schneiders "EL-MAR" beschrieben – vom ersten Entwurf einer Kontur bis zum fertig geschnittenen Werkstück aus Schaumstoff. Die vorangegangenen Kapitel haben die einzelnen Komponenten des Systems im Detail behandelt: den Arduino Uno als Steuerzentrale mit der GRBL-Firmware, das CNC Shield V3 mit den Schrittmotortreibern, die Endschalter zur Positionsbestimmung, den Druckknopf zur Bedienung, das MOSFET-PWM-Schaltmodul zur Ansteuerung des Heißdrahtes sowie den Universal G-Code Sender (UGS) als Bediensoftware auf dem angeschlossenen Computer. Dieses Kapitel führt diese Bausteine zu einem durchgängigen Arbeitsablauf zusammen.

19.1. Idee und Entwurf der Kontur

Am Anfang jedes Werkstücks steht die Idee, welche Form aus dem Schaumstoffblock herausgeschnitten werden soll. Diese Form wird zunächst als zweidimensionale Kontur entworfen, beispielsweise als Skizze oder als digitale Zeichnung. Da der Heißdraht beim Schneiden entlang einer durchgehenden Bahn geführt wird, muss die Kontur als geschlossener oder durchgehend befahrbarer Linienzug vorliegen, ohne Sprungstellen, die der Draht nicht abfahren kann.

Bei der Erstellung der Kontur ist außerdem die Größe des verwendeten Schaumstoffblocks sowie der Verfahrbereich der Achsen zu berücksichtigen. Die Kontur darf den durch die Endschalter begrenzten Arbeitsbereich der Maschine nicht überschreiten, da es sonst beim späteren Schneidvorgang zu einer Kollision mit den mechanischen Endanschlägen kommen kann.

19.2. Erzeugung der Steuerbefehle (G-Code)

Aus der entworfenen Kontur werden anschließend die Steuerbefehle für die Maschine abgeleitet. Diese liegen im G-Code-Format vor, einer standardisierten Befehlssprache für die Steuerung von CNC-Maschinen. Der G-Code beschreibt die einzelnen Bewegungen der Achsen in Form von Koordinaten und Verfahrgeschwindigkeiten, die der Heißdraht beim Schneidvorgang nacheinander abfährt. Der G-Code kann dabei entweder manuell geschrieben oder mithilfe einer CAM-Software aus einem CAD-Modell erzeugt und als Datei (z. B. .nc oder .gcode) exportiert werden.

Die Umsetzung der G-Code-Befehle in konkrete Schrittmotorsignale übernimmt die auf dem Arduino Uno installierte GRBL-Firmware. GRBL interpretiert die im G-Code enthaltenen Bewegungsanweisungen und erzeugt daraus die STEP- und DIR-Signale, die über das CNC Shield V3 an die Motortreiber der einzelnen Achsen weitergegeben werden. Auf diese Weise wird aus einer abstrakten Bewegungsbeschreibung eine reale, koordinierte Bewegung der Achsen.

Die Übertragung des G-Codes an den Arduino Uno erfolgt über den Universal G-Code Sender (UGS). UGS verbindet sich seriell über USB mit dem Controller, zeigt den geladenen Werkzeugpfad als 2D/3D-Vorschau im Visualizer-Fenster an und übernimmt im sogenannten Batch-Mode die geordnete, gepufferte Übertragung der einzelnen G-Code-Zeilen an GRBL.

19.3. Einrichten und Referenzieren der Maschine

Bevor ein Werkstück geschnitten werden kann, muss die Maschine in einen definierten Ausgangszustand gebracht werden. Dazu fahren die Achsen nacheinander in Richtung der Endschalter, bis diese ausgelöst werden. Auf diese Weise erkennt die Steuerung die jeweilige Nullposition der Achsen, unabhängig davon, an welcher Stelle sich der Schneidrahmen zuvor befunden hat. Diese sogenannte Referenzfahrt (Homing) wird durch den Befehl `$H` ausgelöst und stellt sicher, dass die im G-Code hinterlegten Koordinaten korrekt auf die tatsächliche Position der Maschine übertragen werden. Der aktuelle Zustand der Maschine – Status (Idle, Run, Hold, Alarm), Achspositionen sowie Vorschub – wird während des gesamten Vorgangs im Digital Read-Out (DRO) von UGS in Echtzeit angezeigt. Vor dem eigentlichen Schneidvorgang wird zusätzlich der Werkstück-Nullpunkt an der Maschine festgelegt und in UGS über die Funktion Zero All zurückgesetzt, sodass die Koordinaten des G-Codes exakt auf die Position des Schaumstoffblocks abgestimmt sind.

19.4. Der Schneidvorgang

Sobald die Maschine referenziert, der Werkstück-Nullpunkt gesetzt und der zugehörige G-Code in UGS geladen wurde, kann der eigentliche Schneidvorgang über die Send-File-Funktion gestartet werden. Die Achsen bewegen den Heißdraht entsprechend der programmierten Bahn durch den Schaumstoffblock. Damit der Draht das Material sauber durchtrennt, muss er auf eine geeignete Temperatur aufgeheizt sein, während die Achsen ihn mit einer gleichmäßigen, auf das Material abgestimmten Geschwindigkeit führen.

Die Temperatur des Heißdrahtes wird über das MOSFET-PWM-Schaltmodul gesteuert. Über ein pulswidenmoduliertes Signal wird die dem Draht zugeführte elektrische Leistung und damit seine Temperatur reguliert. Eine zu niedrige Temperatur führt zu einem unsauberen Schnitt oder zu erhöhtem Widerstand beim Vorschub, eine zu hohe Temperatur kann das Material unnötig stark schmelzen und die Schnittkante beschädigen. Die Abstimmung von Drahttemperatur und Vorschubgeschwindigkeit der Achsen ist daher entscheidend für die Qualität des Schnittergebnisses.

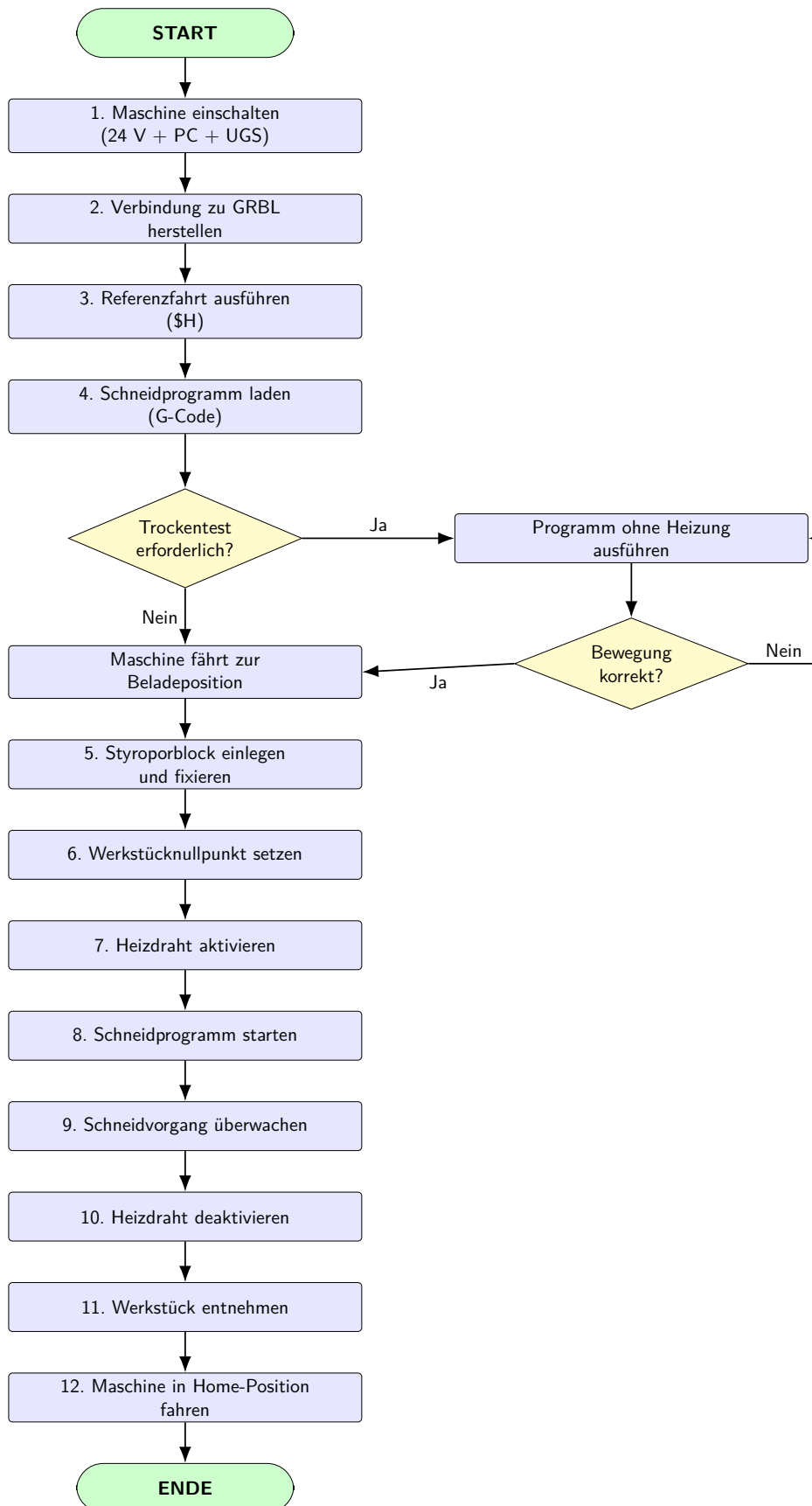
Während des Schneidvorgangs kann der Fortschritt im Statusbalken sowie im Konsolenfenster von UGS verfolgt werden. Bei einer Fehlfunktion, etwa dem unbeabsichtigten Auslösen eines Endschalters, meldet GRBL einen entsprechenden Alarmzustand (z. B. ALARM:1 bei einem Hard-Limit), der den Vorgang automatisch anhält. Der Druckknopf dient als einfaches, lokales Bedienelement, über das ein laufender Vorgang bei Bedarf zusätzlich manuell unterbrochen werden kann.

19.5. Fertigstellung des Werkstücks

Nach Abschluss der programmierten Schnittbahn fahren die Achsen in der Regel in eine definierte Ruheposition zurück. Das fertig geschnittene Werkstück kann anschließend aus dem verbleibenden Schaumstoffblock entnommen werden. Je nach Anwendung kann eine Nachbearbeitung sinnvoll sein, etwa das Entfernen kleiner Schmelzgrate an der Schnittkante.

Mit der Entnahme des Werkstücks schließt sich der Kreis vom anfänglichen Entwurf der Kontur bis zum fertigen, physischen Bauteil. Der beschriebene Ablauf verdeutlicht, wie die in den vorangegangenen Kapiteln einzeln vorgestellten Komponenten – Arduino Uno mit GRBL, CNC Shield V3, Endschalter, Druckknopf, MOSFET-PWM-Schaltmodul und der Bediensoftware UGS – im Zusammenspiel den vollständigen Funktionsumfang des Heißdraht Schneiders ergeben.

20. Programmablaufplan



Teil V.

Erfahrungen

21. Erfahrungen bei Aufbau, Verkabelung, Inbetriebnahme und Steuerung der CNC-Heißdrahtschneidanlage

21.1. Hinweis zur Erstellung dieses Erfahrungsberichts

Der vorliegende Erfahrungsbericht basiert auf den tatsächlich während des Projekts entstandenen Notizen, Diskussionen, Messergebnissen und dokumentierten Problemen. Dabei wurde die textliche Ausformulierung, Strukturierung sowie Dokumentation von Arbeitsschritten, Problemen und Lösungsansätzen durch KI unterstützt. Sämtliche beschriebenen Erfahrungen, Entscheidungen, Schwierigkeiten und technischen Erkenntnisse stammen jedoch aus der praktischen Projektarbeit und wurden von den Teammitgliedern überprüft, ergänzt und freigegeben.

Die KI wurde somit als Werkzeug zur Aufbereitung und Zusammenfassung der Projekterfahrungen verwendet, nicht zur Erzeugung der zugrunde liegenden Inhalte.

21.2. Erfahrungen im Projektverlauf

Im Verlauf des Projekts wurde deutlich, dass der Aufbau einer CNC-Heißdrahtschneidanlage nicht nur aus der mechanischen Konstruktion besteht, sondern in hohem Maße von einer zuverlässigen elektrischen Verdrahtung, einer systematischen Inbetriebnahme und einem guten Verständnis der Steuerungssoftware abhängt. Besonders im Bereich der Elektronik, der GRBL-Konfiguration und der praktischen Fehlersuche war der Lernaufwand deutlich größer als zunächst erwartet.

Zu Beginn lag der Fokus vor allem darauf, die grundlegenden Komponenten miteinander in Betrieb zu nehmen. Verwendet wurden ein Arduino Uno R3, ein CNC Shield V3, Schrittmotortreiber, NEMA17-Schrittmotoren, ein 24-V-Netzteil, Endschalter, ein PWM-MOSFET-Modul sowie später der Heizdraht. Obwohl diese Komponenten auf den ersten Blick gut dokumentiert und weit verbreitet wirken, zeigte sich in der Praxis, dass gerade bei günstigen CNC-Shield-Varianten, Klon-Platinen und unterschiedlichen Treibermodulen viele Details beachtet werden müssen. Die Beschriftung auf den Platinen, die tatsächlich verwendeten Pins, die GRBL-Version und die Pinbelegung des CNC Shields stimmten nicht immer intuitiv mit den Erwartungen überein.

Ein wesentlicher erster Schritt war die Inbetriebnahme der Schrittmotoren. Anfangs lag trotz angeschlossener 24-V-Versorgung kein Haltemoment an den Motoren an. Der entscheidende Punkt war dabei die Enable-Brücke zwischen EN und GND auf dem CNC Shield. Erst nachdem diese Brücke gesetzt wurde, wurden die Treiber aktiv und die Motoren erhielten Haltemoment. Diese Erfahrung zeigte früh, dass selbst kleine Jumper oder scheinbar nebensächliche Steckbrücken über die komplette Funktionsfähigkeit des Systems entscheiden können.

Die ersten Motortests wurden zunächst mit einfachen Arduino-Testsketchen durchgeführt. Dabei konnten die STEP- und DIR-Pins der X-, Y- und Z-Achse erfolgreich angesteuert werden. Später wurde GRBL 1.1h installiert und mit Universal Gcode Sender verbunden. Auch hier gab es zunächst Schwierigkeiten, da zuerst eine ungeeig-

nete bzw. veraltete GRBL-Bibliothek verwendet wurde. Erst nach der Installation der offiziellen GRBL-Version und dem Hochladen des Beispiels grblUpload meldete sich der Arduino korrekt mit Grbl 1.1h ['\$' for help]. Dieser Schritt war entscheidend, weil dadurch die Grundlage für eine standardisierte CNC-Steuerung geschaffen wurde.

Während der Inbetriebnahme zeigte sich außerdem eine grundlegende Einschränkung der gewählten Steuerungsplattform, die in der frühen Projektphase nicht ausreichend berücksichtigt worden war. Ursprünglich war vorgesehen, die Maschine mit vier vollständig unabhängigen Achsen zu betreiben. Diese Annahme beruhte unter anderem auf Vorbildern ähnlicher Heißdrahtschneidanlagen, die häufig einen Arduino Mega 2560 in Verbindung mit einem RAMPS-Board verwenden. In solchen Konfigurationen kommen häufig spezielle GRBL-Varianten oder andere Firmwarelösungen zum Einsatz, welche tatsächlich vier unabhängig interpolierbare Achsen unterstützen.

Im Projekt wurde jedoch ein Arduino Uno R3 mit CNC Shield V3 eingesetzt. Zu Beginn wurde davon ausgegangen, dass die vier Treibersteckplätze des Shields auch vier unabhängig steuerbare Achsen ermöglichen würden. Erst während der intensiveren Beschäftigung mit GRBL und der tatsächlichen Pinbelegung wurde deutlich, dass das auf dem Arduino Uno eingesetzte Standard-GRBL 1.1h lediglich drei Bewegungsachsen unterstützt. Der vierte Treibersteckplatz (A-Achse) dient standardmäßig lediglich als Klon einer vorhandenen Achse und kann nicht ohne Weiteres als eigenständige vierte CNC-Achse betrieben werden.

Rückblickend handelte es sich hierbei weniger um ein technisches Problem als vielmehr um eine Fehleinschätzung in der Planungsphase. Innerhalb des Teams wurde die Funktionalität verschiedener Hardwareplattformen teilweise gleichgesetzt, obwohl sich Arduino Uno mit CNC Shield V3 und Arduino Mega 2560 mit RAMPS sowohl hinsichtlich der verfügbaren Ein- und Ausgänge als auch hinsichtlich der eingesetzten Firmware erheblich unterscheiden. Die Unterschiede wurden erst im Verlauf der praktischen Inbetriebnahme vollständig verstanden.

Für das konkrete Projekt stellte sich diese Einschränkung letztlich als akzeptabel heraus, da die rechte vertikale Achse als Klon der linken vertikalen Achse betrieben werden konnte. Dennoch war diese Erkenntnis ein wichtiger Lernschritt. Sie verdeutlichte, wie entscheidend eine frühzeitige Überprüfung der tatsächlichen Fähigkeiten einer Steuerungsplattform ist und dass äußerlich ähnliche Systeme nicht zwangsläufig dieselben Funktionen bereitstellen. Daraus ergab sich die finale Achslogik: X und Z bilden die beiden horizontalen Achsen der linken und rechten Seite, Y und A bilden die beiden vertikalen Achsen, wobei A als Klon von Y arbeitet.

Trotz der beschriebenen Einschränkungen der eingesetzten Steuerungsplattform wurde die Maschine von Beginn an mit vier mechanisch unabhängigen Achsen konstruiert. Jede Achse verfügt über einen eigenen Antrieb und kann grundsätzlich separat verfahren werden. Die Beschränkung auf drei unabhängig interpolierbare Achsen ergibt sich daher nicht aus der mechanischen Konstruktion, sondern ausschließlich aus der Kombination aus Arduino Uno R3, CNC Shield V3 und der darauf eingesetzten Standardversion von GRBL.

21.3. Erfahrungen im Projektverlauf

Im Verlauf des Projekts wurde bewusst entschieden, die vorhandene Steuerungsplattform beizubehalten, da sie für die grundlegende Funktion der Heißdrahtschneidanlage ausreichend war und eine zuverlässige Inbetriebnahme ermöglichte. Gleichzeitig wurde bei der Konstruktion darauf geachtet, zukünftige Erweiterungen nicht auszuschließen. Die Maschine kann daher als hardwareseitig für eine spätere Vierachssteuerung vorbereitet betrachtet werden. Durch den Austausch der Steuerung gegen leistungsfähigere Plattformen, beispielsweise auf Basis eines Arduino Mega 2560 mit RAMPS-Board oder anderer moderner CNC-Steuerungen, wäre grundsätzlich eine Ansteuerung aller vier Achsen als vollständig unabhängige Bewegungsachsen möglich. Die hierfür erforder-

derlichen Anpassungen betreffen überwiegend die Elektronik und Firmware, während die mechanische Konstruktion weitgehend unverändert weiterverwendet werden könnte. Rückblickend erwies sich dieser Umstand sogar als Vorteil. Obwohl die ursprünglich angestrebte Vierachssteuerung mit der gewählten Hardware nicht realisiert werden konnte, entstand eine mechanische Plattform, die über die tatsächlich genutzte Funktionalität hinaus erweiterbar bleibt. Dadurch bietet die Maschine nicht nur die Möglichkeit zum Heißdrahtschneiden in der aktuell verwendeten Konfiguration, sondern stellt gleichzeitig eine geeignete Basis für zukünftige Entwicklungen und weiterführende Untersuchungen im Bereich mehrachsiger CNC-Systeme dar.

Ein großer Teil der Arbeit bestand aus Fehlersuche. Besonders prägend war der Ausfall der Motorversorgung auf einem CNC Shield. Obwohl am Eingang des Shields stabile 24 V gemessen wurden, hatten die Motoren plötzlich kein Haltemoment mehr, die Treiber blieben kalt und GRBL nahm Befehle zwar an, bewegte aber nur intern die Positionswerte. Durch systematisches Messen per Multimeter wurde festgestellt, dass am VMOT-Pin der Treiber keine 24 V mehr anlagen. Schließlich wurde ein kleines glasartiges Sicherungsbauteil auf dem Shield als unterbrochen identifiziert. Eine provisorische Überbrückung führte sofort wieder zu Haltemoment, wodurch die Fehlerursache eindeutig auf die unterbrochene VMOT-Versorgung eingegrenzt werden konnte. Diese Fehlersuche war sehr lehrreich, weil sie zeigte, dass Software, Arduino und GRBL völlig korrekt arbeiten können, während die Leistungsversorgung der Motoren trotzdem unterbrochen ist.

Auch ein späterer Kurzschluss machte deutlich, wie empfindlich solche Aufbauten sein können. Besonders kritisch sind das Umstecken von Motoren oder Treibern bei eingeschalteter 24-V-Versorgung, lose Brücken im Leistungsstrompfad und ungesicherte Messungen mit Messspitzen. Daraus ergab sich als feste Arbeitsregel: Vor jedem Umstecken oder Ändern an Motoren, Treibern, Endschaltern, Jumpen oder Leistungsverdrahtung müssen 24 V ausgeschaltet, USB getrennt und einige Sekunden gewartet werden. Diese Regel wurde im weiteren Verlauf konsequent beachtet.

Ein weiterer wichtiger Punkt war die korrekte Zuordnung der Motorleitungen. Bei Schrittmotoren ist nicht entscheidend, welche Leitung als „Plus“ oder „Minus“ bezeichnet wird, sondern dass die beiden Spulenpaare korrekt zusammen auf dem jeweiligen Treiberausgang liegen. Mit dem Multimeter wurden die Leitungen auf Durchgang geprüft, um die beiden Spulenpaare zu identifizieren. Ein falsches Mischen der Spulenleitungen kann dazu führen, dass ein Motor nur ruckelt, gar nicht läuft oder im schlimmsten Fall Treiber belastet werden. Die Drehrichtung kann dagegen einfach durch Drehen eines Motorsteckers bzw. Vertauschen eines Spulenpaares oder alternativ über GRBL-Parameter angepasst werden.

Besonders zeitintensiv war die Kalibrierung der Achsen. Zunächst war unklar, wie Microstepping, mechanische Übersetzung, Riemensteigung, Spindelsteigung und GRBL-Parameter zusammenspielen. Entscheidend wurde schließlich die Erkenntnis, dass GRBL die Microstepping-Jumper auf dem Shield nicht automatisch erkennt. Die Hardware-Jumper verändern nur das Verhalten des Treibers; softwareseitig muss dies über die Schritte pro Millimeter angepasst werden. Die relevanten Werte sind \$100 für X, \$101 für Y und \$102 für Z. Durch praktische Messfahrten wurde die Maschine schließlich auf sinnvolle Werte gebracht. Für die aktuelle Konfiguration mit 1/2-Mikroschrittauflösung ergaben sich als Kalibrierwerte $X = 10$ Schritte/mm, $Y = 50$ Schritte/mm und $Z = 10$ Schritte/mm. A übernimmt als Y-Klon entsprechend den Y-Wert.

Bei der Kalibrierung kam es zwischenzeitlich zu großer Verwirrung, weil nach Änderungen der Steps/mm-Werte teilweise die jeweils erste Fahrbewegung nicht dem erwarteten Verhalten entsprach. Dadurch wurden anfangs Messwerte aufgenommen, die scheinbar nichtlinear wirkten und zu widersprüchlichen Schlussfolgerungen führten. Erst durch wiederholtes, systematisches Messen mit definierten Jog-Bewegungen wurde klar, welche Werte tatsächlich reproduzierbar waren. Diese Erfahrung war wichtig, weil

sie zeigt, dass bei CNC-Inbetriebnahmen einzelne Messungen oft nicht ausreichen. Man muss Bewegungen mehrfach testen, reproduzierbar messen und nach jeder Änderung prüfen, ob die Maschine wirklich im erwarteten Zustand ist.

Auch die mechanische Laufruhe war ein großes Thema. Im Vollschrittbetrieb und bei niedrigen Vorschüben traten deutliche Schwingungen und Resonanzen auf. Teilweise wurde der gesamte Rahmen angeregt. Zunächst lag der Verdacht nahe, dass die Motoren oder Treiber falsch eingestellt waren. Später zeigte sich jedoch, dass die Schwingungen stark von der Drehzahl, der Beschleunigung, dem Microstepping und der mechanischen Reibung (Stick-Slip-Effekte) abhingen. Besonders auffällig war, dass sehr langsame Bewegungen teilweise deutlich schlechter klangen als höhere Vorschübe. Bei höheren Feed Rates liefen die Motoren wesentlich ruhiger, während bei bestimmten Zwischenbereichen wieder Resonanzen auftraten. Dies entspricht dem typischen Verhalten von Schrittmotoren, die in bestimmten Drehzahlbereichen mechanische Resonanzen anregen können.

Eine große Verbesserung brachte das Schmieren der Spindeln. Diese waren zuvor vollständig entfettet und liefen trocken. Nach dem Auftragen von Langzeitfett liefen die Spindeln deutlich ruhiger. Wahrscheinlich wirkte das Fett nicht nur reibungsmindernd, sondern auch dämpfend zwischen Spindel und Mutter. Dadurch wurde die Geräuschentwicklung deutlich reduziert. Diese Erfahrung zeigte, dass elektrische oder softwareseitige Probleme nicht isoliert betrachtet werden dürfen. Mechanik, Elektronik und Steuerung beeinflussen sich gegenseitig.

Im Bereich der GRBL-Einstellungen wurden zahlreiche Parameter angepasst und getestet. Besonders wichtig waren \$1=255, damit die Motoren dauerhaft bestromt bleiben und Haltemoment behalten, die Steps/mm-Werte \$100, \$101, \$102, die Maximalgeschwindigkeiten \$110, \$111, \$112 und die Beschleunigungen \$120, \$121, \$122. Die Beschleunigung wurde von einem sehr konservativen Wert auf 30 mm/s² erhöht, um schneller aus stark ratternden Niedrigdrehzahlbereichen herauszukommen. Die maximale Geschwindigkeit wurde auf 2000 mm/min gesetzt. Diese Werte erwiesen sich für die Maschine als sinnvoller Startpunkt.

Die Endschalter und das Homing waren ebenfalls nicht trivial. Die Endschaltermodule wurden über ihre Kontakte geprüft. Je nach Anschluss und Logik zeigte GRBL über Pn:X, Pn:Y oder Pn:XYZ an, welche Endschalter als aktiv erkannt wurden. Dabei wurde deutlich, dass \$5 für die Invertierung der Limit-Pins entscheidend ist. Außerdem zeigte sich bei CNC Shield V3 und GRBL 1.1h eine mögliche Besonderheit im Zusammenhang mit Z-Limit und Spindle/PWM-Pins. In Foren wurde beschrieben, dass bei manchen Shield-Versionen die Zuordnung zwischen Z-Endschalter und Spindle-Pin durch Änderungen in GRBL 1.1 problematisch sein kann. Diese Beobachtung passte teilweise zu den eigenen Problemen mit dem Z-Endschalter. Deshalb wurde gelernt, nicht blind auf die Beschriftung eines Shields zu vertrauen, sondern im Zweifel mit Multimeter und GRBL-Statusmeldungen zu prüfen, welcher Pin tatsächlich welche Funktion hat.

Die Steuerung über Universal Gcode Sender war insgesamt sehr hilfreich, brachte aber ebenfalls Lernaufwand mit sich. UGS wurde genutzt, um GRBL zu verbinden, Einstellungen auszulesen, Achsen zu joggen und G-Code-Programme auszuführen. Dabei wurde klar, dass UGS und die Arduino IDE nicht gleichzeitig denselben COM-Port nutzen können. Wenn keine Verbindung möglich war, lag dies häufig an einem blockierten Port, einem offenen seriellen Monitor, einem falschen COM-Port oder einem Arduino, auf dem GRBL nicht korrekt lief. Bei Upload-Problemen half es, das CNC Shield vom Arduino zu entfernen und den Arduino separat zu testen. Dadurch konnte zwischen Arduino-/Firmware-Problemen und Shield-/Hardwareproblemen unterschieden werden.

Das Arbeiten mit G-Code wurde Schritt für Schritt aufgebaut. Zunächst wurden nur einzelne Bewegungen ausgeführt, später Quadrate, Diagonalen und komplexere Testformen. Dabei wurde gelernt, dass jede G-Code-Zeile einen eigenen Bewegungsauftrag

darstellt. Wenn mehrere Achsen in derselben Zeile stehen, interpoliert GRBL diese Bewegung so, dass die Achsen gleichzeitig am Ziel ankommen. Stehen Bewegungen in getrennten Zeilen, werden sie nacheinander ausgeführt. Für den Heißdrahtschneider ist dies entscheidend, weil X und Z bewusst gleiche oder unterschiedliche Wege fahren können. Dadurch lässt sich der Draht horizontal schräg stellen, was später für komplexere Schnitte und unterschiedliche Profile relevant ist.

Auch die Bedeutung grundlegender G-Code-Befehle wurde im Verlauf des Projekts praktisch erarbeitet. Der Befehl G21 stellt die Maßeinheit auf Millimeter ein, G90 aktiviert absolute Koordinaten und G94 legt fest, dass Vorschübe in Millimetern pro Minute angegeben werden. Für relative Bewegungen wurde G91 verwendet. Der Befehl M3 S... aktiviert den Heißdraht mit einer vorgegebenen Leistung, während M5 den Heißdraht wieder abschaltet. Mit G4 P... kann eine definierte Wartezeit eingefügt werden, beispielsweise um dem Heizdraht Zeit zum Aufheizen zu geben. Außerdem zeigte sich, dass der Vorschubwert F möglichst in jedem Programm explizit gesetzt werden sollte, damit keine unbeabsichtigt übernommenen Vorschubwerte aus vorherigen Programmen verwendet werden.

Die Integration des Heißdrahtsystems stellte einen weiteren wichtigen Schritt dar. Der Heizdraht wird nicht direkt vom Arduino angesteuert, sondern über ein PWM-fähiges MOSFET-Modul geschaltet. Dabei war zunächst unklar, welcher Ausgang des CNC Shields tatsächlich das von GRBL erzeugte PWM-Signal bereitstellt. Erst nach intensiver Recherche und mehreren Tests konnte die korrekte Zuordnung verifiziert werden. Wichtig ist dabei, dass das PWM-Signal des Arduino mit dem Signaleingang des MOSFET-Moduls verbunden wird und dass Arduino und MOSFET eine gemeinsame Masseverbindung (GND) besitzen.

Für die Leistungssteuerung des Heizdrahts wurden mehrere GRBL-Parameter verwendet. Der Parameter \$30=1000 definiert den maximal möglichen Leistungswert. Der Parameter \$31=0 definiert den minimalen Leistungswert. Dadurch ergibt sich ein Leistungsbereich von 0 bis 1000. Wird beispielsweise der Befehl M3 S1000 ausgeführt, entspricht dies 100 % der eingestellten Ausgangsleistung. M3 S500 entspricht 50 % und M3 S0 beziehungsweise M5 entspricht 0 % Leistung. Auf diese Weise kann die Heizleistung des Drahtes über den S-Wert im G-Code feinstufig angepasst werden.

Zusätzlich wurde der Parameter \$32 betrachtet. Mit \$32=1 wird der sogenannte Laser Mode von GRBL aktiviert. Obwohl diese Funktion ursprünglich für Laseranwendungen entwickelt wurde, eignet sie sich auch für Heißdrahtschneider, da der Heizdraht ebenfalls eine kontinuierliche Energiequelle darstellt. In diesem Modus werden Bewegungen flüssiger geplant und die Leistungsregelung während der Bewegung besser an die tatsächliche Verfahrensgeschwindigkeit angepasst. Dadurch können gleichmäßigere Schneidbedingungen erreicht werden als im normalen Betriebsmodus (\$32=0).

Beim Heizdraht zeigte sich, dass schon kleine PWM-Werte zu hoher Temperatur führen können. Bei einer Drahtlänge von etwa 600 mm und 0,4 mm Durchmesser, Werkstoff NiCr, kann bei 24 V sehr viel Leistung entstehen. Deshalb wurden Werte im Bereich von etwa S50 bis S150 als sinnvoller Startbereich betrachtet. Wenn der Draht bereits bei sehr kleinen S-Werten extrem heiß wird, muss geprüft werden, ob das MOSFET-Modul tatsächlich per PWM regelt oder dauerhaft durchschaltet. Der Befehl M5 muss den Draht zuverlässig abschalten.

Ein OLED-Display mit SSD1306-Controller wurde ebenfalls betrachtet. Es stellte sich jedoch heraus, dass Standard-GRBL auf dem Arduino Uno keine direkte Displayunterstützung bietet. Der Speicher des Uno ist bereits durch GRBL stark ausgelastet. Eine Integration eines Displays in GRBL wäre daher aufwendig und für den Projektabschluss nicht notwendig. Sinnvoller wäre ein separater Mikrocontroller für Anzeigezwecke oder später eine leistungsfähigere Steuerung wie grblHAL oder FluidNC. Für die aktuelle Maschine bleibt UGS am Laptop die praktikablere Lösung.

Insgesamt zeigte das Projekt sehr deutlich, dass die Inbetriebnahme einer selbstgebauten CNC-Maschine einen erheblichen praktischen Lernanteil besitzt. Viele Probleme

lassen sich nicht allein durch Datenblätter oder Tutorials lösen, sondern müssen durch Messen, Testen, Ausschließen und systematisches Vorgehen eingegrenzt werden. Besonders wichtig war dabei, immer nur eine Änderung auf einmal vorzunehmen und danach das Verhalten zu prüfen. Sobald mehrere Parameter gleichzeitig geändert wurden, wurde die Fehlersuche deutlich schwieriger.

Rückblickend waren die größten Herausforderungen nicht einzelne Bauteile, sondern das Zusammenspiel aller Teilsysteme: Mechanik, Motoren, Treiber, Shield, Arduino, GRBL, UGS, Endschalter, PWM-MOSFET und Heizdraht. Erst als verstanden wurde, welche Aufgabe jedes Teilsystem übernimmt und wie es mit den anderen zusammenhängt, konnte die Maschine zuverlässig betrieben werden.

Ein besonders prägendes Erlebnis war die abschließende Vorführung der Maschine. Nach einer intensiven Arbeitsphase in den letzten Stunden vor der Präsentation konnte die Anlage schließlich vollständig montiert, verkabelt und in Betrieb genommen werden. Sowohl die Achsbewegungen als auch die Heizdrahtsteuerung funktionierten einwandfrei, und es konnten einfache wie auch komplexere Formen aus Styropor mittels G-Code geschnitten werden. Kurz vor der Vorführung im Rahmen der Abgabe der Hardware traten jedoch erneut Probleme auf. Obwohl das System wenige Stunden zuvor noch funktionsfähig gewesen war, kam es zu weiteren Hardwareausfällen und Kontaktproblemen. Unter anderem musste kurzfristig ein Arduino ersetzt werden, zusätzlich traten Fehler in der Verkabelung sowie am PWM-Modul für den Heizdraht auf.

Dadurch konnte die Maschine während der offiziellen Vorführung nicht in dem Zustand präsentiert werden, in dem sie zuvor bereits erfolgreich betrieben worden war. Dies war zunächst enttäuschend, verdeutlichte jedoch gleichzeitig eine wichtige Erkenntnis des Projekts: Gerade bei komplexen Prototypen entscheidet nicht nur die grundsätzliche Funktion, sondern auch die langfristige Zuverlässigkeit und Robustheit der Gesamtkonstruktion. Die dabei gewonnenen Erfahrungen hinsichtlich Fehlersuche, Verkabelung, Dokumentation und systematischer Inbetriebnahme waren letztlich ebenso wertvoll wie die erfolgreiche Realisierung der Maschine selbst.

Das Projekt erforderte daher nicht nur mechanisches Konstruieren und Montieren, sondern auch Kenntnisse in Elektrotechnik, Fehlersuche, Firmware, CNC-Steuerung, G-Code und praktischer Inbetriebnahme. Besonders wertvoll war, dass viele Fehler real auftraten und gelöst werden mussten: fehlendes Haltemoment, falsche Treiberfreigabe, defekte bzw. unterbrochene Motorversorgung, falsch interpretierte Endschalterzustände, unklare Shield-Pinbelegung, Upload-Probleme, COM-Port-Probleme, Resonanzen, falsch kalibrierte Achsen, heiße Treiber und PWM-Probleme am Heizdraht.

Gerade diese Schwierigkeiten haben das Verständnis für CNC-Systeme stark vertieft. Am Ende war die Maschine nicht nur mechanisch aufgebaut, sondern auch steuerungstechnisch verstanden. Die Achsen konnten kalibriert, Bewegungen interpoliert, G-Code-Programme ausgeführt, Homing vorbereitet und der Heizdraht über PWM angesteuert werden. Damit wurde aus einzelnen Komponenten eine funktionierende CNC-Heizdrahtschneidanlage.

Für zukünftige Projekte würde aus diesen Erfahrungen folgen, von Anfang an noch konsequenter modular zu testen: zuerst den reinen Arduino mit GRBL, dann das Shield ohne Treiber, dann einzelne Treiber und Motoren, danach Endschalter, danach alle Achsen, danach PWM und erst zuletzt der Heizdraht. Außerdem sollten Schaltplan, Pinbelegung, Jumperstellungen und GRBL-Parameter während der Inbetriebnahme fortlaufend dokumentiert werden. Gerade bei mehreren Umbauten, Ersatzteilen und Fehlersuchen ist eine solche Dokumentation entscheidend, um später nachvollziehen zu können, welcher Zustand tatsächlich funktioniert hat.

Zusammenfassend war die Inbetriebnahme anspruchsvoller und zeitintensiver als erwartet, aber auch der lehrreichste Teil des Projekts. Die aufgetretenen Probleme waren nicht nur Hindernisse, sondern führten zu einem wesentlich besseren Verständnis der Maschine. Besonders der Umgang mit GRBL, UGS, Achskalibrierung, Endschaltern,

Treibern und PWM-Heißdrahtsteuerung war für das Projekt zentral. Die gewonnenen Erfahrungen stellen daher einen wesentlichen Bestandteil des Projektergebnisses dar. Neben der fertigen Maschine liefert insbesondere die Dokumentation der aufgetretenen Probleme, Fehlannahmen und Lösungswege wertvolle Erkenntnisse für zukünftige Projekte.

Teil VI.

Anhang

22. Bill of Material

22.1. Mechanische Komponenten

Tabelle 22.1.: Mechanische Komponenten

Bild	Menge	Beschreibung	Lieferant	Preis (€)
	1	Creality Ender 5 Plus	creality3dofficial.de	423,24 (492,00 USD)
	2 (noch benötigt: 2 [aus Hochschule/Werkstatt])	64761000 Präz.-Wellenstahl CR Ø 10h7 x 1000mm	conrad.de	23,91
	1 (noch benötigt: 1)	Donau Elektronik Schneidematte 600 x 450 x 3 mm Art.-Nr.:2335839 - 62	Conrad.de	17,88
	1 (noch benötigt: 1)	Gewindeöse R 88136 Typ 48 M3x10 D5 Art.-Nr.:806545461 - 62	Conrad.de	3,06
	4 (noch benötigt: 4)	Alu-Pressklemmen Z2 für Drahtseil Ø 2,0 mm, Art.-Nr.: 848585038-62	Conrad.de	14,00
	1 (noch benötigt: 1)	Thomsen Widerstandsdraht Widerstand/m 5.65 Ω/m 10 m	Conrad.de	5,45
	1 (noch benötigt: 1)	BambuLab A02-G2-1.75-1000-spl PLA Metal Filament PLA 1.75 mm 1000 g Grün (metallic) 1 St.	Conrad.de	23,52
	1 (noch benötigt: 1)	ruthex Gewindeeinsatz M2 + M3 + M4 + M5 GE-KASTEN-001	Conrad.de	26,04

Tabelle 22.2.: Mechanische Komponenten

Bild	Menge	Beschreibung	Lieferant	Preis (€)
	1 (noch benötigt: 1)	Pösamo Drahtseil rostfrei 2 mm, 10 m, Art.-Nr.: 813549436-62	Conrad.de	6,30
	1 (noch benötigt: 1)	TOOLCRAFT 1062685 Gewindestift M3 16 mm Edelstahl A1, A2 50 St.	Conrad.de	9,24
	4 (noch benötigt: 4)	64601006 Linearkugellager KB-3-ST Ø 10	Conrad.de	4,46
	2 (noch benötigt: 2)	igus E-Kette® E2 micro Serie 06 06.10.018.0 Energieführungskette für kleinste Biegeradien	Conrad.de	21,00
	1 (noch benötigt: 1)	PE HD (PE 300) Schwarz, 5 mm, Grundplatte	Plattenshop24.com	8,03

22.2. Elektronische / Elektrische Komponenten

Tabelle 22.3.: Elektronische Komponenten

Bild	Menge	Beschreibung	Lieferant	Preis (€)
	1	Arduino Uno Rev3 SMD, Art.-Nr.: ARDUINO UNO	reichelt.de	16,95
	1 (Treiber vorhanden?)	CNC Controller Shield mit 4x A4988 Treibern, Art.-Nr.: ARD SHD CNC KIT	reichelt.de	14,95
	1 (noch benötigt: 1)	DC-DC Wandler LM2596, Art.-Nr.: DEBO DCDC DOWN 1	reichelt.de	1,95
	1	40-Pin Jumper-Kabel Buchse-Stecker	roboter-bausatz.de	2,95
	2 (noch benötigt: 6)	Creality Endschalter, Art.-Nr.: CRE-4004180004	3djake.de	4,99
	1 (noch benötigt: 1)	OLED Display 0,96", SSD1306, Art.-Nr.: DEBOOLED20.96	reichelt.de	6,99
	1 (noch benötigt: 1)	2,50MOSFET Modul IRF9540N, Art.-Nr.: DEBOCOMMOSFET	reichelt.de	5,29

Literaturverzeichnis

- [3dj] *Creality Endschalter*. 2026. URL: <https://www.3djake.de/creality-3d-drucker-ersatzteile/endschalter> (besucht am 26.04.2026).
- [Aaa] *Adafruit GFX Library*. [pdf]. 2024.
- [Aab] *Adafruit SSD1306 Library*. [pdf]. 2024.
- [All22] Allegro MicroSystems. *A4988 DMOS Microstepping Driver with Translator and Overcurrent Protection*. 4988-DS Rev. 8. Datasheet. Allegro MicroSystems. 2022. URL: <https://www.allegromicro.com>.
- [All24] M. Alles. *Einführung in die elektronische Schaltungstechnik*. Springer Vieweg, 2024. ISBN: 978-3-658-45570-5.
- [Arda] *Digital Pins - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/variablen/constants/inputOutputPullup/> (besucht am 25.04.2026).
- [Ardb] *Serial.begin() - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/funktionen/communication/serial/begin/> (besucht am 10.05.2026).
- [Ardc] *Serial.print() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/communication/serial/print/> (besucht am 25.04.2026).
- [Ardd] *delay() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/time/delay/> (besucht am 25.04.2026).
- [Arde] *digitalRead() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/digitalread/> (besucht am 25.04.2026).
- [Ardf] *digitalWrite() - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/funktionen/digital-io/digitalwrite/> (besucht am 10.05.2026).
- [Ardg] *loop() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/loop/> (besucht am 25.04.2026).
- [Ardh] *millis() - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/funktionen/time/millis/> (besucht am 10.05.2026).
- [Ardi] *pinMode() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/pinmode/> (besucht am 25.04.2026).
- [Ardj] *setup() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/setup/> (besucht am 25.04.2026).
- [Ard24] Arduino Documentation. *The Arduino Web Editor*. 2024. URL: <https://docs.arduino.cc/learn/starting-guide/the-arduino-web-editor/>.

- [Ard25a] Arduino. *Arduino Cloud*. 2025. URL: <https://docs.arduino.cc/cloud/iot-cloud> (besucht am 14.05.2026).
- [Ard25b] Arduino. *Libraries*. 2025. URL: <https://docs.arduino.cc/libraries/> (besucht am 14.05.2026).
- [Ard25c] Arduino. *Overview of the Arduino UNO Components*. 2025. URL: <https://docs.arduino.cc/tutorials/uno-rev3/intro-to-board/> (besucht am 12.05.2026).
- [Ard26a] Arduino. *Arduino Language Reference*. 2026. URL: <https://docs.arduino.cc/language-reference/> (besucht am 12.05.2026).
- [Ard26b] Arduino. *Arduino UNO R3 Datasheet*. 2026. URL: <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf> (besucht am 12.05.2026).
- [Ard26c] Arduino. *UNO R3*. 2026. URL: <https://docs.arduino.cc/hardware/uno-rev3/> (besucht am 12.05.2026).
- [Ard26d] Arduino. *analogRead()*. 2026. URL: <https://docs.arduino.cc/language-reference/en/functions/analog-io/analogread/> (besucht am 12.05.2026).
- [Ard26e] Arduino. *analogWrite()*. 2026. URL: <https://docs.arduino.cc/language-reference/en/functions/analog-io/analogwrite/> (besucht am 12.05.2026).
- [Bab23] G. Babel. *Elektrische Antriebe in der Fahrzeugtechnik: Lehr- und Arbeitsbuch*. 5. Wiesbaden und Heidelberg: Springer Vieweg, 2023. ISBN: 978-3-658-40585-4. DOI: [10.1007/978-3-658-40586-1](https://doi.org/10.1007/978-3-658-40586-1).
- [Bas] *Creativity Endschanter*. 2026. URL: <https://www.bastelgarage.ch/creativity-endschanter-limit-switch> (besucht am 26.04.2026).
- [Ber18] H. Bernstein. *Elektrotechnik/Elektronik für Maschinenbauer - Einfach und praxisgerecht*. 3. Aufl. Berlin Heidelberg New York: Springer-Verlag, 2018, S. 235–236. ISBN: 978-3-658-20838-7.
- [Ber24] H. Bernstein. *Messelektronik und Sensoren. Grundlagen der Messtechnik, Sensoren, analoge und digitale Signalverarbeitung*. 2. Aufl. Springer Vieweg Wiesbaden, 2024. ISBN: 978-3-658-38929-1. DOI: <https://doi.org/10.1007/978-3-658-38929-1>.
- [Bon25] M. Bonnet. *Kunststofftechnik. Grundlagen, Verarbeitung, Werkstoffauswahl und Fallbeispiele*. 4. Aufl. Wiesbaden: Springer Vieweg, 2025. ISBN: 978-3-658-45674-0. DOI: [10.1007/978-3-658-45674-0](https://doi.org/10.1007/978-3-658-45674-0).
- [DDFP24] R. W. De Doncker, N. Fritz und D. Pham. “Leistungselektronik”. In: *Elektromobilität*. Hrsg. von A. Kampker und H. H. Heimes. Berlin, Heidelberg: Springer Vieweg, 2024, S. 187–202. DOI: [10.1007/978-3-662-65812-3_10](https://doi.org/10.1007/978-3-662-65812-3_10).
- [Dr.20] Dr. Fritz Faulhaber GmbH & Co. KG, Hrsg. *Faulhaber Tutorial: Schritverluste verhindern bei Schrittmotoren*. [pdf]. Schönaich Deutschland, 2020. URL: www.faulhaber.com/de/know-how/tutorials/schrittmotoren-tutorial-schritverluste-verhindern-bei-schrittmotoren/.
- [FAD18] M. Fezari und A. Al Dahoud. “Integrated Development Environment “IDE” For Arduino”. In: (Okt. 2018).
- [FS22] A. H. Fritz und J. Schmütz, Hrsg. *Fertigungstechnik*. 13. Aufl. Berlin, Heidelberg: Springer Vieweg, 2022. ISBN: 978-3-662-64875-9. DOI: [10.1007/978-3-662-64875-9](https://doi.org/10.1007/978-3-662-64875-9).

- [Gloa] Global Electric Motor Solution LLC, Hrsg. *NEMA 17, 1.8° 42mm GM42BYG Stepper Motors - Zeichnung*. [pdf].
- [Glob] Global Electric Motor Solution LLC, Hrsg. *NEMA 17, 1.8° 42mm GM42BYG Stepper Motors*. [pdf]. URL: gemsmotor.com/stepper/nema17-stepper-motor.pdf.
- [Hag21] R. Hagl. *Elektrische Antriebstechnik*. 3., überarbeitete und erweiterte Auflage. München: Hanser, 2021. ISBN: 978-3-446-46572-5. DOI: [10.3139/9783446468214](https://doi.org/10.3139/9783446468214).
- [Heh20] P. Hehenberger. *Computerunterstützte Produktion. Eine kompakte Einführung*. 2. Aufl. Springer Vieweg, 2020. ISBN: 978-3-662-60875-3. DOI: <https://doi.org/10.1007/978-3-662-60876-0>.
- [Hel21] W. Helbig. *Praxiswissen in der Messtechnik. Arbeitsbuch für Techniker, Ingenieure und Studenten*. 1. Aufl. Bd. 1. Springer Vieweg Wiesbaden, 2021. ISBN: 978-3-658-27802-1. DOI: <https://doi.org/10.1007/978-3-658-27802-1>.
- [Her+12] E. Hering u. a. *Elektrotechnik und Elektronik für Maschinenbauer*. Bd. 2. Springer, 14. Feb. 2012. ISBN: 978-3-642-12881-3.
- [Ise08] R. Isermann. *Mechatronische Systeme. Grundlagen*. 2. Aufl. Springer, 2008. ISBN: 978-3-540-32336-5. DOI: <https://doi.org/10.1007/978-3-540-32512-3>.
- [Jc19] S. K. Jeon und contributors. *GRBL – An Open Source Embedded G-Code Parser and CNC Milling Controller*. <https://github.com/gnea/grbl>. Firmware, Version v1.1h; zuletzt aufgerufen am 22.06.2026. 2019.
- [Joya] *ARCADE-BUTTONS MIT MIKROSCHALTER*. 2024.
- [Joyb] *MOSFET PWM-Schaltmodul Datenblatt*. 2025. URL: <https://joy-it.net> (besucht am 20.05.2026).
- [Joy22] Joy-IT / SIMAC Electronics GmbH. *ARD-CNC-KIT1 Datenblatt*. Datenblatt, veröffentlicht am 07.09.2022. SIMAC Electronics GmbH. 2022.
- [Joy23] Joy-IT / SIMAC Electronics GmbH. *ARD-CNC-KIT1 Anleitung*. Anleitung, veröffentlicht am 28.11.2023. SIMAC Electronics GmbH. 2023.
- [McC24a] M. McCauley. *AccelStepper Library - Classes*. [pdf]. 2024.
- [McC24b] M. McCauley. *AccelStepper Library*. [pdf]. 2024.
- [McC24c] M. McComb. *Micro-stepping for Stepper Motors*. Hrsg. von Microchip Technology. [pdf]. 2024.
- [Mp2] *MP2307 LM2596 Kis-3r33S DC-DC Converter Buck Module*. Technisches Datenblatt. Eingangsspannung 4.75–24V, Ausgangsspannung 0.93–18V, Wirkungsgrad bis 98%.
- [Rs2] *Drucktaster von Eaton: Ein Leitfaden*. 2026. URL: <https://de.rs-online.com/web/content/discovery-portal/produktatgeber/drucktaster-leitfaden> (besucht am 09.05.2026).
- [SK21] D. Schröder und R. Kennel. *Elektrische Antriebe – Grundlagen*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021. ISBN: 978-3-662-63100-3. DOI: [10.1007/978-3-662-63101-0](https://doi.org/10.1007/978-3-662-63101-0).
- [Sai25] SainSmart Resource Center. *Grbl V1.1 Quick Reference*. <https://www.sainsmart.com/blogs/news/grbl-v1-1-quick-reference>. Zuletzt aufgerufen am 22.06.2026. 2025.
- [Sch17] D. Schröder. “Kleinantriebe”. In: *Elektrische Antriebe – Grundlagen*. Berlin, Heidelberg: Springer Vieweg, 2017. DOI: [10.1007/978-3-662-55448-7_9](https://doi.org/10.1007/978-3-662-55448-7_9).

-
- [Sch20] U. Schlienz. *Schaltnetzteile und ihre Peripherie. Dimensionierung, Einsatz, EMV*. 7. Aufl. Wiesbaden: Springer Vieweg, 2020. ISBN: 978-3-658-29489-2. DOI: <https://doi.org/10.1007/978-3-658-29490-8>.
- [Smi11] A. G. Smith. *Introduction to Arduino: A Piece of Cake!* Self-published, 2011. ISBN: 978-1463698348.
- [Sto24] P. Stoffregen. *Encoder Library*. Hrsg. von PJRC. [pdf]. 2024. URL: https://www.pjrc.com/teensy/td_libs_Encoder.html.
- [TR14] H.-R. Tränkler und L. Reindl, Hrsg. *Sensortechnik: Handbuch für Praxis und Wissenschaft*. 2. Aufl. VDI-Buch. Published: 13 January 2015. eBook Packages: Computer Science and Engineering (German Language). Berlin, Heidelberg: Springer Vieweg Berlin, Heidelberg, 2014, S. XIV, 1596. ISBN: 978-3-642-29942-1. DOI: 10.1007/978-3-642-29942-1.
- [TSG23] U. Tietze, C. Schenk und E. Gamm. *Halbleiter-Schaltungstechnik*. Springer Vieweg, 2023. ISBN: 978-3-662-68591-4.
- [WU25] Winder und Universal G-Code Sender Contributors. *Universal G-Code Sender*. <https://github.com/winder/Universal-G-Code-Sender>. Open-Source-Software; zuletzt aufgerufen am 22.06.2026. 2025.
- [Zac22] F. Zach. *Leistungselektronik. Ein Handbuch Band 1 / Band 2*. 6. Aufl. Springer Vieweg, 2022. ISBN: 978-3-658-31435-4. DOI: <https://doi.org/10.1007/978-3-658-31436-1>.
- [Zie25] D. Zielke. *Mikrosysteme*. Bd. 1. Springer, 5. März 2025. ISBN: 978-3-662-71232-0.